DATA MINING COURSE READINGS

Chapter 06: Cluster Analysis & Anomaly Detection

Curated Textbook Readings from Tan et al. · Matching Lecture: IE500_DM_06_Clustering.pdf

REFERENCE TEXTBOOK

Introduction to Data Mining (2nd Edition, Global Edition)

Authors: Pang-Ning Tan, Michael Steinbach, Anuj Karpatne, Vipin Kumar | Pearson, 2019

Curated by: Ehsan Shahbazi | Shahid Beheshti University (SBU)

INCLUDED CHAPTER SECTIONS & TOPIC SCOPE

- **Sections 5.1·5.4: Clustering Algorithms** — Book p. 311·353
  K-Means, SSE objective, initial centroids, Agglomerative Hierarchical, DBSCAN
- **Section 5.5: Cluster Evaluation** — Book p. 354·379
  Silhouette coefficient, unsupervised cluster cohesion and separation, clustering tendency
- **Section 2.4.1, 2.4.5: Proximity Measures** — Book p. 93·95, 100·108
  Minkowski norms (L1 Manhattan, L2 Euclidean), distance metric properties
- **Sections 9.1·9.5: Anomaly Detection** — Book p. 724·748
  Anomaly definitions, statistical approaches (IQR, MAD), LOF proximity, cluster-based outliers

STUDENT STUDY GUIDE & SCOPE

Comprehensive coverage of unsupervised learning: partitioning (K-Means), density-based (DBSCAN), hierarchical clustering, cluster validity (Silhouette score), and modern anomaly detection (LOF, statistical, and cluster-based).

5

Cluster Analysis: Basic Concepts and Algorithms

Cluster analysis divides data into groups (clusters) that are meaningful, useful, or both. If meaningful groups are the goal, then the clusters should capture the natural structure of the data. In some cases, however, cluster analysis is used for data summarization in order to reduce the size of the data. Whether for understanding or utility, cluster analysis has long played an important role in a wide variety of fields: psychology and other social sciences, biology, statistics, pattern recognition, information retrieval, machine learning, and data mining.

There have been many applications of cluster analysis to practical problems. We provide some specific examples, organized by whether the purpose of the clustering is understanding or utility.

Clustering for Understanding Classes, or conceptually meaningful groups of objects that share common characteristics, play an important role in how people analyze and describe the world. Indeed, human beings are skilled at dividing objects into groups (clustering) and assigning particular objects to these groups (classification). For example, even relatively young children can quickly label the objects in a photograph. In the context of understanding data, clusters are potential classes and cluster analysis is the study of techniques for automatically finding classes. The following are some examples:

- **Biology.** Biologists have spent many years creating a taxonomy (hierarchical classification) of all living things: kingdom, phylum, class, order, family, genus, and species. Thus, it is perhaps not surprising that

much of the early work in cluster analysis sought to create a discipline of mathematical taxonomy that could automatically find such classification structures. More recently, biologists have applied clustering to analyze the large amounts of genetic information that are now available. For example, clustering has been used to find groups of genes that have similar functions.

- **Information Retrieval.** The World Wide Web consists of billions of web pages, and the results of a query to a search engine can return thousands of pages. Clustering can be used to group these search results into a small number of clusters, each of which captures a particular aspect of the query. For instance, a query of "movie" might return web pages grouped into categories such as reviews, trailers, stars, and theaters. Each category (cluster) can be broken into subcategories (subclusters), producing a hierarchical structure that further assists a user's exploration of the query results.

- **Climate.** Understanding the Earth's climate requires finding patterns in the atmosphere and ocean. To that end, cluster analysis has been applied to find patterns in atmospheric pressure and ocean temperature that have a significant impact on climate.

- **Psychology and Medicine.** An illness or condition frequently has a number of variations, and cluster analysis can be used to identify these different subcategories. For example, clustering has been used to identify different types of depression. Cluster analysis can also be used to detect patterns in the spatial or temporal distribution of a disease.

- **Business.** Businesses collect large amounts of information about current and potential customers. Clustering can be used to segment customers into a small number of groups for additional analysis and marketing activities.

Clustering for Utility Cluster analysis provides an abstraction from individual data objects to the clusters in which those data objects reside. Additionally, some clustering techniques characterize each cluster in terms of a cluster prototype; i.e., a data object that is representative of the objects in the cluster. These cluster prototypes can be used as the basis for a number of additional data analysis or data processing techniques. Therefore, in the context of utility, cluster analysis is the study of techniques for finding the most representative cluster prototypes.

- **Summarization.** Many data analysis techniques, such as regression or principal component analysis, have a time or space complexity of $O(m^2)$ or higher (where m is the number of objects), and thus, are not practical for large data sets. However, instead of applying the algorithm to the entire data set, it can be applied to a reduced data set consisting only of cluster prototypes. Depending on the type of analysis, the number of prototypes, and the accuracy with which the prototypes represent the data, the results can be comparable to those that would have been obtained if all the data could have been used.

- **Compression.** Cluster prototypes can also be used for data compression. In particular, a table is created that consists of the prototypes for each cluster; i.e., each prototype is assigned an integer value that is its position (index) in the table. Each object is represented by the index of the prototype associated with its cluster. This type of compression is known as **vector quantization** and is often applied to image, sound, and video data, where (1) many of the data objects are highly similar to one another, (2) some loss of information is acceptable, and (3) a substantial reduction in the data size is desired.

- **Efficiently Finding Nearest Neighbors.** Finding nearest neighbors can require computing the pairwise distance between all points. Often clusters and their cluster prototypes can be found much more efficiently. If objects are relatively close to the prototype of their cluster, then we can use the prototypes to reduce the number of distance computations that are necessary to find the nearest neighbors of an object. Intuitively, if two cluster prototypes are far apart, then the objects in the corresponding clusters cannot be nearest neighbors of each other. Consequently, to find an object's nearest neighbors, it is necessary to compute only the distance to objects in nearby clusters, where the nearness of two clusters is measured by the distance between their prototypes. This idea is made more precise in Exercise 30 of Chapter 2, which is on pages 131–132.

This chapter provides an introduction to cluster analysis. We begin with a high-level overview of clustering, including a discussion of the various approaches to dividing objects into sets of clusters and the different types of clusters. We then describe three specific clustering techniques that represent broad categories of algorithms and illustrate a variety of concepts: K-means, agglomerative hierarchical clustering, and DBSCAN. The final section of this chapter is devoted to cluster validity—methods for evaluating the goodness

of the clusters produced by a clustering algorithm. More advanced clustering concepts and algorithms will be discussed in Chapter 8. Whenever possible, we discuss the strengths and weaknesses of different schemes. In addition, the Bibliographic Notes provide references to relevant books and papers that explore cluster analysis in greater depth.

5.1 Overview

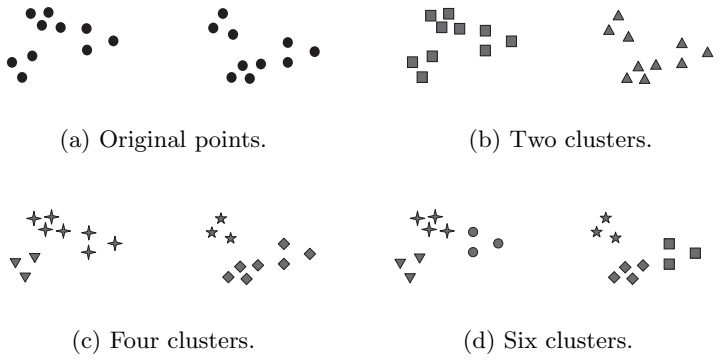
Before discussing specific clustering techniques, we provide some necessary background. First, we further define cluster analysis, illustrating why it is difficult and explaining its relationship to other techniques that group data. Then we explore two important topics: (1) different ways to group a set of objects into a set of clusters, and (2) types of clusters.

5.1.1 What Is Cluster Analysis?

Cluster analysis groups data objects based on information found only in the data that describes the objects and their relationships. The goal is that the objects within a group be similar (or related) to one another and different from (or unrelated to) the objects in other groups. The greater the similarity (or homogeneity) within a group and the greater the difference between groups, the better or more distinct the clustering.

In many applications, the notion of a cluster is not well defined. To better understand the difficulty of deciding what constitutes a cluster, consider Figure 5.1, which shows 20 points and three different ways of dividing them into clusters. The shapes of the markers indicate cluster membership. Figures 5.1(b) and 5.1(d) divide the data into two and six parts, respectively. However, the apparent division of each of the two larger clusters into three subclusters may simply be an artifact of the human visual system. Also, it may not be unreasonable to say that the points form four clusters, as shown in Figure 5.1(c). This figure illustrates that the definition of a cluster is imprecise and that the best definition depends on the nature of data and the desired results.

Cluster analysis is related to other techniques that are used to divide data objects into groups. For instance, clustering can be regarded as a form of classification in that it creates a labeling of objects with class (cluster) labels. However, it derives these labels only from the data. In contrast, classification in the sense of Chapter 3 is **supervised classification**; i.e., new, unlabeled objects are assigned a class label using a model developed from objects with known class labels. For this reason, cluster analysis is sometimes referred to as **unsupervised classification**. When the term classification is used



(a) Original points. (b) Two clusters.

(c) Four clusters. (d) Six clusters.

Figure 5.1. Three different ways of clustering the same set of points.

without any qualification within data mining, it typically refers to supervised classification.

Also, while the terms **segmentation** and **partitioning** are sometimes used as synonyms for clustering, these terms are frequently used for approaches outside the traditional bounds of cluster analysis. For example, the term partitioning is often used in connection with techniques that divide graphs into subgraphs and that are not strongly connected to clustering. Segmentation often refers to the division of data into groups using simple techniques; e.g., an image can be split into segments based only on pixel intensity and color, or people can be divided into groups based on their income. Nonetheless, some work in graph partitioning and in image and segmentation is related to cluster analysis.

5.1.2 Different Types of Clusterings

An entire collection of clusters is commonly referred to as a **clustering**, and in this section, we distinguish various types of clusterings: hierarchical (nested) versus partitional (unnested), exclusive versus overlapping versus fuzzy, and complete versus partial.

Hierarchical versus Partitional The most commonly discussed distinction among different types of clusterings is whether the set of clusters is nested or unnested, or in more traditional terminology, hierarchical or partitional. A **partitional clustering** is simply a division of the set of data objects into non-overlapping subsets (clusters) such that each data object is in exactly one

subset. Taken individually, each collection of clusters in Figures 5.1 (b–d) is a partitional clustering.

If we permit clusters to have subclusters, then we obtain a **hierarchical clustering**, which is a set of nested clusters that are organized as a tree. Each node (cluster) in the tree (except for the leaf nodes) is the union of its children (subclusters), and the root of the tree is the cluster containing all the objects. Often, but not always, the leaves of the tree are singleton clusters of individual data objects. If we allow clusters to be nested, then one interpretation of Figure 5.1(a) is that it has two subclusters (Figure 5.1(b)), each of which, in turn, has three subclusters (Figure 5.1(d)). The clusters shown in Figures 5.1 (a–d), when taken in that order, also form a hierarchical (nested) clustering with, respectively, 1, 2, 4, and 6 clusters on each level. Finally, note that a hierarchical clustering can be viewed as a sequence of partitional clusterings and a partitional clustering can be obtained by taking any member of that sequence; i.e., by cutting the hierarchical tree at a particular level.

Exclusive versus Overlapping versus Fuzzy The clusterings shown in Figure 5.1 are all **exclusive**, as they assign each object to a single cluster. There are many situations in which a point could reasonably be placed in more than one cluster, and these situations are better addressed by non-exclusive clustering. In the most general sense, an **overlapping** or **non-exclusive clustering** is used to reflect the fact that an object can *simultaneously* belong to more than one group (class). For instance, a person at a university can be both an enrolled student and an employee of the university. A non-exclusive clustering is also often used when, for example, an object is "between" two or more clusters and could reasonably be assigned to any of these clusters. Imagine a point halfway between two of the clusters of Figure 5.1. Rather than make a somewhat arbitrary assignment of the object to a single cluster, it is placed in all of the "equally good" clusters.

In a **fuzzy clustering** (Section 8.2.1), every object belongs to every cluster with a membership weight that is between 0 (absolutely doesn't belong) and 1 (absolutely belongs). In other words, clusters are treated as fuzzy sets. (Mathematically, a fuzzy set is one in which an object belongs to every set with a weight that is between 0 and 1. In fuzzy clustering, we often impose the additional constraint that the sum of the weights for each object must equal 1.) Similarly, probabilistic clustering techniques (Section 8.2.2) compute the probability with which each point belongs to each cluster, and these probabilities must also sum to 1. Because the membership weights or probabilities for any object sum to 1, a fuzzy or probabilistic clustering does

not address true multiclass situations, such as the case of a student employee, where an object belongs to multiple classes. Instead, these approaches are most appropriate for avoiding the arbitrariness of assigning an object to only one cluster when it is close to several. In practice, a fuzzy or probabilistic clustering is often converted to an exclusive clustering by assigning each object to the cluster in which its membership weight or probability is highest.

Complete versus Partial A **complete clustering** assigns every object to a cluster, whereas a **partial clustering** does not. The motivation for a partial clustering is that some objects in a data set may not belong to well-defined groups. Many times objects in the data set represent noise, outliers, or "uninteresting background." For example, some newspaper stories share a common theme, such as global warming, while other stories are more generic or one-of-a-kind. Thus, to find the important topics in last month's stories, we often want to search only for clusters of documents that are tightly related by a common theme. In other cases, a complete clustering of the objects is desired. For example, an application that uses clustering to organize documents for browsing needs to guarantee that all documents can be browsed.

5.1.3 Different Types of Clusters

Clustering aims to find useful groups of objects (clusters), where usefulness is defined by the goals of the data analysis. Not surprisingly, several different notions of a cluster prove useful in practice. In order to visually illustrate the differences among these types of clusters, we use two-dimensional points, as shown in Figure 5.2, as our data objects. We stress, however, that the types of clusters described here are equally valid for other kinds of data.

Well-Separated A cluster is a set of objects in which each object is closer (or more similar) to every other object in the cluster than to any object not in the cluster. Sometimes a threshold is used to specify that all the objects in a cluster must be sufficiently close (or similar) to one another. This idealistic definition of a cluster is satisfied only when the data contains natural clusters that are quite far from each other. Figure 5.2(a) gives an example of well-separated clusters that consists of two groups of points in a two-dimensional space. The distance between any two points in different groups is larger than the distance between any two points within a group. Well-separated clusters do not need to be globular, but can have any shape.

Prototype-Based A cluster is a set of objects in which each object is closer (more similar) to the prototype that defines the cluster than to the prototype of any other cluster. For data with continuous attributes, the prototype of a cluster is often a centroid, i.e., the average (mean) of all the points in the cluster. When a centroid is not meaningful, such as when the data has categorical attributes, the prototype is often a medoid, i.e., the most representative point of a cluster. For many types of data, the prototype can be regarded as the most central point, and in such instances, we commonly refer to prototype-based clusters as **center-based clusters**. Not surprisingly, such clusters tend to be globular. Figure 5.2(b) shows an example of center-based clusters.

Graph-Based If the data is represented as a graph, where the nodes are objects and the links represent connections among objects (see Section 2.1.2), then a cluster can be defined as a **connected component**; i.e., a group of objects that are connected to one another, but that have no connection to objects outside the group. An important example of graph-based clusters is a **contiguity-based cluster**, where two objects are connected only if they are within a specified distance of each other. This implies that each object in a contiguity-based cluster is closer to some other object in the cluster than to any point in a different cluster. Figure 5.2(c) shows an example of such clusters for two-dimensional points. This definition of a cluster is useful when clusters are irregular or intertwined. However, this approach can have trouble when noise is present since, as illustrated by the two spherical clusters of Figure 5.2(c), a small bridge of points can merge two distinct clusters.

Other types of graph-based clusters are also possible. One such approach (Section 5.3.2) defines a cluster as a **clique**; i.e., a set of nodes in a graph that are completely connected to each other. Specifically, if we add connections between objects in the order of their distance from one another, a cluster is formed when a set of objects forms a clique. Like prototype-based clusters, such clusters tend to be globular.

Density-Based A cluster is a dense region of objects that is surrounded by a region of low density. Figure 5.2(d) shows some density-based clusters for data created by adding noise to the data of Figure 5.2(c). The two circular clusters are not merged, as in Figure 5.2(c), because the bridge between them fades into the noise. Likewise, the curve that is present in Figure 5.2(c) also fades into the noise and does not form a cluster in Figure 5.2(d). A density-based definition of a cluster is often employed when the clusters are irregular or

intertwined, and when noise and outliers are present. By contrast, a contiguity-based definition of a cluster would not work well for the data of Figure 5.2(d) because the noise would tend to form bridges between clusters.

Shared-Property (Conceptual Clusters) More generally, we can define a cluster as a set of objects that share some property. This definition encompasses all the previous definitions of a cluster; e.g., objects in a center-based cluster share the property that they are all closest to the same centroid or medoid. However, the shared-property approach also includes new types of clusters. Consider the clusters shown in Figure 5.2(e). A triangular area (cluster) is adjacent to a rectangular one, and there are two intertwined circles (clusters). In both cases, a clustering algorithm would need a very specific concept of a cluster to successfully detect these clusters. The process of finding such clusters is called conceptual clustering. However, too sophisticated a notion of a cluster would take us into the area of pattern recognition, and thus, we only consider simpler types of clusters in this book.

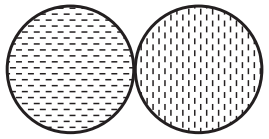
Road Map

In this chapter, we use the following three simple, but important techniques to introduce many of the concepts involved in cluster analysis.

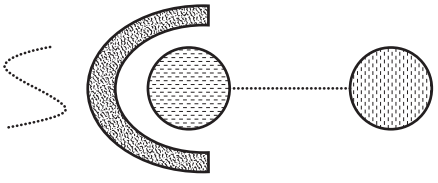
- **K-means**. This is a prototype-based, partitional clustering technique that attempts to find a user-specified number of clusters (K), which are represented by their centroids.
- **Agglomerative Hierarchical Clustering.** This clustering approach refers to a collection of closely related clustering techniques that produce a hierarchical clustering by starting with each point as a singleton cluster and then repeatedly merging the two closest clusters until a single, all-encompassing cluster remains. Some of these techniques have a natural interpretation in terms of graph-based clustering, while others have an interpretation in terms of a prototype-based approach.
- **DBSCAN**. This is a density-based clustering algorithm that produces a partitional clustering, in which the number of clusters is automatically determined by the algorithm. Points in low-density regions are classified as noise and omitted; thus, DBSCAN does not produce a complete clustering.



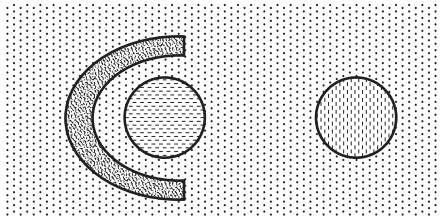
(a) Well-separated clusters. Each point is closer to all of the points in its cluster than to any point in another cluster.



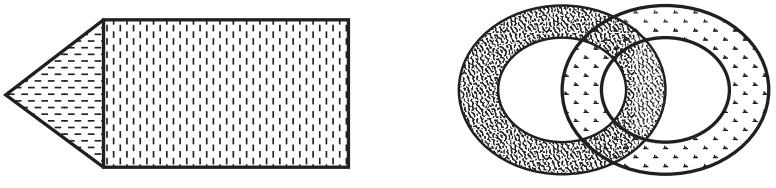
(b) Center-based clusters. Each point is closer to the center of its cluster than to the center of any other cluster.



(c) Contiguity-based clusters. Each point is closer to at least one point in its cluster than to any point in another cluster.



(d) Density-based clusters. Clusters are regions of high density separated by regions of low density.



(e) Conceptual clusters. Points in a cluster share some general property that derives from the entire set of points. (Points in the intersection of the circles belong to both.)

Figure 5.2. Different types of clusters as illustrated by sets of two-dimensional points.

5.2 K-means

Prototype-based clustering techniques create a one-level partitioning of the data objects. There are a number of such techniques, but two of the most prominent are K-means and K-medoid. K-means defines a prototype in terms of a centroid, which is usually the mean of a group of points, and is typically

applied to objects in a continuous n-dimensional space. K-medoid defines a prototype in terms of a medoid, which is the most representative point for a group of points, and can be applied to a wide range of data since it requires only a proximity measure for a pair of objects. While a centroid almost never corresponds to an actual data point, a medoid, by its definition, must be an actual data point. In this section, we will focus solely on K-means, which is one of the oldest and most widely-used clustering algorithms.

5.2.1 The Basic K-means Algorithm

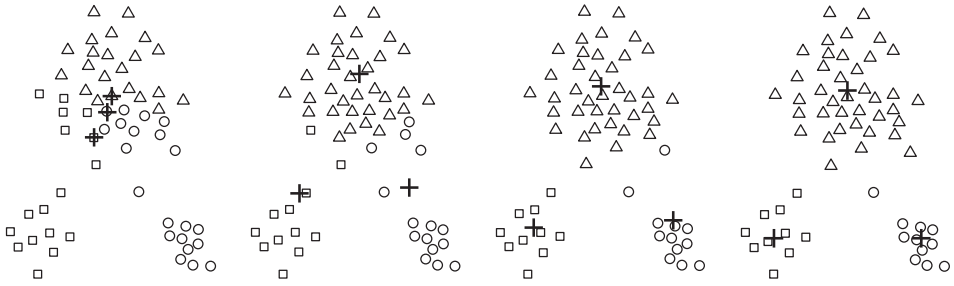
The K-means clustering technique is simple, and we begin with a description of the basic algorithm. We first choose K initial centroids, where K is a user-specified parameter, namely, the number of clusters desired. Each point is then assigned to the closest centroid, and each collection of points assigned to a centroid is a cluster. The centroid of each cluster is then updated based on the points assigned to the cluster. We repeat the assignment and update steps until no point changes clusters, or equivalently, until the centroids remain the same.

K-means is formally described by Algorithm 5.1. The operation of K-means is illustrated in Figure 5.3, which shows how, starting from three centroids, the final clusters are found in four assignment-update steps. In these and other figures displaying K-means clustering, each subfigure shows (1) the centroids at the start of the iteration and (2) the assignment of the points to those centroids. The centroids are indicated by the "+" symbol; all points belonging to the same cluster have the same marker shape.

Algorithm 5.1 Basic K-means algorithm.

1: Select K points as initial centroids.
2: **repeat**
3: Form K clusters by assigning each point to its closest centroid.
4: Recompute the centroid of each cluster.
5: **until** Centroids do not change.

In the first step, shown in Figure 5.3(a), points are assigned to the initial centroids, which are all in the larger group of points. For this example, we use the mean as the centroid. After points are assigned to a centroid, the centroid is then updated. Again, the figure for each step shows the centroid at the beginning of the step and the assignment of points to those centroids. In the second step, points are assigned to the updated centroids, and the centroids



(a) Iteration 1. (b) Iteration 2. (c) Iteration 3. (d) Iteration 4.

Figure 5.3. Using the K-means algorithm to find three clusters in sample data.

are updated again. In steps 2, 3, and 4, which are shown in Figures 5.3 (b), (c), and (d), respectively, two of the centroids move to the two small groups of points at the bottom of the figures. When the K-means algorithm terminates in Figure 5.3(d), because no more changes occur, the centroids have identified the natural groupings of points.

For a number of combinations of proximity functions and types of centroids, K-means always converges to a solution; i.e., K-means reaches a state in which no points are shifting from one cluster to another, and hence, the centroids don't change. Because most of the convergence occurs in the early steps, however, the condition on line 5 of Algorithm 5.1 is often replaced by a weaker condition, e.g., repeat until only 1% of the points change clusters.

We consider each of the steps in the basic K-means algorithm in more detail and then provide an analysis of the algorithm's space and time complexity.

Assigning Points to the Closest Centroid

To assign a point to the closest centroid, we need a proximity measure that quantifies the notion of "closest" for the specific data under consideration. Euclidean (L_2) distance is often used for data points in Euclidean space, while cosine similarity is more appropriate for documents. However, several types of proximity measures can be appropriate for a given type of data. For example, Manhattan (L_1) distance can be used for Euclidean data, while the Jaccard measure is often employed for documents.

Usually, the similarity measures used for K-means are relatively simple since the algorithm repeatedly calculates the similarity of each point to each centroid. In some cases, however, such as when the data is in low-dimensional

Table 5.1. Table of notation.

Symbol	Description
$\mathbf{x}$	An object.
C_i	The ith cluster.
$\mathbf{c}_i$	The centroid of cluster C_i.
$\mathbf{c}$	The centroid of all points.
m_i	The number of objects in the ith cluster.
m	The number of objects in the data set.
K	The number of clusters.

Euclidean space, it is possible to avoid computing many of the similarities, thus significantly speeding up the K-means algorithm. Bisecting K-means (described in Section 5.2.3) is another approach that speeds up K-means by reducing the number of similarities computed.

Centroids and Objective Functions

Step 4 of the K-means algorithm was stated rather generally as "recompute the centroid of each cluster," since the centroid can vary, depending on the proximity measure for the data and the goal of the clustering. The goal of the clustering is typically expressed by an objective function that depends on the proximities of the points to one another or to the cluster centroids; e.g., minimize the squared distance of each point to its closest centroid. We illustrate this with two examples. However, the key point is this: after we have specified a proximity measure and an objective function, the centroid that we should choose can often be determined mathematically. We provide mathematical details in Section 5.2.6, and provide a non-mathematical discussion of this observation here.

Data in Euclidean Space Consider data whose proximity measure is Euclidean distance. For our objective function, which measures the quality of a clustering, we use the **sum of the squared error (SSE)**, which is also known as scatter. In other words, we calculate the error of each data point, i.e., its Euclidean distance to the closest centroid, and then compute the total sum of the squared errors. Given two different sets of clusters that are produced by two different runs of K-means, we prefer the one with the smallest squared error since this means that the prototypes (centroids) of this clustering are a better representation of the points in their cluster. Using the notation in Table 5.1, the SSE is formally defined as follows:

$$\text{SSE} = \sum_{i=1}^{K} \sum_{\mathbf{x} \in C_i} dist(\mathbf{c}_i, \mathbf{x})^2 \tag{5.1}$$

where *dist* is the standard Euclidean (L_2) distance between two objects in Euclidean space.

Given these assumptions, it can be shown (see Section 5.2.6) that the centroid that minimizes the SSE of the cluster is the mean. Using the notation in Table 5.1, the centroid (mean) of the ith cluster is defined by Equation 5.2.

$$\mathbf{c}_i = \frac{1}{m_i} \sum_{\mathbf{x} \in C_i} \mathbf{x} \tag{5.2}$$

To illustrate, the centroid of a cluster containing the three two-dimensional points, (1,1), (2,3), and (6,2), is $((1 + 2 + 6)/3, ((1 + 3 + 2)/3) = (3, 2)$.

Steps 3 and 4 of the K-means algorithm directly attempt to minimize the SSE (or more generally, the objective function). Step 3 forms clusters by assigning points to their nearest centroid, which minimizes the SSE for the given set of centroids. Step 4 recomputes the centroids so as to further minimize the SSE. However, the actions of K-means in Steps 3 and 4 are guaranteed to only find a local minimum with respect to the SSE because they are based on optimizing the SSE for specific choices of the centroids and clusters, rather than for all possible choices. We will later see an example in which this leads to a suboptimal clustering.

Document Data To illustrate that K-means is not restricted to data in Euclidean space, we consider document data and the cosine similarity measure. Here we assume that the document data is represented as a document-term matrix as described on page 57. Our objective is to maximize the similarity of the documents in a cluster to the cluster centroid; this quantity is known as the **cohesion** of the cluster. For this objective it can be shown that the cluster centroid is, as for Euclidean data, the mean. The analogous quantity to the total SSE is the total cohesion, which is given by Equation 5.3.

$$\text{Total Cohesion} = \sum_{i=1}^{K} \sum_{\mathbf{x} \in C_i} cosine(\mathbf{x}, \mathbf{c}_i) \tag{5.3}$$

The General Case There are a number of choices for the proximity function, centroid, and objective function that can be used in the basic K-means

Table 5.2. K-means: Common choices for proximity, centroids, and objective functions.

Proximity Function	Centroid	Objective Function
Manhattan (L_1)	median	Minimize sum of the L_1 distance of an object to its cluster centroid
Squared Euclidean (L_2^2)	mean	Minimize sum of the squared L_2 distance of an object to its cluster centroid
cosine	mean	Maximize sum of the cosine similarity of an object to its cluster centroid
Bregman divergence	mean	Minimize sum of the Bregman divergence of an object to its cluster centroid

algorithm and that are guaranteed to converge. Table 5.2 shows some possible choices, including the two that we have just discussed. Notice that for Manhattan (L_1) distance and the objective of minimizing the sum of the distances, the appropriate centroid is the median of the points in a cluster.

The last entry in the table, Bregman divergence (Section 2.4.8), is actually a class of proximity measures that includes the squared Euclidean distance, L_2^2, the Mahalanobis distance, and cosine similarity. The importance of Bregman divergence functions is that any such function can be used as the basis of a K-means style clustering algorithm with the mean as the centroid. Specifically, if we use a Bregman divergence as our proximity function, then the resulting clustering algorithm has the usual properties of K-means with respect to convergence, local minima, etc. Furthermore, the properties of such a clustering algorithm can be developed for all possible Bregman divergences. For example, K-means algorithms that use cosine similarity or squared Euclidean distance are particular instances of a general clustering algorithm based on Bregman divergences.

For the rest of our K-means discussion, we use two-dimensional data since it is easy to explain K-means and its properties for this type of data. But, as suggested by the last few paragraphs, K-means is a general clustering algorithm and can be used with a wide variety of data types, such as documents and time series.

Choosing Initial Centroids

When random initialization of centroids is used, different runs of K-means typically produce different total SSEs. We illustrate this with the set of two-dimensional points shown in Figure 5.3, which has three natural clusters of points. Figure 5.4(a) shows a clustering solution that is the global minimum of



(a) Optimal clustering.

(b) Suboptimal clustering.

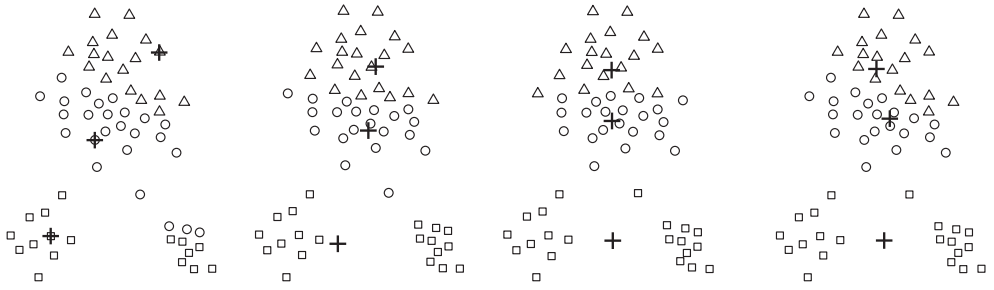
Figure 5.4. Three optimal and non-optimal clusters.

the SSE for three clusters, while Figure 5.4(b) shows a suboptimal clustering that is only a local minimum.

Choosing the proper initial centroids is the key step of the basic K-means procedure. A common approach is to choose the initial centroids randomly, but the resulting clusters are often poor.

Example 5.1 (Poor Initial Centroids)**.** Randomly selected initial centroids can be poor. We provide an example of this using the same data set used in Figures 5.3 and 5.4. Figures 5.3 and 5.5 show the clusters that result from two particular choices of initial centroids. (For both figures, the positions of the cluster centroids in the various iterations are indicated by crosses.) In Figure 5.3, even though all the initial centroids are from one natural cluster, the minimum SSE clustering is still found. In Figure 5.5, however, even though the initial centroids seem to be better distributed, we obtain a suboptimal clustering, with higher squared error. ■

Example 5.2 (Limits of Random Initialization)**.** One technique that is commonly used to address the problem of choosing initial centroids is to perform multiple runs, each with a different set of randomly chosen initial centroids, and then select the set of clusters with the minimum SSE. While simple, this strategy might not work very well, depending on the data set and the number of clusters sought. We demonstrate this using the sample data set shown in Figure 5.6(a). The data consists of two pairs of clusters, where the clusters in each (top-bottom) pair are closer to each other than to the clusters in the other pair. Figure 5.6 (b–d) shows that if we start with two initial centroids per pair of clusters, then even when both centroids are in a single cluster, the centroids



(a) Iteration 1. (b) Iteration 2. (c) Iteration 3. (d) Iteration 4.

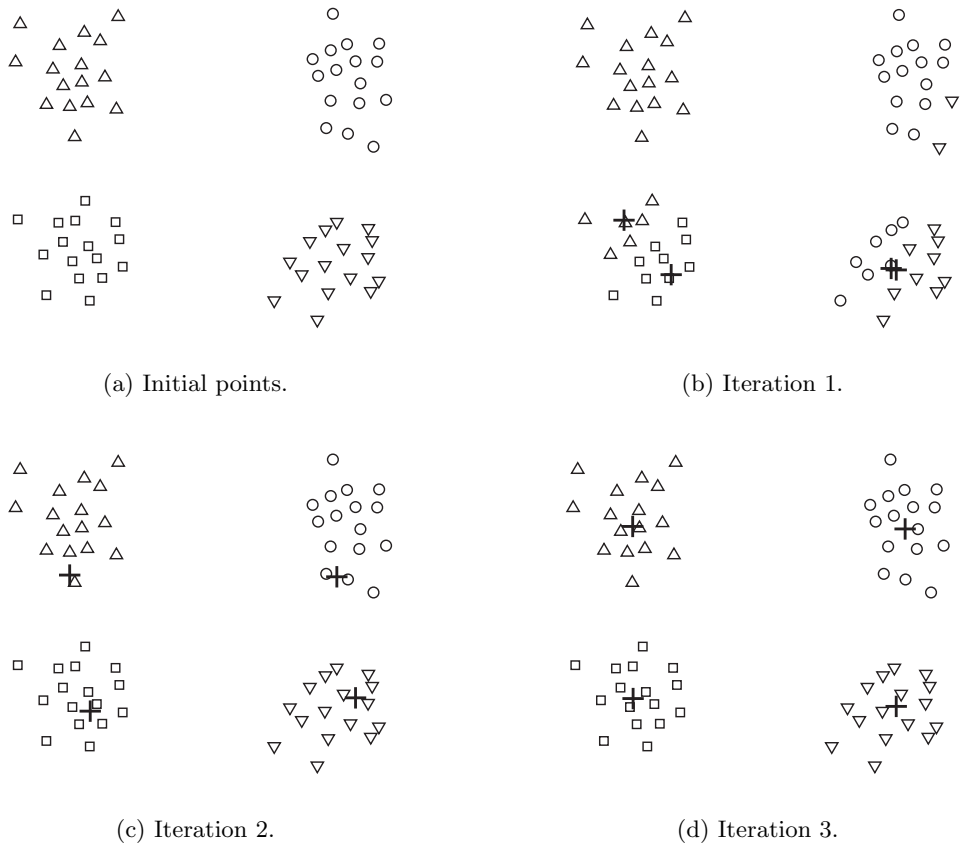
Figure 5.5. Poor starting centroids for K-means.

will redistribute themselves so that the "true" clusters are found. However, Figure 5.7 shows that if a pair of clusters has only one initial centroid and the other pair has three, then two of the true clusters will be combined and one true cluster will be split.

Note that an optimal clustering will be obtained as long as two initial centroids fall anywhere in a pair of clusters, since the centroids will redistribute themselves, one to each cluster. Unfortunately, as the number of clusters becomes larger, it is increasingly likely that at least one pair of clusters will have only one initial centroid—see Exercise 4 on page 385. In this case, because the pairs of clusters are farther apart than clusters within a pair, the K-means algorithm will not redistribute the centroids between pairs of clusters, and thus, only a local minimum will be achieved. ■

Because of the problems with using randomly selected initial centroids, which even repeated runs might not overcome, other techniques are often employed for initialization. One effective approach is to take a sample of points and cluster them using a hierarchical clustering technique. K clusters are extracted from the hierarchical clustering, and the centroids of those clusters are used as the initial centroids. This approach often works well, but is practical only if (1) the sample is relatively small, e.g., a few hundred to a few thousand (hierarchical clustering is expensive), and (2) K is relatively small compared to the sample size.

The following procedure is another approach to selecting initial centroids. Select the first point at random or take the centroid of all points. Then, for each successive initial centroid, select the point that is farthest from any of the initial centroids already selected. In this way, we obtain a set of initial centroids

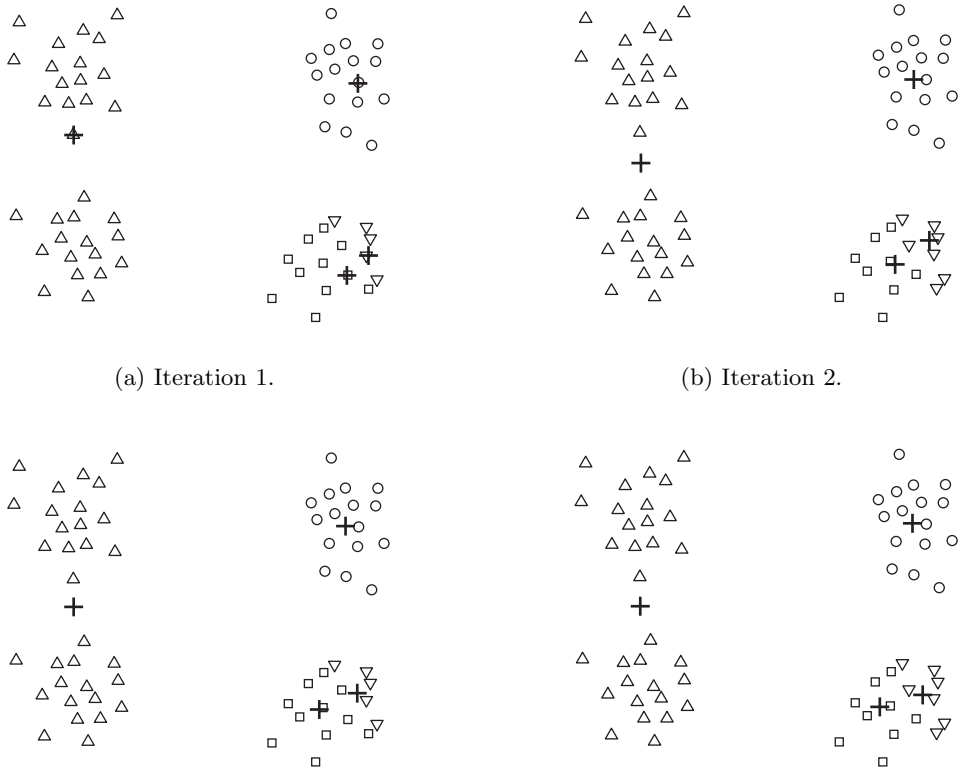


(a) Initial points. (b) Iteration 1.

(c) Iteration 2. (d) Iteration 3.

Figure 5.6. Two pairs of clusters with a pair of initial centroids within each pair of clusters.

that is guaranteed to be not only randomly selected but also well separated. Unfortunately, such an approach can select outliers, rather than points in dense regions (clusters), which can lead to a situation where many clusters have just one point—an outlier—which reduces the number of centroids for forming clusters for the majority of points. Also, it is expensive to compute the farthest point from the current set of initial centroids. To overcome these problems, this approach is often applied to a sample of the points. Because outliers are rare, they tend not to show up in a random sample. In contrast, points from every dense region are likely to be included unless the sample size is very small. Also, the computation involved in finding the initial centroids is greatly reduced because the sample size is typically much smaller than the number of points.



(a) Iteration 1.

(b) Iteration 2.

(c) Iteration 3.

(d) Iteration 4.

Figure 5.7. Two pairs of clusters with more or fewer than two initial centroids within a pair of clusters.

K-means++ More recently, a new approach for initializing K-means, called K-means++, has been developed. This procedure is guaranteed to find a K-means clustering solution that is optimal to within a factor of $O \log(k)$, which in practice translates into noticeably better clustering results in terms of lower SSE. This technique is similar to the idea just discussed of picking the first centroid at random and then picking each remaining centroid as the point as far from the remaining centroids as possible. Specifically, K-means++ picks centroids incrementally until k centroids have been picked. At every such step, each point has a probability of being picked as the new centroid that is proportional to the square of its distance to its closest centroid. It might seem that this approach might tend to choose outliers for centroids, but because outliers are rare, by definition, this is unlikely.

The details of K-means++ initialization are given by Algorithm 5.2. The rest of the algorithm is the same as ordinary K-means.

Algorithm 5.2 K-means++ initialization algorithm.

1: For the first centroid, pick one of the points at random.
2: **for** $i = 1$ to *number of trials* **do**
3: Compute the distance, $d(x)$, of each point to its closest centroid.
4: Assign each point a probability proportional to each point's $d(x)^2$.
5: Pick new centroid from the remaining points using the weighted probabilities.
6: **end for**

Later, we will discuss two other approaches that are also useful for producing better-quality (lower SSE) clusterings: using a variant of K-means that is less susceptible to initialization problems (bisecting K-means) and using postprocessing to "fix up" the set of clusters produced. K-means++ could be combined with either approach.

Time and Space Complexity

The space requirements for K-means are modest because only the data points and centroids are stored. Specifically, the storage required is $O((m + K)n)$, where m is the number of points and n is the number of attributes. The time requirements for K-means are also modest—basically linear in the number of data points. In particular, the time required is $O(I \times K \times m \times n)$, where I is the number of iterations required for convergence. As mentioned, I is often small and can usually be safely bounded, as most changes typically occur in the first few iterations. Therefore, K-means is linear in m, the number of points, and is efficient as well as simple provided that K, the number of clusters, is significantly less than m.

5.2.2 K-means: Additional Issues

Handling Empty Clusters

One of the problems with the basic K-means algorithm is that empty clusters can be obtained if no points are allocated to a cluster during the assignment step. If this happens, then a strategy is needed to choose a replacement centroid, since otherwise, the squared error will be larger than necessary. One approach is to choose the point that is farthest away from any current centroid. If nothing else, this eliminates the point that currently contributes most to the

total squared error. (A K-means++ approach could be used as well.) Another approach is to choose the replacement centroid at random from the cluster that has the highest SSE. This will typically split the cluster and reduce the overall SSE of the clustering. If there are several empty clusters, then this process can be repeated several times.

Outliers

When the squared error criterion is used, outliers can unduly influence the clusters that are found. In particular, when outliers are present, the resulting cluster centroids (prototypes) are typically not as representative as they otherwise would be and thus, the SSE will be higher. Because of this, it is often useful to discover outliers and eliminate them beforehand. It is important, however, to appreciate that there are certain clustering applications for which outliers should not be eliminated. When clustering is used for data compression, every point must be clustered, and in some cases, such as financial analysis, apparent outliers, e.g., unusually profitable customers, can be the most interesting points.

An obvious issue is how to identify outliers. A number of techniques for identifying outliers will be discussed in Chapter 9. If we use approaches that remove outliers before clustering, we avoid clustering points that will not cluster well. Alternatively, outliers can also be identified in a postprocessing step. For instance, we can keep track of the SSE contributed by each point, and eliminate those points with unusually high contributions, especially over multiple runs. Also, we often want to eliminate small clusters because they frequently represent groups of outliers.

Reducing the SSE with Postprocessing

An obvious way to reduce the SSE is to find more clusters, i.e., to use a larger K. However, in many cases, we would like to improve the SSE, but don't want to increase the number of clusters. This is often possible because K-means typically converges to a local minimum. Various techniques are used to "fix up" the resulting clusters in order to produce a clustering that has lower SSE. The strategy is to focus on individual clusters since the total SSE is simply the sum of the SSE contributed by each cluster. (We will use the terms *total SSE* and *cluster SSE*, respectively, to avoid any potential confusion.) We can change the total SSE by performing various operations on the clusters, such as splitting or merging clusters. One commonly used approach is to employ alternate cluster splitting and merging phases. During a splitting phase, clusters are divided,

while during a merging phase, clusters are combined. In this way, it is often possible to escape local SSE minima and still produce a clustering solution with the desired number of clusters. The following are some techniques used in the splitting and merging phases.

Two strategies that decrease the total SSE by increasing the number of clusters are the following:

Split a cluster: The cluster with the largest SSE is usually chosen, but we could also split the cluster with the largest standard deviation for one particular attribute.

Introduce a new cluster centroid: Often the point that is farthest from any cluster center is chosen. We can easily determine this if we keep track of the SSE contributed by each point. Another approach is to choose randomly from all points or from the points with the highest SSE with respect to their closest centroids.

Two strategies that decrease the number of clusters, while trying to minimize the increase in total SSE, are the following:

Disperse a cluster: This is accomplished by removing the centroid that corresponds to the cluster and reassigning the points to other clusters. Ideally, the cluster that is dispersed should be the one that increases the total SSE the least.

Merge two clusters: The clusters with the closest centroids are typically chosen, although another, perhaps better, approach is to merge the two clusters that result in the smallest increase in total SSE. These two merging strategies are the same ones that are used in the hierarchical clustering techniques known as the centroid method and Ward's method, respectively. Both methods are discussed in Section 5.3.

Updating Centroids Incrementally

Instead of updating cluster centroids after all points have been assigned to a cluster, the centroids can be updated incrementally, after each assignment of a point to a cluster. Notice that this requires either zero or two updates to cluster centroids at each step, since a point either moves to a new cluster (two updates) or stays in its current cluster (zero updates). Using an incremental update strategy guarantees that empty clusters are not produced because all

clusters start with a single point, and if a cluster ever has only one point, then that point will always be reassigned to the same cluster.

In addition, if incremental updating is used, the relative weight of the point being added can be adjusted; e.g., the weight of points is often decreased as the clustering proceeds. While this can result in better accuracy and faster convergence, it can be difficult to make a good choice for the relative weight, especially in a wide variety of situations. These update issues are similar to those involved in updating weights for artificial neural networks.

Yet another benefit of incremental updates has to do with using objectives other than "minimize SSE." Suppose that we are given an arbitrary objective function to measure the goodness of a set of clusters. When we process an individual point, we can compute the value of the objective function for each possible cluster assignment, and then choose the one that optimizes the objective. Specific examples of alternative objective functions are given in Section 5.5.2.

On the negative side, updating centroids incrementally introduces an order dependency. In other words, the clusters produced usually depend on the order in which the points are processed. Although this can be addressed by randomizing the order in which the points are processed, the basic K-means approach of updating the centroids after all points have been assigned to clusters has no order dependency. Also, incremental updates are slightly more expensive. However, K-means converges rather quickly, and therefore, the number of points switching clusters quickly becomes relatively small.

5.2.3 Bisecting K-means

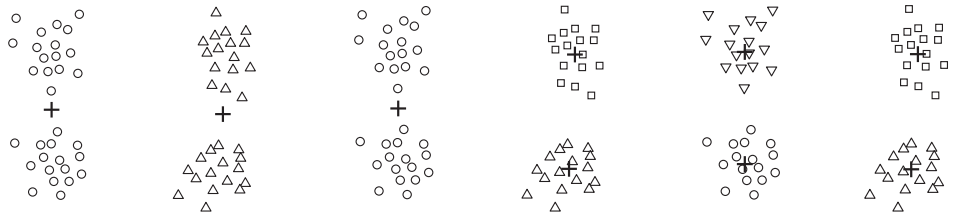
The bisecting K-means algorithm is a straightforward extension of the basic K-means algorithm that is based on a simple idea: to obtain K clusters, split the set of all points into two clusters, select one of these clusters to split, and so on, until K clusters have been produced. The details of bisecting K-means are given by Algorithm 5.3.

There are a number of different ways to choose which cluster to split. We can choose the largest cluster at each step, choose the one with the largest SSE, or use a criterion based on both size and SSE. Different choices result in different clusters.

Because we are using the K-means algorithm "locally," i.e., to bisect individual clusters, the final set of clusters does not represent a clustering that is a local minimum with respect to the total SSE. Thus, we often refine the resulting clusters by using their cluster centroids as the initial centroids for the standard K-means algorithm.

Algorithm 5.3 Bisecting K-means algorithm.

1: Initialize the list of clusters to contain the cluster consisting of all points.
2: **repeat**
3: Remove a cluster from the list of clusters.
4: {Perform several "trial" bisections of the chosen cluster.}
5: **for** $i = 1$ to *number of trials* **do**
6: Bisect the selected cluster using basic K-means.
7: **end for**
8: Select the two clusters from the bisection with the lowest total SSE.
9: Add these two clusters to the list of clusters.
10: **until** The list of clusters contains K clusters.



(a) Iteration 1. (b) Iteration 2. (c) Iteration 3.

Figure 5.8. Bisecting K-means on the four clusters example.

Example 5.3 (Bisecting K-means and Initialization)**.** To illustrate that bisecting K-means is less susceptible to initialization problems, we show, in Figure 5.8, how bisecting K-means finds four clusters in the data set originally shown in Figure 5.6(a). In iteration 1, two pairs of clusters are found; in iteration 2, the rightmost pair of clusters is split; and in iteration 3, the leftmost pair of clusters is split. Bisecting K-means has less trouble with initialization because it performs several trial bisections and takes the one with the lowest SSE, and because there are only two centroids at each step. ■

Finally, by recording the sequence of clusterings produced as K-means bisects clusters, we can also use bisecting K-means to produce a hierarchical clustering.

5.2.4 K-means and Different Types of Clusters

K-means and its variations have a number of limitations with respect to finding different types of clusters. In particular, K-means has difficulty detecting the

"natural" clusters, when clusters have non-spherical shapes or widely different sizes or densities. This is illustrated by Figures 5.9, 5.10, and 5.11. In Figure 5.9, K-means cannot find the three natural clusters because one of the clusters is much larger than the other two, and hence, the larger cluster is broken, while one of the smaller clusters is combined with a portion of the larger cluster. In Figure 5.10, K-means fails to find the three natural clusters because the two smaller clusters are much denser than the larger cluster. Finally, in Figure 5.11, K-means finds two clusters that mix portions of the two natural clusters because the shape of the natural clusters is not globular.

The difficulty in these three situations is that the K-means objective function is a mismatch for the kinds of clusters we are trying to find because it is minimized by globular clusters of equal size and density or by clusters that are well separated. However, these limitations can be overcome, in some sense, if the user is willing to accept a clustering that breaks the natural clusters into a number of subclusters. Figure 5.12 shows what happens to the three previous data sets if we find six clusters instead of two or three. Each smaller cluster is pure in the sense that it contains only points from one of the natural clusters.

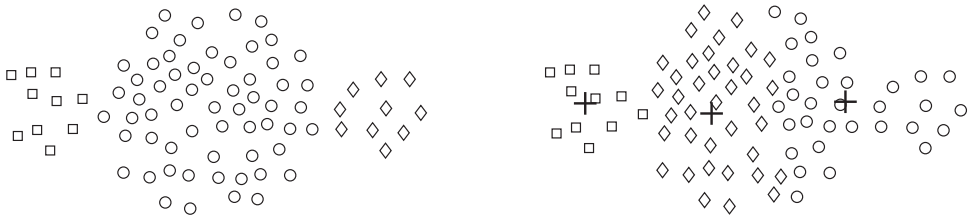
5.2.5 Strengths and Weaknesses

K-means is simple and can be used for a wide variety of data types. It is also quite efficient, even though multiple runs are often performed. Some variants, including bisecting K-means, are even more efficient, and are less susceptible to initialization problems. K-means is not suitable for all types of data, however. It cannot handle non-globular clusters or clusters of different sizes and densities, although it can typically find pure subclusters if a large enough number of clusters is specified. K-means also has trouble clustering data that contains outliers. Outlier detection and removal can help significantly in such situations. Finally, K-means is restricted to data for which there is a notion of a center (centroid). A related technique, K-medoid clustering, does not have this restriction, but is more expensive.

5.2.6 K-means as an Optimization Problem

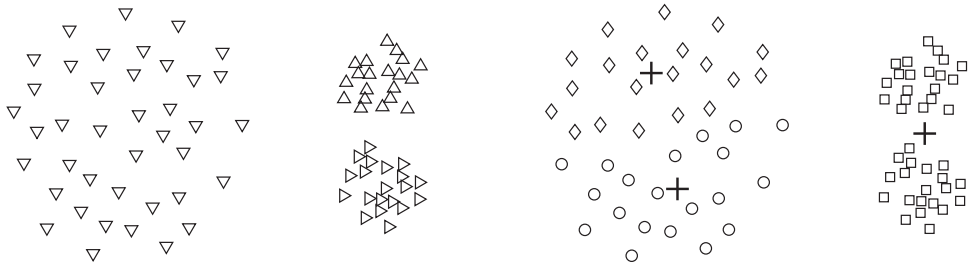
Here, we delve into the mathematics behind K-means. This section, which can be skipped without loss of continuity, requires knowledge of calculus through partial derivatives. Familiarity with optimization techniques, especially those based on gradient descent, can also be helpful.

As mentioned earlier, given an objective function such as "minimize SSE," clustering can be treated as an optimization problem. One way to solve this



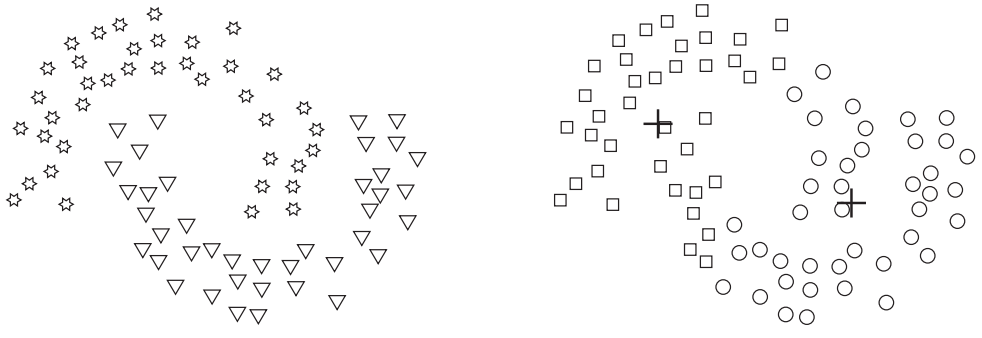
(a) Original points. (b) Three K-means clusters.

Figure 5.9. K-means with clusters of different size.



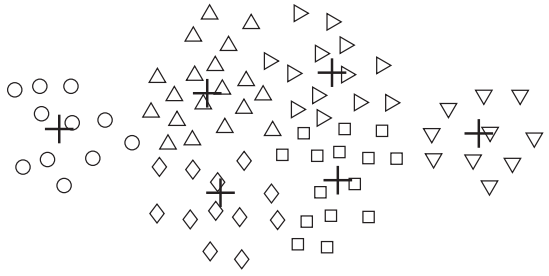
(a) Original points. (b) Three K-means clusters.

Figure 5.10. K-means with clusters of different density.

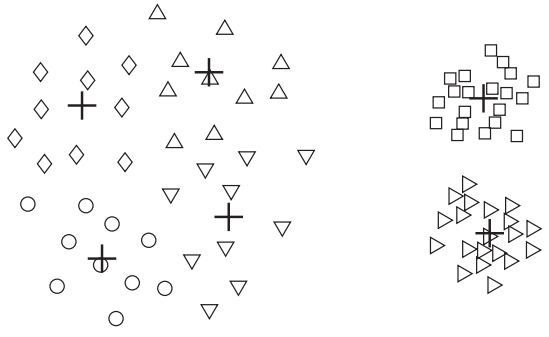


(a) Original points. (b) Two K-means clusters.

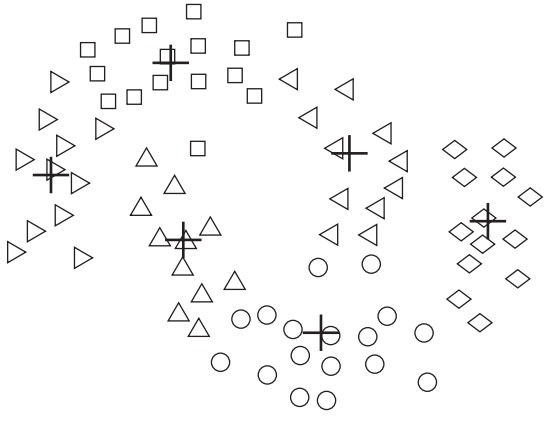
Figure 5.11. K-means with non-globular clusters.



(a) Unequal sizes.



(b) Unequal densities.



(c) Non-spherical shapes.

Figure 5.12. Using K-means to find clusters that are subclusters of the natural clusters.

problem—to find a global optimum—is to enumerate all possible ways of dividing the points into clusters and then choose the set of clusters that best satisfies the objective function, e.g., that minimizes the total SSE. Of course, this exhaustive strategy is computationally infeasible and as a result, a more practical approach is needed, even if such an approach finds solutions that are not guaranteed to be optimal. One technique, which is known as **gradient descent**, is based on picking an initial solution and then repeating the following two steps: compute the change to the solution that best optimizes the objective function and then update the solution.

We assume that the data is one-dimensional, i.e., $dist(x, y) = (x - y)^2$. This does not change anything essential, but greatly simplifies the notation.

Derivation of K-means as an Algorithm to Minimize the SSE

In this section, we show how the centroid for the K-means algorithm can be mathematically derived when the proximity function is Euclidean distance and the objective is to minimize the SSE. Specifically, we investigate how we can best update a cluster centroid so that the cluster SSE is minimized. In mathematical terms, we seek to minimize Equation 5.1, which we repeat here, specialized for one-dimensional data.

$$\text{SSE} = \sum_{i=1}^{K} \sum_{x \in C_i} (c_i - x)^2 \tag{5.4}$$

Here, C_i is the ith cluster, x is a point in C_i, and c_i is the mean of the ith cluster. See Table 5.1 for a complete list of notation.

We can solve for the kth centroid c_k, which minimizes Equation 5.4, by differentiating the SSE, setting it equal to 0, and solving, as indicated below.

$$\begin{aligned}
\frac{\partial}{\partial c_k}\text{SSE} &= \frac{\partial}{\partial c_k}\sum_{i=1}^{K}\sum_{x \in C_i} (c_i - x)^2 \\
&= \sum_{i=1}^{K}\sum_{x \in C_i} \frac{\partial}{\partial c_k}(c_i - x)^2 \\
&= \sum_{x \in C_k} 2 \times (c_k - x_k) = 0
\end{aligned}$$

$$\sum_{x \in C_k} 2 \times (c_k - x_k) = 0 \Rightarrow m_k c_k = \sum_{x \in C_k} x_k \Rightarrow c_k = \frac{1}{m_k}\sum_{x \in C_k} x_k$$

Thus, as previously indicated, the best centroid for minimizing the SSE of a cluster is the mean of the points in the cluster.

Derivation of K-means for SAE

To demonstrate that the K-means algorithm can be applied to a variety of different objective functions, we consider how to partition the data into K clusters such that the sum of the Manhattan (L_1) distances of points from the center of their clusters is minimized. We are seeking to minimize the sum of the L_1 absolute errors (SAE) as given by the following equation, where $dist_{L_1}$ is the L_1 distance. Again, for notational simplicity, we use one-dimensional data, i.e., $dist_{L_1} = |c_i - x|$.

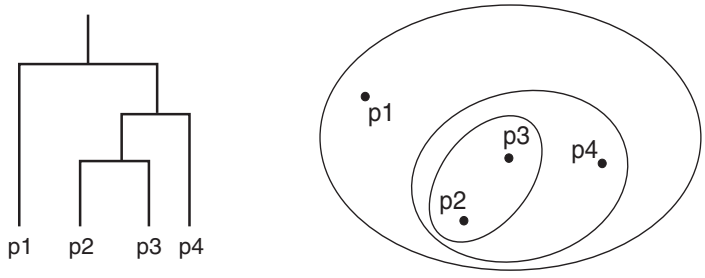
$$\text{SAE} = \sum_{i=1}^{K} \sum_{x \in C_i} dist_{L_1}(c_i, x) \tag{5.5}$$

We can solve for the kth centroid c_k, which minimizes Equation 5.5, by differentiating the SAE, setting it equal to 0, and solving.

$$\begin{aligned} \frac{\partial}{\partial c_k}\text{SAE} &= \frac{\partial}{\partial c_k}\sum_{i=1}^{K}\sum_{x \in C_i} |c_i - x| \\ &= \sum_{i=1}^{K}\sum_{x \in C_i} \frac{\partial}{\partial c_k}|c_i - x| \\ &= \sum_{x \in C_k} \frac{\partial}{\partial c_k}|c_k - x| = 0 \end{aligned}$$

$$\sum_{x \in C_k} \frac{\partial}{\partial c_k}|c_k - x| = 0 \Rightarrow \sum_{x \in C_k} sign(x - c_k) = 0$$

If we solve for c_k, we find that $c_k = median\{x \in C_k\}$, the median of the points in the cluster. The median of a group of points is straightforward to compute and less susceptible to distortion by outliers.



(a) Dendrogram. (b) Nested cluster diagram.

Figure 5.13. A hierarchical clustering of four points shown as a dendrogram and as nested clusters.

5.3 Agglomerative Hierarchical Clustering

Hierarchical clustering techniques are a second important category of clustering methods. As with K-means, these approaches are relatively old compared to many clustering algorithms, but they still enjoy widespread use. There are two basic approaches for generating a hierarchical clustering:

Agglomerative: Start with the points as individual clusters and, at each step, merge the closest pair of clusters. This requires defining a notion of cluster proximity.

Divisive: Start with one, all-inclusive cluster and, at each step, split a cluster until only singleton clusters of individual points remain. In this case, we need to decide which cluster to split at each step and how to do the splitting.

Agglomerative hierarchical clustering techniques are by far the most common, and, in this section, we will focus exclusively on these methods. A divisive hierarchical clustering technique is described in Section 8.4.2.

A hierarchical clustering is often displayed graphically using a tree-like diagram called a **dendrogram**, which displays both the cluster-subcluster relationships and the order in which the clusters were merged (agglomerative view) or split (divisive view). For sets of two-dimensional points, such as those that we will use as examples, a hierarchical clustering can also be graphically represented using a nested cluster diagram. Figure 5.13 shows an example of these two types of figures for a set of four two-dimensional points. These points were clustered using the single-link technique that is described in Section 5.3.2.

5.3.1 Basic Agglomerative Hierarchical Clustering Algorithm

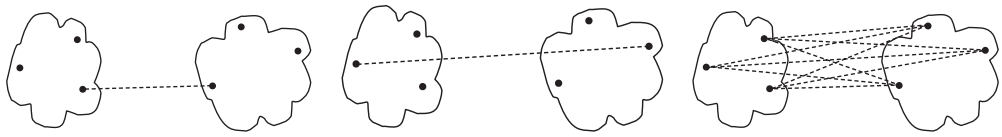
Many agglomerative hierarchical clustering techniques are variations on a single approach: starting with individual points as clusters, successively merge the two closest clusters until only one cluster remains. This approach is expressed more formally in Algorithm 5.4.

Algorithm 5.4 Basic agglomerative hierarchical clustering algorithm.

1: Compute the proximity matrix, if necessary.
2: **repeat**
3: Merge the closest two clusters.
4: Update the proximity matrix to reflect the proximity between the new cluster and the original clusters.
5: **until** Only one cluster remains.

Defining Proximity between Clusters

The key operation of Algorithm 5.4 is the computation of the proximity between two clusters, and it is the definition of cluster proximity that differentiates the various agglomerative hierarchical techniques that we will discuss. Cluster proximity is typically defined with a particular type of cluster in mind—see Section 5.1.3. For example, many agglomerative hierarchical clustering techniques, such as MIN, MAX, and Group Average, come from a graph-based view of clusters. **MIN** defines cluster proximity as the proximity between the closest two points that are in different clusters, or using graph terms, the shortest edge between two nodes in different subsets of nodes. This yields contiguity-based clusters as shown in Figure 5.2(c). Alternatively, **MAX** takes the proximity between the farthest two points in different clusters to be the cluster proximity, or using graph terms, the longest edge between two nodes in different subsets of nodes. (If our proximities are distances, then the names, MIN and MAX, are short and suggestive. For similarities, however, where higher values indicate closer points, the names seem reversed. For that reason, we usually prefer to use the alternative names, **single link** and **complete link**, respectively.) Another graph-based approach, the **group average** technique, defines cluster proximity to be the average pairwise proximities (average length of edges) of all pairs of points from different clusters. Figure 5.14 illustrates these three approaches.



(a) MIN (single link). (b) MAX (complete link). (c) Group average.

Figure 5.14. Graph-based definitions of cluster proximity.

If, instead, we take a prototype-based view, in which each cluster is represented by a centroid, different definitions of cluster proximity are more natural. When using centroids, the cluster proximity is commonly defined as the proximity between cluster centroids. An alternative technique, **Ward's** method, also assumes that a cluster is represented by its centroid, but it measures the proximity between two clusters in terms of the increase in the SSE that results from merging the two clusters. Like K-means, Ward's method attempts to minimize the sum of the squared distances of points from their cluster centroids.

Time and Space Complexity

The basic agglomerative hierarchical clustering algorithm just presented uses a proximity matrix. This requires the storage of $\frac{1}{2}m^2$ proximities (assuming the proximity matrix is symmetric) where m is the number of data points. The space needed to keep track of the clusters is proportional to the number of clusters, which is $m-1$, excluding singleton clusters. Hence, the total space complexity is $O(m^2)$.

The analysis of the basic agglomerative hierarchical clustering algorithm is also straightforward with respect to computational complexity. $O(m^2)$ time is required to compute the proximity matrix. After that step, there are $m-1$ iterations involving steps 3 and 4 because there are m clusters at the start and two clusters are merged during each iteration. If performed as a linear search of the proximity matrix, then for the ith iteration, Step 3 requires $O((m-i+1)^2)$ time, which is proportional to the current number of clusters squared. Step 4 requires $O(m-i+1)$ time to update the proximity matrix after the merger of two clusters. (A cluster merger affects $O(m-i+1)$ proximities for the techniques that we consider.) Without modification, this would yield a time complexity of $O(m^3)$. If the distances from each cluster to all other clusters are stored as a sorted list (or heap), it is possible to reduce the cost of finding

the two closest clusters to $O(m - i + 1)$. However, because of the additional complexity of keeping data in a sorted list or heap, the overall time required for a hierarchical clustering based on Algorithm 5.4 is $O(m^2 \log m)$.

The space and time complexity of hierarchical clustering severely limits the size of data sets that can be processed. We discuss scalability approaches for clustering algorithms, including hierarchical clustering techniques, in Section 8.5. Note, however, that the bisecting K-means algorithm presented in Section 5.2.3 is a scalable algorithm that produces a hierarchical clustering.

5.3.2 Specific Techniques

Sample Data

To illustrate the behavior of the various hierarchical clustering algorithms, we will use sample data that consists of six two-dimensional points, which are shown in Figure 5.15. The x and y coordinates of the points and the Euclidean distances between them are shown in Tables 5.3 and 5.4, respectively.

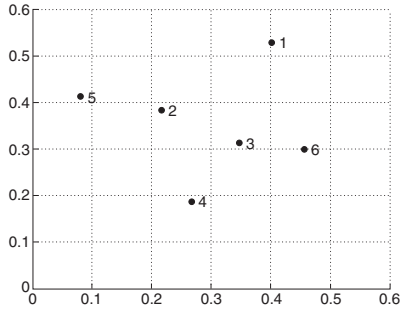


Figure 5.15. Set of six two-dimensional points.

Point	x Coordinate	y Coordinate
p1	0.4005	0.5306
p2	0.2148	0.3854
p3	0.3457	0.3156
p4	0.2652	0.1875
p5	0.0789	0.4139
p6	0.4548	0.3022

Table 5.3. xy-coordinates of six points.

	p1	p2	p3	p4	p5	p6
p1	0.00	0.24	0.22	0.37	0.34	0.23
p2	0.24	0.00	0.15	0.20	0.14	0.25
p3	0.22	0.15	0.00	0.15	0.28	0.11
p4	0.37	0.20	0.15	0.00	0.29	0.22
p5	0.34	0.14	0.28	0.29	0.00	0.39
p6	0.23	0.25	0.11	0.22	0.39	0.00

Table 5.4. Euclidean distance matrix for six points.

Single Link or MIN

For the single link or MIN version of hierarchical clustering, the proximity of two clusters is defined as the minimum of the distance (maximum of the similarity) between any two points in the two different clusters. Using graph terminology, if you start with all points as singleton clusters and add links between points one at a time, shortest links first, then these single links combine the points into clusters. The single link technique is good at handling non-elliptical shapes, but is sensitive to noise and outliers.

Example 5.4 (Single Link)**.** Figure 5.16 shows the result of applying the single link technique to our example data set of six points. Figure 5.16(a) shows the nested clusters as a sequence of nested ellipses, where the numbers associated with the ellipses indicate the order of the clustering. Figure 5.16(b) shows the same information, but as a dendrogram. The height at which two clusters are merged in the dendrogram reflects the distance of the two clusters. For instance, from Table 5.4, we see that the distance between points 3 and 6 is 0.11, and that is the height at which they are joined into one cluster in the dendrogram. As another example, the distance between clusters $\{3, 6\}$ and $\{2, 5\}$ is given by

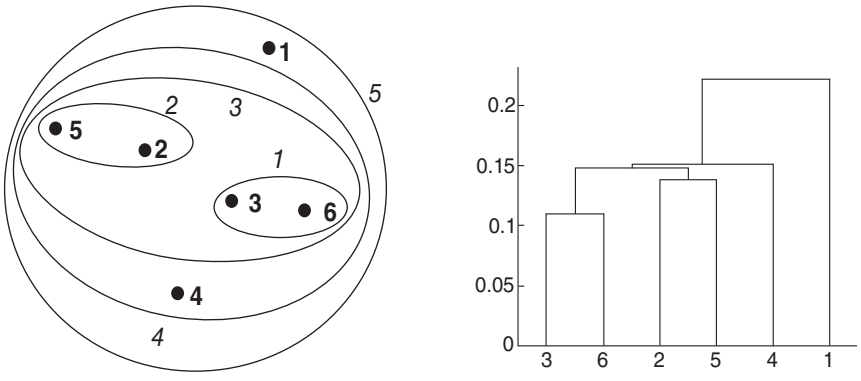
$$\begin{aligned} dist(\{3,6\},\{2,5\}) &= \min(dist(3,2), dist(6,2), dist(3,5), dist(6,5)) \\ &= \min(0.15, 0.25, 0.28, 0.39) \\ &= 0.15. \end{aligned}$$

■

Complete Link or MAX or CLIQUE

For the complete link or MAX version of hierarchical clustering, the proximity of two clusters is defined as the maximum of the distance (minimum of the similarity) between any two points in the two different clusters. Using graph terminology, if you start with all points as singleton clusters and add links between points one at a time, shortest links first, then a group of points is not a cluster until all the points in it are completely linked, i.e., form a *clique*. Complete link is less susceptible to noise and outliers, but it can break large clusters and it favors globular shapes.

Example 5.5 (Complete Link)**.** Figure 5.17 shows the results of applying MAX to the sample data set of six points. As with single link, points 3 and 6



(a) Single link clustering. (b) Single link dendrogram.

Figure 5.16. Single link clustering of the six points shown in Figure 5.15.

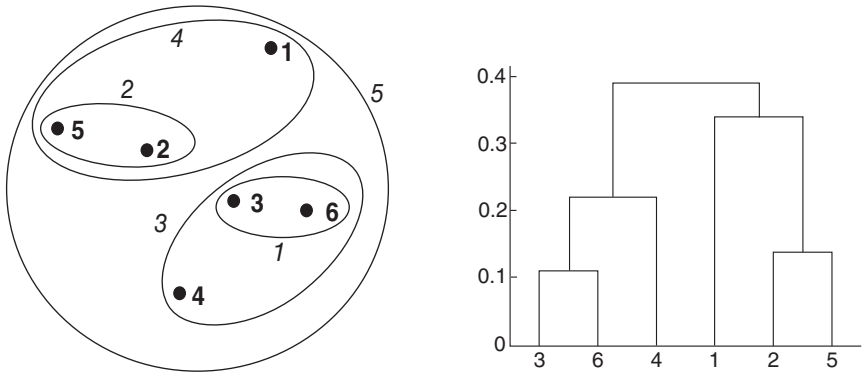
are merged first. However, $\{3, 6\}$ is merged with $\{4\}$, instead of $\{2, 5\}$ or $\{1\}$ because

$$\begin{aligned}
dist(\{3,6\},\{4\}) &= \max(dist(3,4), dist(6,4)) \\
&= \max(0.15, 0.22) \\
&= 0.22. \\
dist(\{3,6\},\{2,5\}) &= \max(dist(3,2), dist(6,2), dist(3,5), dist(6,5)) \\
&= \max(0.15, 0.25, 0.28, 0.39) \\
&= 0.39. \\
dist(\{3,6\},\{1\}) &= \max(dist(3,1), dist(6,1)) \\
&= \max(0.22, 0.23) \\
&= 0.23.
\end{aligned}$$

■

Group Average

For the group average version of hierarchical clustering, the proximity of two clusters is defined as the average pairwise proximity among all pairs of points in the different clusters. This is an intermediate approach between the single and complete link approaches. Thus, for group average, the cluster



(a) Complete link clustering. (b) Complete link dendrogram.

Figure 5.17. Complete link clustering of the six points shown in Figure 5.15.

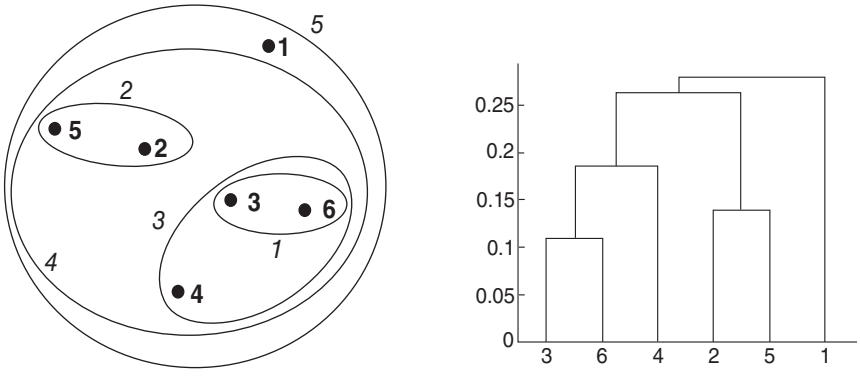
proximity $proximity(C_i, C_j)$ of clusters C_i and C_j, which are of size m_i and m_j, respectively, is expressed by the following equation:

$$proximity(C_i, C_j) = \frac{\sum_{\substack{\mathbf{x} \in C_i \\ \mathbf{y} \in C_j}} proximity(\mathbf{x}, \mathbf{y})}{m_i \times m_j}. \tag{5.6}$$

Example 5.6 (Group Average)**.** Figure 5.18 shows the results of applying the group average approach to the sample data set of six points. To illustrate how group average works, we calculate the distance between some clusters.

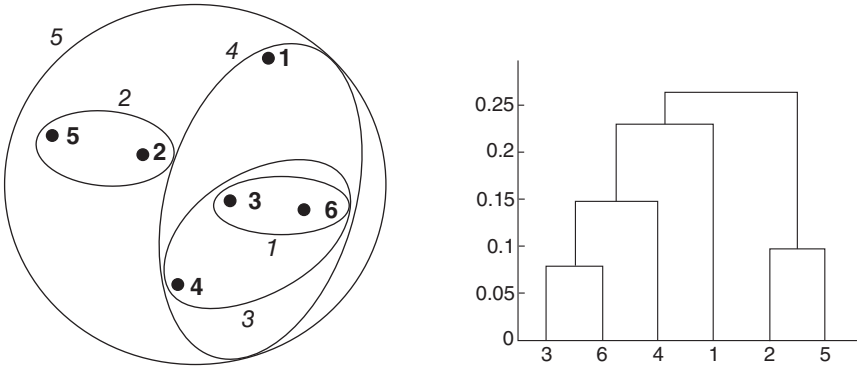
$$\begin{aligned}
dist(\{3,6,4\},\{1\}) &= (0.22 + 0.37 + 0.23)/(3 \times 1) \\
&= 0.28 \\
dist(\{2,5\},\{1\}) &= (0.24 + 0.34)/(2 \times 1) \\
&= 0.29 \\
dist(\{3,6,4\},\{2,5\}) &= (0.15 + 0.28 + 0.25 + 0.39 + 0.20 + 0.29)/(3 \times 2) \\
&= 0.26
\end{aligned}$$

Because $dist(\{3,6,4\},\{2,5\})$ is smaller than $dist(\{3,6,4\},\{1\})$ and $dist(\{2,5\},\{1\})$, clusters $\{3,6,4\}$ and $\{2,5\}$ are merged at the fourth stage. ■



(a) Group average clustering. (b) Group average dendrogram.

Figure 5.18. Group average clustering of the six points shown in Figure 5.15.



(a) Ward's clustering. (b) Ward's dendrogram.

Figure 5.19. Ward's clustering of the six points shown in Figure 5.15.

Ward's Method and Centroid Methods

For Ward's method, the proximity between two clusters is defined as the increase in the squared error that results when two clusters are merged. Thus, this method uses the same objective function as K-means clustering. While it might seem that this feature makes Ward's method somewhat distinct from other hierarchical techniques, it can be shown mathematically that Ward's

method is very similar to the group average method when the proximity between two points is taken to be the square of the distance between them.

Example 5.7 (Ward's Method)**.** Figure 5.19 shows the results of applying Ward's method to the sample data set of six points. The clustering that is produced is different from those produced by single link, complete link, and group average. ■

Centroid methods calculate the proximity between two clusters by calculating the distance between the centroids of clusters. These techniques may seem similar to K-means, but as we have remarked, Ward's method is the correct hierarchical analog.

Centroid methods also have a characteristic—often considered bad—that is not possessed by the other hierarchical clustering techniques that we have discussed: the possibility of **inversions**. Specifically, two clusters that are merged can be more similar (less distant) than the pair of clusters that were merged in a previous step. For the other methods, the distance between merged clusters monotonically increases (or is, at worst, non-increasing) as we proceed from singleton clusters to one all-inclusive cluster.

5.3.3 The Lance-Williams Formula for Cluster Proximity

Any of the cluster proximities that we have discussed in this section can be viewed as a choice of different parameters (in the Lance-Williams formula shown below in Equation 5.7) for the proximity between clusters Q and R, where R is formed by merging clusters A and B. In this equation, $p(.,.)$ is a proximity function, while m_A, m_B, and m_Q are the number of points in clusters A, B, and Q, respectively. In other words, after we merge clusters A and B to form cluster R, the proximity of the new cluster, R, to an existing cluster, Q, is a linear function of the proximities of Q with respect to the original clusters A and B. Table 5.5 shows the values of these coefficients for the techniques that we have discussed.

$$p(R,Q) = \alpha_A\, p(A,Q) + \alpha_B\, p(B,Q) + \beta\, p(A,B) + \gamma\, |p(A,Q) - p(B,Q)| \quad (5.7)$$

Any hierarchical clustering technique that can be expressed using the Lance-Williams formula does not need to keep the original data points. Instead, the proximity matrix is updated as clustering occurs. While a general formula is appealing, especially for implementation, it is easier to understand the different hierarchical methods by looking directly at the definition of cluster proximity that each method uses.

Table 5.5. Table of Lance-Williams coefficients for common hierarchical clustering approaches.

Clustering Method	α_A	α_B	β	γ
Single Link	1/2	1/2	0	−1/2
Complete Link	1/2	1/2	0	1/2
Group Average	$\frac{m_A}{m_A+m_B}$	$\frac{m_B}{m_A+m_B}$	0	0
Centroid	$\frac{m_A}{m_A+m_B}$	$\frac{m_B}{m_A+m_B}$	$\frac{-m_A m_B}{(m_A+m_B)^2}$	0
Ward's	$\frac{m_A+m_Q}{m_A+m_B+m_Q}$	$\frac{m_B+m_Q}{m_A+m_B+m_Q}$	$\frac{-m_Q}{m_A+m_B+m_Q}$	0

5.3.4 Key Issues in Hierarchical Clustering

Lack of a Global Objective Function

We previously mentioned that agglomerative hierarchical clustering cannot be viewed as globally optimizing an objective function. Instead, agglomerative hierarchical clustering techniques use various criteria to decide locally, at each step, which clusters should be merged (or split for divisive approaches). This approach yields clustering algorithms that avoid the difficulty of attempting to solve a hard combinatorial optimization problem. (It can be shown that the general clustering problem for an objective function such as "minimize SSE" is computationally infeasible.) Furthermore, such approaches do not have difficulties in choosing initial points. Nonetheless, the time complexity of $O(m^2 \log m)$ and the space complexity of $O(m^2)$ are prohibitive in many cases.

Ability to Handle Different Cluster Sizes

One aspect of agglomerative hierarchical clustering that we have not yet discussed is how to treat the relative sizes of the pairs of clusters that are merged. (This discussion applies only to cluster proximity schemes that involve sums, such as centroid, Ward's, and group average.) There are two approaches: **weighted**, which treats all clusters equally, and **unweighted**, which takes the number of points in each cluster into account. Note that the terminology of weighted or unweighted refers to the data points, not the clusters. In other words, treating clusters of unequal size equally—the weighted approach—gives different weights to the points in different clusters, while taking the cluster size into account—the unweighted approach—gives points in different clusters the same weight.

We will illustrate this using the group average technique discussed in Section 5.3.2, which is the unweighted version of the group average technique. In the clustering literature, the full name of this approach is the Unweighted

Pair Group Method using Arithmetic averages (UPGMA). In Table 5.5, which gives the formula for updating cluster similarity, the coefficients for UPGMA involve the size, m_A and m_B of each of the clusters, A and B that were merged: $\alpha_A = m_A/(m_A + m_B), \alpha_B = m_B/(m_A + m_B), \beta = 0, \gamma = 0$. For the weighted version of group average—known as WPGMA—the coefficients are constants that are independent of the cluster sizes: $\alpha_A = 1/2, \alpha_B = 1/2, \beta = 0, \gamma = 0$. In general, unweighted approaches are preferred unless there is reason to believe that individual points should have different weights; e.g., perhaps classes of objects have been unevenly sampled.

Merging Decisions Are Final

Agglomerative hierarchical clustering algorithms tend to make good local decisions about combining two clusters because they can use information about the pairwise similarity of all points. However, once a decision is made to merge two clusters, it cannot be undone at a later time. This approach prevents a local optimization criterion from becoming a global optimization criterion. For example, although the "minimize squared error" criterion from K-means is used in deciding which clusters to merge in Ward's method, the clusters at each level do not represent local minima with respect to the total SSE. Indeed, the clusters are not even stable, in the sense that a point in one cluster can be closer to the centroid of some other cluster than it is to the centroid of its current cluster. Nonetheless, Ward's method is often used as a robust method of initializing a K-means clustering, indicating that a local "minimize squared error" objective function does have a connection to a global "minimize squared error" objective function.

There are some techniques that attempt to overcome the limitation that merges are final. One approach attempts to fix up the hierarchical clustering by moving branches of the tree around so as to improve a global objective function. Another approach uses a partitional clustering technique such as K-means to create many small clusters, and then performs hierarchical clustering using these small clusters as the starting point.

5.3.5 Outliers

Outliers pose the most serious problems for Ward's method and centroid-based hierarchical clustering approaches because they increase SSE and distort centroids. For clustering approaches, such as single link, complete link, and group average, outliers are potentially less problematic. As hierarchical clustering proceeds for these algorithms, outliers or small groups of outliers tend to form

singleton or small clusters that do not merge with any other clusters until much later in the merging process. By discarding singleton or small clusters that are not merging with other clusters, outliers can be removed.

5.3.6 Strengths and Weaknesses

The strengths and weaknesses of specific agglomerative hierarchical clustering algorithms were discussed above. More generally, such algorithms are typically used because the underlying application, e.g., creation of a taxonomy, requires a hierarchy. Also, some studies have suggested that these algorithms can produce better-quality clusters. However, agglomerative hierarchical clustering algorithms are expensive in terms of their computational and storage requirements. The fact that all merges are final can also cause trouble for noisy, high-dimensional data, such as document data. In turn, these two problems can be addressed to some degree by first partially clustering the data using another technique, such as K-means.

5.4 DBSCAN

Density-based clustering locates regions of high density that are separated from one another by regions of low density. DBSCAN is a simple and effective density-based clustering algorithm that illustrates a number of important concepts that are important for any density-based clustering approach. In this section, we focus solely on DBSCAN after first considering the key notion of density. Other algorithms for finding density-based clusters are described in the next chapter.

5.4.1 Traditional Density: Center-Based Approach

Although there are not as many approaches for defining density as there are for defining similarity, there are several distinct methods. In this section we discuss the center-based approach on which DBSCAN is based. Other definitions of density will be presented in Chapter 8.

In the center-based approach, density is estimated for a particular point in the data set by counting the number of points within a specified radius, *Eps*, of that point. This includes the point itself. This technique is graphically illustrated by Figure 5.20. The number of points within a radius of *Eps* of point A is 7, including A itself.

This method is simple to implement, but the density of any point will depend on the specified radius. For instance, if the radius is large enough,

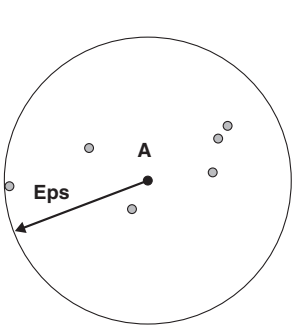


Figure 5.20. Center-based density.

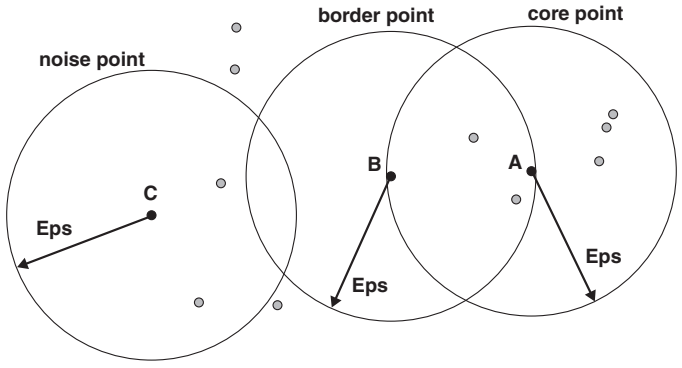


Figure 5.21. Core, border, and noise points.

then all points will have a density of m, the number of points in the data set. Likewise, if the radius is too small, then all points will have a density of 1. An approach for deciding on the appropriate radius for low-dimensional data is given in the next section in the context of our discussion of DBSCAN.

Classification of Points According to Center-Based Density

The center-based approach to density allows us to classify a point as being (1) in the interior of a dense region (a core point), (2) on the edge of a dense region (a border point), or (3) in a sparsely occupied region (a noise or background point). Figure 5.21 graphically illustrates the concepts of core, border, and noise points using a collection of two-dimensional points. The following text provides a more precise description.

Core points: These points are in the interior of a density-based cluster. A point is a core point if there are at least $MinPts$ within a distance of Eps, where $MinPts$ and Eps are user-specified parameters. In Figure 5.21, point A is a core point for the radius (Eps) if $MinPts \geq 7$.

Border points: A border point is not a core point, but falls within the neighborhood of a core point. In Figure 5.21, point B is a border point. A border point can fall within the neighborhoods of several core points.

Noise points: A noise point is any point that is neither a core point nor a border point. In Figure 5.21, point C is a noise point.

5.4.2 The DBSCAN Algorithm

Given the previous definitions of core points, border points, and noise points, the DBSCAN algorithm can be informally described as follows. Any two core points that are close enough—within a distance *Eps* of one another—are put in the same cluster. Likewise, any border point that is close enough to a core point is put in the same cluster as the core point. (Ties need to be resolved if a border point is close to core points from different clusters.) Noise points are discarded. The formal details are given in Algorithm 5.5. This algorithm uses the same concepts and finds the same clusters as the original DBSCAN, but is optimized for simplicity, not efficiency.

Algorithm 5.5 DBSCAN algorithm.

1: Label all points as core, border, or noise points.
2: Eliminate noise points.
3: Put an edge between all core points within a distance *Eps* of each other.
4: Make each group of connected core points into a separate cluster.
5: Assign each border point to one of the clusters of its associated core points.

Time and Space Complexity

The basic time complexity of the DBSCAN algorithm is $O(m \times$ time to find points in the *Eps*-neighborhood), where m is the number of points. In the worst case, this complexity is $O(m^2)$. However, in low-dimensional spaces (especially 2D space), data structures such as kd-trees allow efficient retrieval of all points within a given distance of a specified point, and the time complexity can be as low as $O(m \log m)$ in the average case. The space requirement of DBSCAN, even for high-dimensional data, is $O(m)$ because it is necessary to keep only a small amount of data for each point, i.e., the cluster label and the identification of each point as a core, border, or noise point.

Selection of DBSCAN Parameters

There is, of course, the issue of how to determine the parameters *Eps* and $MinPts$. The basic approach is to look at the behavior of the distance from a point to its kth nearest neighbor, which we will call the k-dist. For points that belong to some cluster, the value of k-dist will be small if k is not larger than the cluster size. Note that there will be some variation, depending on the density of the cluster and the random distribution of points, but on average,



Figure 5.22. Sample data.

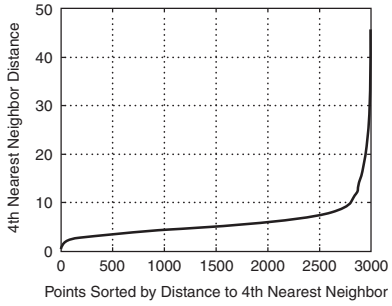


Figure 5.23. K-dist plot for sample data.

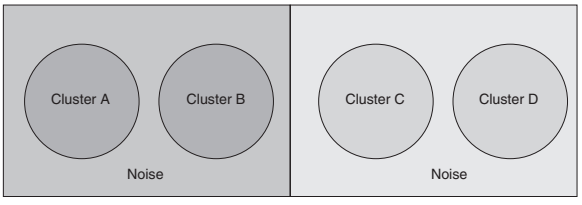


Figure 5.24. Four clusters embedded in noise.

the range of variation will not be huge if the cluster densities are not radically different. However, for points that are not in a cluster, such as noise points, the k-dist will be relatively large. Therefore, if we compute the k-dist for all the data points for some k, sort them in increasing order, and then plot the sorted values, we expect to see a sharp change at the value of k-dist that corresponds to a suitable value of Eps. If we select this distance as the Eps parameter and take the value of k as the $MinPts$ parameter, then points for which k-dist is less than Eps will be labeled as core points, while other points will be labeled as noise or border points.

Figure 5.22 shows a sample data set, while the k-dist graph for the data is given in Figure 5.23. The value of Eps that is determined in this way depends on k, but does not change dramatically as k changes. If the value of k is too small, then even a small number of closely spaced points that are noise or outliers will be incorrectly labeled as clusters. If the value of k is too large, then small clusters (of size less than k) are likely to be labeled as noise. The original DBSCAN algorithm used a value of $k = 4$, which appears to be a reasonable value for most two-dimensional data sets.

Clusters of Varying Density

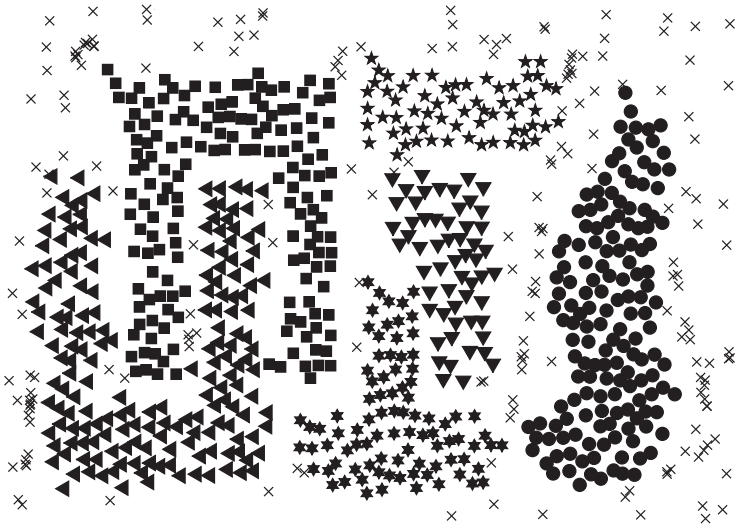
DBSCAN can have trouble with density if the density of clusters varies widely. Consider Figure 5.24, which shows four clusters embedded in noise. The density of the clusters and noise regions is indicated by their darkness. The noise around the pair of denser clusters, A and B, has the same density as clusters C and D. For a fixed $MinPts$, if the Eps threshold is chosen so that DBSCAN finds C and D as distinct clusters, with the points surrounding them as noise, then A and B and the points surrounding them will become a single cluster. If the Eps threshold is set so that DBSCAN finds A and B as separate clusters, and the points surrounding them are marked as noise, then C, D, and the points surrounding them will also be marked as noise.

An Example

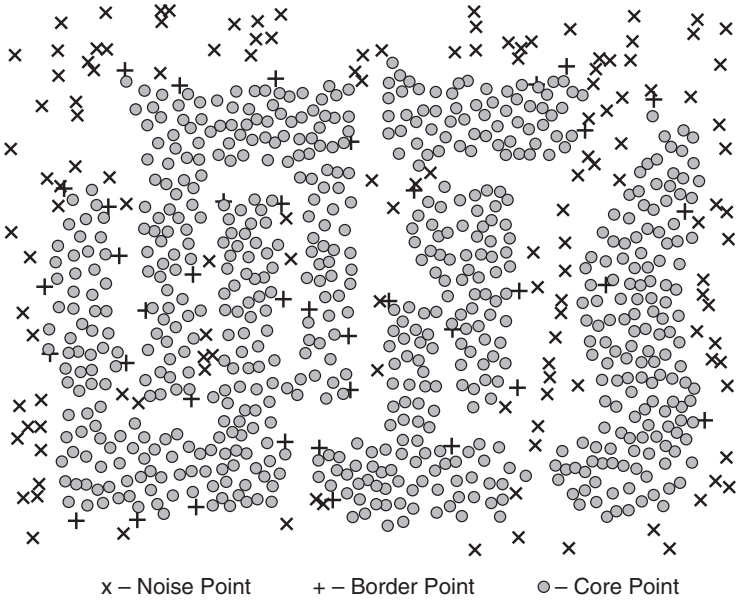
To illustrate the use of DBSCAN, we show the clusters that it finds in the relatively complicated two-dimensional data set shown in Figure 5.22. This data set consists of 3000 two-dimensional points. The Eps threshold for this data was found by plotting the sorted distances of the fourth nearest neighbor of each point (Figure 5.23) and identifying the value at which there is a sharp increase. We selected $Eps = 10$, which corresponds to the knee of the curve. The clusters found by DBSCAN using these parameters, i.e., $MinPts = 4$ and $Eps = 10$, are shown in Figure 5.25(a). The core points, border points, and noise points are displayed in Figure 5.25(b).

5.4.3 Strengths and Weaknesses

Because DBSCAN uses a density-based definition of a cluster, it is relatively resistant to noise and can handle clusters of arbitrary shapes and sizes. Thus, DBSCAN can find many clusters that could not be found using K-means, such as those in Figure 5.22. As indicated previously, however, DBSCAN has trouble when the clusters have widely varying densities. It also has trouble with high-dimensional data because density is more difficult to define for such data. One possible approach to dealing with such issues is given in Section 8.4.9. Finally, DBSCAN can be expensive when the computation of nearest neighbors requires computing all pairwise proximities, as is usually the case for high-dimensional data.



(a) Clusters found by DBSCAN.



(b) Core, border, and noise points.

Figure 5.25. DBSCAN clustering of 3000 two-dimensional points.

5.5 Cluster Evaluation

In supervised classification, the evaluation of the resulting classification model is an integral part of the process of developing a classification model, and there are well-accepted evaluation measures and procedures, e.g., accuracy and cross-validation, respectively. However, because of its very nature, cluster evaluation is not a well-developed or commonly used part of cluster analysis. Nonetheless, cluster evaluation, or **cluster validation** as it is more traditionally called, is important, and this section will review some of the most common and easily applied approaches.

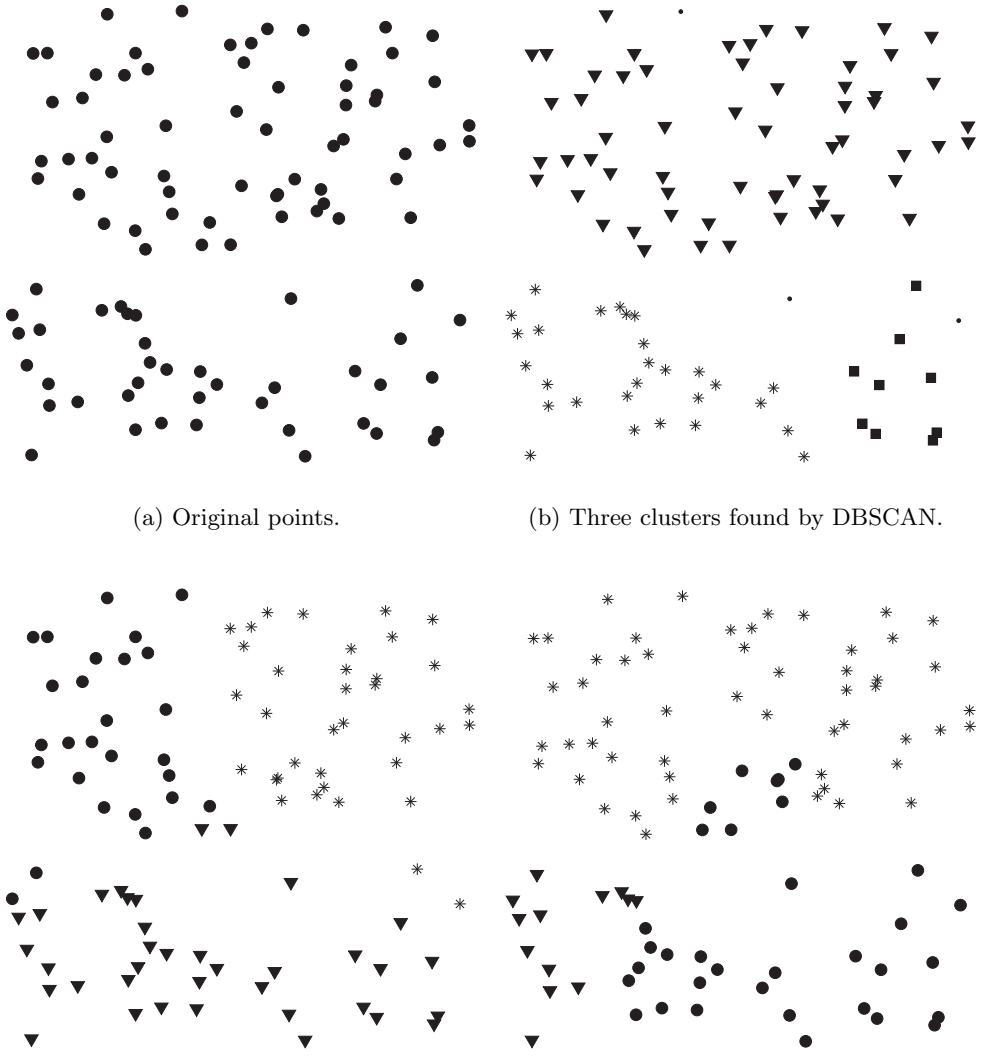
There might be some confusion as to why cluster evaluation is necessary. Many times, cluster analysis is conducted as a part of an exploratory data analysis. Hence, evaluation seems to be an unnecessarily complicated addition to what is supposed to be an informal process. Furthermore, because there are a number of different types of clusters—in some sense, each clustering algorithm defines its own type of cluster—it can seem that each situation might require a different evaluation measure. For instance, K-means clusters might be evaluated in terms of the SSE, but for density-based clusters, which need not be globular, SSE would not work well at all.

Nonetheless, cluster evaluation should be a part of any cluster analysis. A key motivation is that almost every clustering algorithm will find clusters in a data set, even if that data set has no natural cluster structure. For instance, consider Figure 5.26, which shows the result of clustering 100 points that are randomly (uniformly) distributed on the unit square. The original points are shown in Figure 5.26(a), while the clusters found by DBSCAN, K-means, and complete link are shown in Figures 5.26(b), 5.26(c), and 5.26(d), respectively. Since DBSCAN found three clusters (after we set *Eps* by looking at the distances of the fourth nearest neighbors), we set K-means and complete link to find three clusters as well. (In Figure 5.26(b) the noise is shown by the small markers.) However, the clusters do not look compelling for any of the three methods. In higher dimensions, such problems cannot be so easily detected.

5.5.1 Overview

Being able to distinguish whether there is non-random structure in the data is just one important aspect of cluster validation. The following is a list of several important issues for cluster validation.

1. Determining the **clustering tendency** of a set of data, i.e., distinguishing whether non-random structure actually exists in the data.



(a) Original points.

(b) Three clusters found by DBSCAN.

(c) Three clusters found by K-means.

(d) Three clusters found by complete link.

Figure 5.26. Clustering of 100 uniformly distributed points.

2. Determining the correct number of clusters.

3. Evaluating how well the results of a cluster analysis fit the data *without* reference to external information.

4. Comparing the results of a cluster analysis to externally known results, such as externally provided class labels.

5. Comparing two sets of clusters to determine which is better.

Notice that items 1, 2, and 3 do not make use of any external information—they are unsupervised techniques—while item 4 requires external information. Item 5 can be performed in either a supervised or an unsupervised manner. A further distinction can be made with respect to items 3, 4, and 5: Do we want to evaluate the entire clustering or just individual clusters?

While it is possible to develop various numerical measures to assess the different aspects of cluster validity mentioned above, there are a number of challenges. First, a measure of cluster validity is sometimes quite limited in the scope of its applicability. For example, most work on measures of clustering tendency has been done for two- or three-dimensional spatial data. Second, we need a framework to interpret any measure. If we obtain a value of 10 for a measure that evaluates how well cluster labels match externally provided class labels, does this value represent a good, fair, or poor match? The goodness of a match often can be measured by looking at the statistical distribution of this value, i.e., how likely it is that such a value occurs by chance. Finally, if a measure is too complicated to apply or to understand, then few will use it.

The evaluation measures, or indices, that are applied to judge various aspects of cluster validity are traditionally classified into the following three types.

Unsupervised. Measures the goodness of a clustering structure without respect to external information. An example of this is the SSE. Unsupervised measures of cluster validity are often further divided into two classes: measures of **cluster cohesion** (compactness, tightness), which determine how closely related the objects in a cluster are, and measures of **cluster separation** (isolation), which determine how distinct or well-separated a cluster is from other clusters. Unsupervised measures are often called **internal indices** because they use only information present in the data set.

Supervised. Measures the extent to which the clustering structure discovered by a clustering algorithm matches some external structure. An example of a supervised index is entropy, which measures how well cluster

labels match externally supplied class labels. Supervised measures are often called **external indices** because they use information not present in the data set.

Relative. Compares different clusterings or clusters. A relative cluster evaluation measure is a supervised or unsupervised evaluation measure that is used for the purpose of comparison. Thus, relative measures are not actually a separate type of cluster evaluation measure, but are instead a specific use of such measures. As an example, two K-means clusterings can be compared using either the SSE or entropy.

In the remainder of this section, we provide specific details concerning cluster validity. We first describe topics related to unsupervised cluster evaluation, beginning with (1) measures based on cohesion and separation, and (2) two techniques based on the proximity matrix. Since these approaches are useful only for partitional sets of clusters, we also describe the popular cophenetic correlation coefficient, which can be used for the unsupervised evaluation of a hierarchical clustering. We end our discussion of unsupervised evaluation with brief discussions about finding the correct number of clusters and evaluating clustering tendency. We then consider supervised approaches to cluster validity, such as entropy, purity, and the Jaccard measure. We conclude this section with a short discussion of how to interpret the values of (unsupervised or supervised) validity measures.

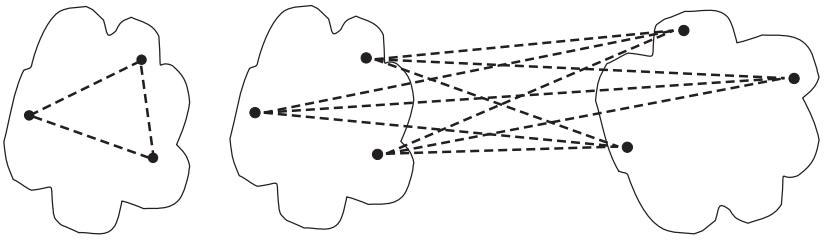
5.5.2 Unsupervised Cluster Evaluation Using Cohesion and Separation

Many internal measures of cluster validity for partitional clustering schemes are based on the notions of cohesion or separation. In this section, we use cluster validity measures for prototype- and graph-based clustering techniques to explore these notions in some detail. In the process, we will also see some interesting relationships between prototype- and graph-based measures.

In general, we can consider expressing overall cluster validity for a set of K clusters as a weighted sum of the validity of individual clusters,

$$\textit{overall validity} = \sum_{i=1}^{K} w_i \; \textit{validity}(C_i). \tag{5.8}$$

The *validity* function can be cohesion, separation, or some combination of these quantities. The weights will vary depending on the cluster validity measure. In some cases, the weights are simply 1 or the size of the cluster, while



(a) Cohesion. (b) Separation.

Figure 5.27. Graph-based view of cluster cohesion and separation.

in other cases to be discussed a bit later, they reflect a more complicated property of the cluster.

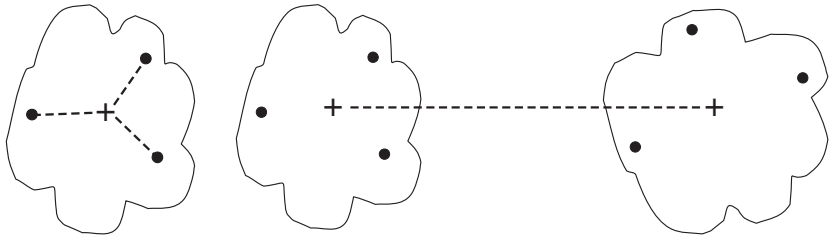
Graph-Based View of Cohesion and Separation

From a graph-based view, the cohesion of a cluster can be defined as the sum of the weights of the links in the proximity graph that connect points within the cluster. See Figure 5.27(a). (Recall that the proximity graph has data objects as nodes, a link between each pair of data objects, and a weight assigned to each link that is the proximity between the two data objects connected by the link.) Likewise, the separation between two clusters can be measured by the sum of the weights of the links from points in one cluster to points in the other cluster. This is illustrated in Figure 5.27(b).

Most simply, the cohesion and separation for a graph-based cluster can be expressed using Equations 5.9 and 5.10, respectively. The *proximity* function can be a similarity or a dissimilarity. For similarity, as in Table 5.6, higher values are better for cohesion while lower values are better for separation. For dissimilarity, the opposite is true, i.e., lower values are better for cohesion while higher values are better for separation. More complicated approaches are possible but typically embody the basic ideas of figure 5.27a and 5.27b.

$$cohesion(C_i) = \sum_{\substack{\mathbf{x} \in C_i \\ \mathbf{y} \in C_i}} proximity(\mathbf{x}, \mathbf{y}) \tag{5.9}$$

$$separation(C_i, C_j) = \sum_{\substack{\mathbf{x} \in C_i \\ \mathbf{y} \in C_j}} proximity(\mathbf{x}, \mathbf{y}) \tag{5.10}$$



(a) Cohesion. (b) Separation.

Figure 5.28. Prototype-based view of cluster cohesion and separation.

Prototype-Based View of Cohesion and Separation

For prototype-based clusters, the cohesion of a cluster can be defined as the sum of the proximities with respect to the prototype (centroid or medoid) of the cluster. Similarly, the separation between two clusters can be measured by the proximity of the two cluster prototypes. This is illustrated in Figure 5.28, where the centroid of a cluster is indicated by a "+".

Cohesion for a prototype-based cluster is given in Equation 5.11, while two measures for separation are given in Equations 5.12 and 5.13, respectively, where $\mathbf{c}_i$ is the prototype (centroid) of cluster C_i and $\mathbf{c}$ is the overall prototype (centroid). There are two measures for separation because, as we will see in the next section, the separation of cluster prototypes from an overall prototype is sometimes directly related to the separation of cluster prototypes from one another. (This is true, for example, for Euclidean distance.) Note that Equation 5.11 is the cluster SSE if we let proximity be the squared Euclidean distance.

$$cohesion(C_i) = \sum_{\mathbf{x} \in C_i} proximity(\mathbf{x}, \mathbf{c}_i) \tag{5.11}$$

$$separation(C_i, C_j) = proximity(\mathbf{c}_i, \mathbf{c}_j) \tag{5.12}$$

$$separation(C_i) = proximity(\mathbf{c}_i, \mathbf{c}) \tag{5.13}$$

Relationship between Prototype-Based Cohesion and Graph-Based Cohesion

While the graph-based and prototype-based approaches to measuring the cohesion and separation of a cluster seem distinct, for some proximity measures they are equivalent. For instance, for the SSE and points in Euclidean space, it can be shown (Equation 5.14) that the average pairwise distance between the points in a cluster is equivalent to the SSE of the cluster. See Exercise 27 on page 392.

$$\text{Cluster SSE} = \sum_{\mathbf{x} \in C_i} dist(\mathbf{c}_i, \mathbf{x})^2 = \frac{1}{2m_i} \sum_{\mathbf{x} \in C_i} \sum_{\mathbf{y} \in C_i} dist(\mathbf{x}, \mathbf{y})^2 \tag{5.14}$$

Relationship of the Two Approaches to Prototype-Based Separation

When proximity is measured by Euclidean distance, the traditional measure of separation between clusters is the between group sum of squares (SSB), which is the sum of the squared distance of a cluster centroid, $\mathbf{c}_i$, to the overall mean, $\mathbf{c}$, of all the data points. The SSB is given by Equation 5.15, where $\mathbf{c}_i$ is the mean of the ith cluster and $\mathbf{c}$ is the overall mean. The higher the total SSB of a clustering, the more separated the clusters are from one another.

$$\text{Total SSB} = \sum_{i=1}^{K} m_i \; dist(\mathbf{c}_i, \mathbf{c})^2 \tag{5.15}$$

It is straightforward to show that the total SSB is directly related to the pairwise distances between the centroids. In particular, if the cluster sizes are equal, i.e., $m_i = m/K$, then this relationship takes the simple form given by Equation 5.16. (See Exercise 28 on page 392.) It is this type of equivalence that motivates the definition of prototype separation in terms of both Equations 5.12 and 5.13.

$$\text{Total SSB} = \frac{1}{2K} \sum_{i=1}^{K} \sum_{j=1}^{K} \frac{m}{K} \; dist(\mathbf{c}_i, \mathbf{c}_j)^2 \tag{5.16}$$

Relationship between Cohesion and Separation

For some validity measures, there is also a strong relationship between cohesion and separation. Specifically, it is possible to show that the sum of the total SSE and the total SSB is a constant; i.e., that it is equal to the total sum of squares (TSS), which is the sum of squares of the distance of each point to

the overall mean of the data. The importance of this result is that minimizing SSE (cohesion) is equivalent to maximizing SSB (separation).

We provide the proof of this fact below, since the approach illustrates techniques that are also applicable to proving the relationships stated in the last two sections. To simplify the notation, we assume that the data is one-dimensional, i.e., $dist(x, y) = (x-y)^2$. Also, we use the fact that the cross-term $\sum_{i=1}^{K} \sum_{x \in C_i} (x - c_i)(c - c_i)$ is 0. (See Exercise 29 on page 392.)

$$
\begin{aligned}
\text{TSS} &= \sum_{i=1}^{K} \sum_{x \in C_i} (x - c)^2 \\
&= \sum_{i=1}^{K} \sum_{x \in C_i} ((x - c_i) - (c - c_i))^2 \\
&= \sum_{i=1}^{K} \sum_{x \in C_i} (x - c_i)^2 - 2 \sum_{i=1}^{K} \sum_{x \in C_i} (x - c_i)(c - c_i) + \sum_{i=1}^{K} \sum_{x \in C_i} (c - c_i)^2 \\
&= \sum_{i=1}^{K} \sum_{x \in C_i} (x - c_i)^2 + \sum_{i=1}^{K} \sum_{x \in C_i} (c - c_i)^2 \\
&= \sum_{i=1}^{K} \sum_{x \in C_i} (x - c_i)^2 + \sum_{i=1}^{K} |C_i|(c - c_i)^2 \\
&= \text{SSE} + \text{SSB}
\end{aligned}
$$

Relationship between Graph- and Centroid-Based Cohesion

It can also be shown that there is a relationship between graph- and centroid-based cohesion measures for Euclidean distance. For simplicity, we once again assume one-dimensional data. Recall that $c_i = 1/m_i \sum_{y \in C_i} y$.

$$
\begin{aligned}
m_i^2 cohesion(C_i) &= m_i^2 \sum_{x \in C_i} proximity(x, c_i) \\
&= \sum_{x \in C_i} m_i^2 (x - c_i)^2 \\
&= \sum_{x \in C_i} (m_i x - m_i c_i)^2 \\
&= \sum_{x \in C_i} (m_i x - m_i (1/m_i \sum_{y \in C_i} y))^2 \\
&= \sum_{x \in C_i} \sum_{y \in C_i} (x - y)^2 \\
&= \sum_{\substack{x \in C_i \\ y \in C_i}} (x - y)^2 \\
&= \sum_{\substack{x \in C_i \\ y \in C_i}} proximity(x, y)
\end{aligned}
$$

More generally, in cases where a centroid makes sense for the data, the simple graph or centroid-based measures of cluster validity we presented are often related.

Overall Measures of Cohesion and Separation

The previous definitions of cluster cohesion and separation gave us some simple and well-defined measures of individual cluster validity that can be combined into an overall measure of cluster validity by using a weighted sum, as indicated in Equation 5.8. However, we need to decide what weights to use. Not surprisingly, the weights used can vary widely. Often, but not always, they are either a function of cluster size or 1, which treats all clusters equally regardless of size.

The CLUstering TOolkit (CLUTO) (see the Bibliographic Notes) uses the cluster evaluation measures described in Table 5.6, as well as some other evaluation measures not mentioned here. Only similarity measures are used: cosine, correlation, Jaccard, and the inverse of Euclidean distance. $\mathcal{I}_1$ is a measure of cohesion in terms of the pairwise similarity of objects in the cluster. $\mathcal{I}_2$ is a measure of cohesion that can be expressed either in terms of the sum of the similarities of objects in the cluster to the cluster centroid or in terms of the pairwise similarities of objects in the cluster. $\mathcal{E}_1$ is a measure of separation.

It can be defined in terms of the similarity of a cluster centroid to the overall centroid or in terms of the pairwise similarities of object in the cluster to objects in other clusters. (Although $\mathcal{E}_1$ is a measure of separation, the second definition shows that it also uses cluster cohesion, albeit in the cluster weight.) $\mathcal{G}_1$, which is a cluster validity measure based on both cohesion and separation, is the sum of the pairwise similarities of all objects in the cluster with all objects outside the cluster—the total weight of the edges of the similarity graph that must be cut to separate the cluster from all other clusters—divided by the sum of the pairwise similarities of objects in the cluster.

Table 5.6. Table of graph-based cluster evaluation measures.

Name	Cluster Measure	Cluster Weight	Type
$\mathcal{I}_1$	$\sum_{\substack{\mathbf{x}\in C_i \\ \mathbf{y}\in C_i}} sim(\mathbf{x},\mathbf{y})$	$\frac{1}{m_i}$	graph-based cohesion
$\mathcal{I}_2$	$\sum_{\mathbf{x}\in C_i} sim(\mathbf{x},\mathbf{c}_i)$		prototype-based cohesion
$\mathcal{I}_2$	$\sqrt{\sum_{\substack{\mathbf{x}\in C_i \\ \mathbf{y}\in C_i}} sim(\mathbf{x},\mathbf{y})}$		prototype-based cohesion
$\mathcal{E}_1$	$sim(\mathbf{c}_i,\mathbf{c})$	m_i	prototype-based separation
$\mathcal{E}_1$	$\sum_{j=1}^{k} \sum_{\substack{\mathbf{x}\in C_i \\ \mathbf{y}\in C_j}} sim(\mathbf{x},\mathbf{y})$	$\frac{m_i}{\sqrt{(\sum_{\substack{\mathbf{x}\in C_i \\ \mathbf{y}\in C_i}} sim(\mathbf{x},\mathbf{y}))}}$	graph-based separation
$\mathcal{G}_1$	$\sum_{\substack{j=1 \\ j\neq i}}^{k} \sum_{\substack{\mathbf{x}\in C_i \\ \mathbf{y}\in C_j}} sim(\mathbf{x},\mathbf{y})$	$\frac{1}{\sum_{\substack{\mathbf{x}\in C_i \\ \mathbf{y}\in C_i}} sim(\mathbf{x},\mathbf{y})}$	graph-based separation and cohesion

Note that any unsupervised measure of cluster validity potentially can be used as an objective function for a clustering algorithm and vice versa. CLUTO

takes this approach by using an algorithm that is similar to the incremental K-means algorithm discussed in Section 5.2.2. Specifically, each point is assigned to the cluster that produces the best value for the cluster evaluation function. The cluster evaluation measure $\mathcal{I}_2$ corresponds to traditional K-means and produces clusters that have good SSE values. The other measures produce clusters that are not as good with respect to SSE, but that are more optimal with respect to the specified cluster validity measure.

Evaluating Individual Clusters and Objects

So far, we have focused on using cohesion and separation in the overall evaluation of a group of clusters. Most of these measures of cluster validity also can be used to evaluate individual clusters and objects. For example, we can rank individual clusters according to their specific value of cluster validity, i.e., cluster cohesion or separation. A cluster that has a high value of cohesion may be considered better than a cluster that has a lower value. This information often can be used to improve the quality of a clustering. If, for example, a cluster is not very cohesive, then we may want to split it into several subclusters. On the other hand, if two clusters are relatively cohesive, but not well separated, we may want to merge them into a single cluster.

We can also evaluate the objects within a cluster in terms of their contribution to the overall cohesion or separation of the cluster. Objects that contribute more to the cohesion and separation are near the "interior" of the cluster. Those objects for which the opposite is true are probably near the "edge" of the cluster. In the following section, we consider a cluster evaluation measure that uses an approach based on these ideas to evaluate points, clusters, and the entire set of clusters.

The Silhouette Coefficient

The popular method of silhouette coefficients combines both cohesion and separation. The following steps explain how to compute the silhouette coefficient for an individual point, a process that consists of the following three steps. We use distances, but an analogous approach can be used for similarities.

1. For the ith object, calculate its average distance to all other objects in its cluster. Call this value a_i.

2. For the ith object and any cluster not containing the object, calculate the object's average distance to all the objects in the given cluster. Find the minimum such value with respect to all clusters; call this value b_i.

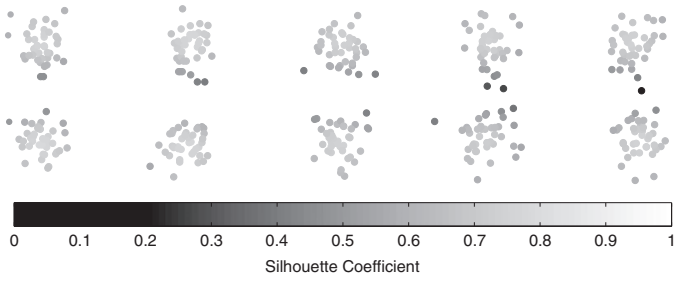


Figure 5.29. Silhouette coefficients for points in ten clusters.

3. For the ith object, the silhouette coefficient is $s_i = (b_i - a_i)/\max(a_i, b_i)$.

The value of the silhouette coefficient can vary between -1 and 1. A negative value is undesirable because this corresponds to a case in which a_i, the average distance to points in the cluster, is greater than b_i, the minimum average distance to points in another cluster. We want the silhouette coefficient to be positive ($a_i < b_i$), and for a_i to be as close to 0 as possible, since the coefficient assumes its maximum value of 1 when $a_i = 0$.

We can compute the average silhouette coefficient of a cluster by simply taking the average of the silhouette coefficients of points belonging to the cluster. An overall measure of the goodness of a clustering can be obtained by computing the average silhouette coefficient of all points.

Example 5.8 (Silhouette Coefficient)**.** Figure 5.29 shows a plot of the silhouette coefficients for points in 10 clusters. Darker shades indicate lower silhouette coefficients. ■

5.5.3 Unsupervised Cluster Evaluation Using the Proximity Matrix

In this section, we examine a couple of unsupervised approaches for assessing cluster validity that are based on the proximity matrix. The first compares an actual and idealized proximity matrix, while the second uses visualization.

General Comments on Unsupervised Cluster Evaluation Measures

In addition to the measures presented above, a large number of other measures have been proposed as unsupervised cluster validity measures. Almost all these measures, including the measures presented above are intended for partitional, center-based clusters. In particular, none of them does well for continuity- or

density-based clusters. Indeed, a recent evaluation—see Bibliographic Notes—of a dozen such measures found that although a number of them did well in terms of handling issues such as noise and differing sizes and density, none of them except a relatively recently proposed measure, Clustering Validation index based on Nearest Neighbors (CVNN), did well on handling arbitrary shapes. The silhouette index, however, did well on all other issues examined except for that.

Most unsupervised cluster evaluation measures, such as the silhouette coefficient, incorporate both cohesion (compactness) and separation. When used with a partitional clustering algorithm such as K-means, these measures will tend to decrease until the "natural" set of clusters is found and start increasing once clusters are being split "too finely" since separation will suffer and cohesion will not improve much. Thus, these measures can provide a way to determine the number of clusters. However, if the definition of a cluster used by the clustering algorithm, differs from that of the cluster evaluation measure, then the set of clusters identified as optimal by the algorithm and validation measure can differ. For instance, CLUTO uses the measures described in Table 5.6 to drive its clustering, and thus, the clustering produced will not usually match the optimal clusters according to the silhouette coefficient. Likewise for standard K-means and SSE. In addition, if there actually are subclusters that are not separated very well from one another, then methods that incorporate both may provide only a coarse view of the cluster structure of the data. Another consideration is that when clustering for summarization, we are not interested in the "natural" cluster structure of the data, but rather want to achieve a certain level of approximation, e.g., want to reduce SSE to a certain level.

More generally, if there are not too many clusters, then it can be better to examine cluster cohesion and separation independently. This can give a more comprehensive view of how cohesive each cluster is and how well each pair of clusters is separated from one another. For instance, given a centroid based clustering, we could compute the pairwise similarity or distance of the centroids, i.e., compute the distance or similarity matrix of the centroids. The approach just outlined is similar to looking at the confusion matrix for a classification problem instead of classification measures, such as accuracy or the F-measure.

Measuring Cluster Validity via Correlation

If we are given the similarity matrix for a data set and the cluster labels from a cluster analysis of the data set, then we can evaluate the "goodness" of

the clustering by looking at the correlation between the similarity matrix and an ideal version of the similarity matrix based on the cluster labels. (With minor changes, the following applies to proximity matrices, but for simplicity, we discuss only similarity matrices.) More specifically, an ideal cluster is one whose points have a similarity of 1 to all points in the cluster, and a similarity of 0 to all points in other clusters. Thus, if we sort the rows and columns of the similarity matrix so that all objects belonging to the same cluster are together, then an ideal cluster similarity matrix has a **block diagonal** structure. In other words, the similarity is non-zero, i.e., 1, inside the blocks of the similarity matrix whose entries represent intra-cluster similarity, and 0 elsewhere. The ideal cluster similarity matrix is constructed by creating a matrix that has one row and one column for each data point—just like an actual similarity matrix—and assigning a 1 to an entry if the associated pair of points belongs to the same cluster. All other entries are 0.

High correlation between the ideal and actual similarity matrices indicates that the points that belong to the same cluster are close to each other, while low correlation indicates the opposite. (Because the actual and ideal similarity matrices are symmetric, the correlation is calculated only among the $n(n-1)/2$ entries below or above the diagonal of the matrices.) Consequently, this is not a good measure for many density- or contiguity-based clusters, because they are not globular and can be closely intertwined with other clusters.

Example 5.9 (Correlation of Actual and Ideal Similarity Matrices)**.** To illustrate this measure, we calculated the correlation between the ideal and actual similarity matrices for the K-means clusters shown in Figure 5.26(c) (random data) and Figure 5.30(a) (data with three well-separated clusters). The correlations were 0.5810 and 0.9235, respectively, which reflects the expected result that the clusters found by K-means in the random data are worse than the clusters found by K-means in data with well-separated clusters. ■

Judging a Clustering Visually by Its Similarity Matrix

The previous technique suggests a more general, qualitative approach to judging a set of clusters: Order the similarity matrix with respect to cluster labels and then plot it. In theory, if we have well-separated clusters, then the similarity matrix should be roughly block-diagonal. If not, then the patterns displayed in the similarity matrix can reveal the relationships between clusters. Again, all of this can be applied to dissimilarity matrices, but for simplicity, we will only discuss similarity matrices.

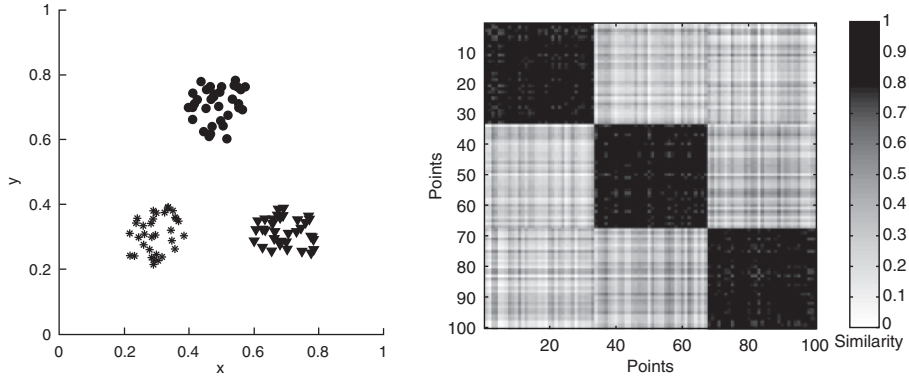
Example 5.10 (Visualizing a Similarity Matrix)**.** Consider the points in Figure 5.30(a), which form three well-separated clusters. If we use K-means to group these points into three clusters, then we should have no trouble finding these clusters because they are well-separated. The separation of these clusters is illustrated by the reordered similarity matrix shown in Figure 5.30(b). (For uniformity, we have transformed the distances into similarities using the formula $s = 1-(d-min_d)/(max_d-min_d)$.) Figure 5.31 shows the reordered similarity matrices for clusters found in the random data set of Figure 5.26 by DBSCAN, K-means, and complete link.

The well-separated clusters in Figure 5.30 show a very strong, block-diagonal pattern in the reordered similarity matrix. However, there are also weak block diagonal patterns—see Figure 5.31—in the reordered similarity matrices of the clusterings found by K-means, DBSCAN, and complete link in the random data. Just as people can find patterns in clouds, data mining algorithms can find clusters in random data. While it is entertaining to find patterns in clouds, it is pointless and perhaps embarrassing to find clusters in noise. ■

This approach may seem hopelessly expensive for large data sets, since the computation of the proximity matrix takes $O(m^2)$ time, where m is the number of objects, but with sampling, this method can still be used. We can take a sample of data points from each cluster, compute the similarity between these points, and plot the result. It is sometimes necessary to oversample small clusters and undersample large ones to obtain an adequate representation of all clusters.

5.5.4 Unsupervised Evaluation of Hierarchical Clustering

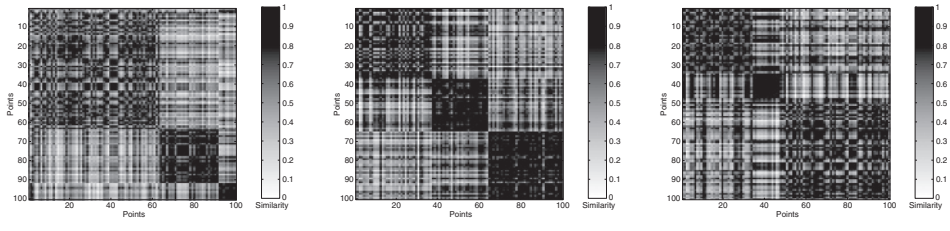
The previous approaches to cluster evaluation are intended for partitional clusterings. Here we discuss the cophenetic correlation, a popular evaluation measure for hierarchical clusterings. The **cophenetic distance** between two objects is the proximity at which an agglomerative hierarchical clustering technique puts the objects in the same cluster for the first time. For example, if at some point in the agglomerative hierarchical clustering process, the smallest distance between the two clusters that are merged is 0.1, then all points in one cluster have a cophenetic distance of 0.1 with respect to the points in the other cluster. In a cophenetic distance matrix, the entries are the cophenetic distances between each pair of objects. The cophenetic distance is different for each hierarchical clustering of a set of points.



(a) Well-separated clusters.

(b) Similarity matrix sorted by K-means cluster labels.

Figure 5.30. Similarity matrix for well-separated clusters.



(a) Similarity matrix sorted by DBSCAN cluster labels.

(b) Similarity matrix sorted by K-means cluster labels.

(c) Similarity matrix sorted by complete link cluster labels.

Figure 5.31. Similarity matrices for clusters from random data.

Example 5.11 (Cophenetic Distance Matrix)**.** Table 5.7 shows the cophenetic distance matrix for the single link clustering shown in Figure 5.16. (The data for this figure consists of the six two-dimensional points given in Table 2.14.)

■

The **Cophenetic Correlation Coefficient** (CPCC) is the correlation between the entries of this matrix and the original dissimilarity matrix and is a standard measure of how well a hierarchical clustering (of a particular type)

Table 5.7. Cophenetic distance matrix for single link and data in Table 2.14 on page 110.

Point	P1	P2	P3	P4	P5	P6
P1	0	0.222	0.222	0.222	0.222	0.222
P2	0.222	0	0.148	0.151	0.139	0.148
P3	0.222	0.148	0	0.151	0.148	0.110
P4	0.222	0.151	0.151	0	0.151	0.151
P5	0.222	0.139	0.148	0.151	0	0.148
P6	0.222	0.148	0.110	0.151	0.148	0

fits the data. One of the most common uses of this measure is to evaluate which type of hierarchical clustering is best for a particular type of data.

Example 5.12 (Cophenetic Correlation Coefficient)**.** We calculated the CPCC for the hierarchical clusterings shown in Figures 5.16–5.19. These values are shown in Table 5.8. The hierarchical clustering produced by the single link technique seems to fit the data less well than the clusterings produced by complete link, group average, and Ward's method.

Table 5.8. Cophenetic correlation coefficient for data of Table 2.14 and four agglomerative hierarchical clustering techniques.

Technique	CPCC
Single Link	0.44
Complete Link	0.63
Group Average	0.66
Ward's	0.64

■

5.5.5 Determining the Correct Number of Clusters

Various unsupervised cluster evaluation measures can be used to approximately determine the correct or natural number of clusters.

Example 5.13 (Number of Clusters)**.** The data set of Figure 5.29 has 10 natural clusters. Figure 5.32 shows a plot of the SSE versus the number of clusters for a (bisecting) K-means clustering of the data set, while Figure 5.33 shows the average silhouette coefficient versus the number of clusters for the same data. There is a distinct knee in the SSE and a distinct peak in the silhouette coefficient when the number of clusters is equal to 10. ■

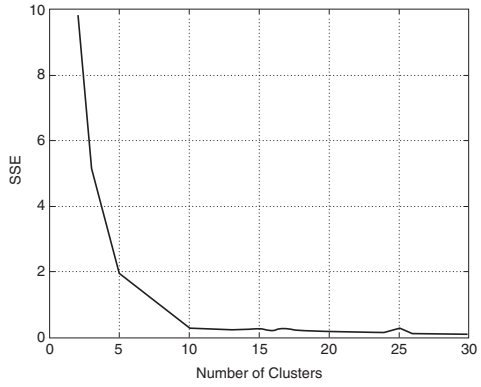


Figure 5.32. SSE versus number of clusters for the data of Figure 5.29 on page 364.

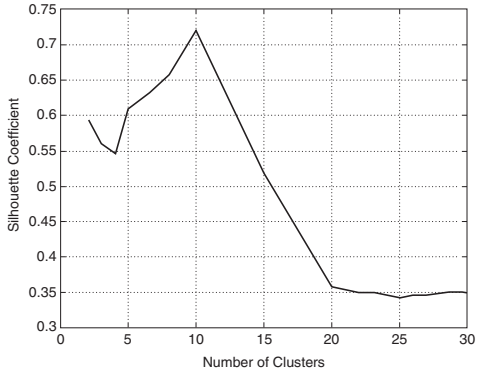


Figure 5.33. Average silhouette coefficient versus number of clusters for the data of Figure 5.29.

Thus, we can try to find the natural number of clusters in a data set by looking for the number of clusters at which there is a knee, peak, or dip in the plot of the evaluation measure when it is plotted against the number of clusters. Of course, such an approach does not always work well. Clusters can be considerably more intertwined or overlapping than those shown in Figure 5.29. Also, the data can consist of nested clusters. Actually, the clusters in Figure 5.29 are somewhat nested; i.e., there are five pairs of clusters since the clusters are closer top to bottom than they are left to right. There is a knee that indicates this in the SSE curve, but the silhouette coefficient curve is not as clear. In summary, while caution is needed, the technique we have just described can provide insight into the number of clusters in the data.

5.5.6 Clustering Tendency

One obvious way to determine if a data set has clusters is to try to cluster it. However, almost all clustering algorithms will dutifully find clusters when given data. To address this issue, we could evaluate the resulting clusters and only claim that a data set has clusters if at least some of the clusters are of good quality. However, this approach does not address the fact the clusters in the data can be of a different type than those sought by our clustering algorithm. To handle this additional problem, we could use multiple algorithms and again evaluate the quality of the resulting clusters. If the clusters are uniformly poor, then this may indeed indicate that there are no clusters in the data.

Alternatively, and this is the focus of measures of clustering tendency, we can try to evaluate whether a data set has clusters without clustering. The most common approach, especially for data in Euclidean space, has been to use statistical tests for spatial randomness. Unfortunately, choosing the correct model, estimating the parameters, and evaluating the statistical significance of the hypothesis that the data is non-random can be quite challenging. Nonetheless, many approaches have been developed, most of them for points in low-dimensional Euclidean space.

Example 5.14 (Hopkins Statistic)**.** For this approach, we generate p points that are randomly distributed across the data space and also sample p actual data points. For both sets of points, we find the distance to the nearest neighbor in the original data set. Let the u_i be the nearest neighbor distances of the artificially generated points, while the w_i are the nearest neighbor distances of the sample of points from the original data set. The Hopkins statistic H is then defined by Equation 5.17.

$$H = \frac{\sum_{i=1}^{p} w_i}{\sum_{i=1}^{p} u_i + \sum_{i=1}^{p} w_i} \tag{5.17}$$

If the randomly generated points and the sample of data points have roughly the same nearest neighbor distances, then H will be near 0.5. Values of H near 0 and 1 indicate, respectively, data that is highly clustered and data that is regularly distributed in the data space. To give an example, the Hopkins statistic for the data of Figure 5.26 was computed for $p = 20$ and 100 different trials. The average value of H was 0.56 with a standard deviation of 0.03. The same experiment was performed for the well-separated points of Figure 5.30. The average value of H was 0.95 with a standard deviation of 0.006. ■

5.5.7 Supervised Measures of Cluster Validity

When we have external information about data, it is typically in the form of externally derived class labels for the data objects. In such cases, the usual procedure is to measure the degree of correspondence between the cluster labels and the class labels. But why is this of interest? After all, if we have the class labels, then what is the point in performing a cluster analysis? Motivations for such an analysis include the comparison of clustering techniques with the "ground truth" or the evaluation of the extent to which a manual classification process can be automatically produced by cluster analysis, e.g., the clustering of news articles. Another potential motivation

could be to evaluate whether objects in the same cluster tend to have the same label for semi-supervised learning techniques.

We consider two different kinds of approaches. The first set of techniques use measures from classification, such as entropy, purity, and the F-measure. These measures evaluate the extent to which a cluster contains objects of a single class. The second group of methods is related to the similarity measures for binary data, such as the Jaccard measure that we saw in Chapter 2. These approaches measure the extent to which two objects that are in the same class are in the same cluster and vice versa. For convenience, we will refer to these two types of measures as **classification-oriented** and **similarity-oriented**, respectively.

Classification-Oriented Measures of Cluster Validity

There are a number of measures that are commonly used to evaluate the performance of a classification model. In this section, we will discuss five: entropy, purity, precision, recall, and the F-measure. In the case of classification, we measure the degree to which predicted class labels correspond to actual class labels, but for the measures just mentioned, nothing fundamental is changed by using cluster labels instead of predicted class labels. Next, we quickly review the definitions of these measures in the context of clustering.

Entropy: The degree to which each cluster consists of objects of a single class. For each cluster, the class distribution of the data is calculated first, i.e., for cluster i we compute p_{ij}, the probability that a member of cluster i belongs to class j as $p_{ij} = m_{ij}/m_i$, where m_i is the number of objects in cluster i and m_{ij} is the number of objects of class j in cluster i. Using this class distribution, the entropy of each cluster i is calculated using the standard formula, $e_i = -\sum_{j=1}^{L} p_{ij} \log_2 p_{ij}$, where L is the number of classes. The total entropy for a set of clusters is calculated as the sum of the entropies of each cluster weighted by the size of each cluster, i.e., $e = \sum_{i=1}^{K} \frac{m_i}{m} e_i$, where K is the number of clusters and m is the total number of data points.

Purity: Another measure of the extent to which a cluster contains objects of a single class. Using the previous terminology, the purity of cluster i is $purity(i) = \max_j p_{ij}$, the overall purity of a clustering is $purity = \sum_{i=1}^{K} \frac{m_i}{m} purity(i)$.

Table 5.9. K-means clustering results for the *LA Times* document data set.

Cluster	Entertainment	Financial	Foreign	Metro	National	Sports	Entropy	Purity
1	3	5	40	506	96	27	1.2270	0.7474
2	4	7	280	29	39	2	1.1472	0.7756
3	1	1	1	7	4	671	0.1813	0.9796
4	10	162	3	119	73	2	1.7487	0.4390
5	331	22	5	70	13	23	1.3976	0.7134
6	5	358	12	212	48	13	1.5523	0.5525
Total	354	555	341	943	273	738	1.1450	0.7203

Precision: The fraction of a cluster that consists of objects of a specified class. The precision of cluster i with respect to class j is $precision(i, j) = p_{ij}$.

Recall: The extent to which a cluster contains all objects of a specified class. The recall of cluster i with respect to class j is $recall(i, j) = m_{ij}/m_j$, where m_j is the number of objects in class j.

F-measure A combination of both precision and recall that measures the extent to which a cluster contains *only* objects of a particular class and *all* objects of that class. The F-measure of cluster i with respect to class j is $F(i, j) = (2 \times precision(i, j) \times recall(i, j))/(precision(i, j) + recall(i, j))$. The F-measure of a set of clusters, partitional or hierarchical is presented on page 376 when we discuss cluster validity for hierarchical clusterings.

Example 5.15 (Supervised Evaluation Measures)**.** We present an example to illustrate these measures. Specifically, we use K-means with the cosine similarity measure to cluster 3204 newspaper articles from the *Los Angeles Times*. These articles come from six different classes: Entertainment, Financial, Foreign, Metro, National, and Sports. Table 5.9 shows the results of a K-means clustering to find six clusters. The first column indicates the cluster, while the next six columns together form the confusion matrix; i.e., these columns indicate how the documents of each category are distributed among the clusters. The last two columns are the entropy and purity of each cluster, respectively.

Ideally, each cluster will contain documents from only one class. In reality, each cluster contains documents from many classes. Nevertheless, many clusters contain documents primarily from just one class. In particular, cluster 3, which contains mostly documents from the Sports section, is exceptionally good, both in terms of purity and entropy. The purity and entropy of the other

clusters is not as good, but can typically be greatly improved if the data is partitioned into a larger number of clusters.

Precision, recall, and the F-measure can be calculated for each cluster. To give a concrete example, we consider cluster 1 and the Metro class of Table 5.9. The precision is $506/677 = 0.75$, recall is $506/943 = 0.26$, and hence, the F value is 0.39. In contrast, the F value for cluster 3 and Sports is 0.94. As in classification, the confusion matrix gives the most detailed information. ■

Similarity-Oriented Measures of Cluster Validity

The measures that we discuss in this section are all based on the premise that any two objects that are in the same cluster should be in the same class and vice versa. We can view this approach to cluster validity as involving the comparison of two matrices: (1) the **ideal cluster similarity matrix** discussed previously, which has a 1 in the ij^{th} entry if two objects, i and j, are in the same cluster and 0, otherwise, and (2) a **class similarity matrix** defined with respect to class labels, which has a 1 in the ij^{th} entry if two objects, i and j, belong to the same class, and a 0 otherwise. As before, we can take the correlation of these two matrices as the measure of cluster validity. This measure is known as Hubert's Γ statistic in the clustering validation literature.

Example 5.16 (Correlation between Cluster and Class Matrices)**.** To demonstrate this idea more concretely, we give an example involving five data points, $p_1, p_2, p_3, p_4, and p_5$, two clusters, $C_1 = \{p_1, p_2, p_3\}$ and $C_2 = \{p_4, p_5\}$, and two classes, $L_1 = \{p_1, p_2\}$ and $L2 = \{p_3, p_4, p_5\}$. The ideal cluster and class similarity matrices are given in Tables 5.10 and 5.11. The correlation between the entries of these two matrices is 0.359.

Table 5.10. Ideal cluster similarity matrix.

Point	p1	p2	p3	p4	p5
p1	1	1	1	0	0
p2	1	1	1	0	0
p3	1	1	1	0	0
p4	0	0	0	1	1
p5	0	0	0	1	1

Table 5.11. Class similarity matrix.

Point	p1	p2	p3	p4	p5
p1	1	1	0	0	0
p2	1	1	0	0	0
p3	0	0	1	1	1
p4	0	0	1	1	1
p5	0	0	1	1	1

■

More generally, we can use any of the measures for binary similarity that we saw in Section 2.4.5. (For example, we can convert these two matrices into binary vectors by appending the rows.) We repeat the definitions of the four quantities used to define those similarity measures, but modify our descriptive text to fit the current context. Specifically, we need to compute the following four quantities for all pairs of distinct objects. (There are $m(m-1)/2$ such pairs, if m is the number of objects.)

f_{00} = number of pairs of objects having a different class and a different cluster
f_{01} = number of pairs of objects having a different class and the same cluster
f_{10} = number of pairs of objects having the same class and a different cluster
f_{11} = number of pairs of objects having the same class and the same cluster

In particular, the simple matching coefficient, which is known as the Rand statistic in this context, and the Jaccard coefficient are two of the most frequently used cluster validity measures.

$$\text{Rand statistic} = \frac{f_{00} + f_{11}}{f_{00} + f_{01} + f_{10} + f_{11}} \tag{5.18}$$

$$\text{Jaccard coefficient} = \frac{f_{11}}{f_{01} + f_{10} + f_{11}} \tag{5.19}$$

Example 5.17 (Rand and Jaccard Measures)**.** Based on these formulas, we can readily compute the Rand statistic and Jaccard coefficient for the example based on Tables 5.10 and 5.11. Noting that $f_{00} = 4$, $f_{01} = 2$, $f_{10} = 2$, and $f_{11} = 2$, the Rand statistic $= (2 + 4)/10 = 0.6$ and the Jaccard coefficient $= 2/(2+2+2\ ) = 0.33$. ■

We also note that the four quantities, f_{00}, f_{01}, f_{10}, and f_{11}, define a *contingency* table as shown in Table 5.12.

Table 5.12. Two-way contingency table for determining whether pairs of objects are in the same class and same cluster.

	Same Cluster	Different Cluster
Same Class	f_{11}	f_{10}
Different Class	f_{01}	f_{00}

Previously, in the context of association analysis—see Section 4.7.1 on page 258—we presented an extensive discussion of measures of association that can

be used for this type of contingency table. (Compare Table 5.12 on page 375 with Table 4.6 on page 258.) Those measures can also be applied to cluster validity.

Cluster Validity for Hierarchical Clusterings

So far in this section, we have discussed supervised measures of cluster validity only for partitional clusterings. Supervised evaluation of a hierarchical clustering is more difficult for a variety of reasons, including the fact that a preexisting hierarchical structure often does not exist. In addition, although relatively pure clusters often exist at certain levels in the hierarchical clustering, as the clustering proceeds, the clusters will become impure. The key idea of the approach presented here, which is based on the F-measure, is to evaluate whether a hierarchical clustering contains, for each class, at least one cluster that is relatively pure and includes most of the objects of that class. To evaluate a hierarchical clustering with respect to this goal, we compute, for each class, the F-measure for each cluster in the cluster hierarchy, and then take the maximum F-measure attained for any cluster. Finally, we calculate an overall F-measure for the hierarchical clustering by computing the weighted average of all per-class F-measures, where the weights are based on the class sizes. More formally, this hierarchical F-measure is defined as follows:

$$F = \sum_j \frac{m_j}{m} \max_i F(i, j)$$

where the maximum is taken over all clusters i at all levels, m_j is the number of objects in class j, and m is the total number of objects. Note that this measure can also be applied for a partitional clustering without modification.

5.5.8 Assessing the Significance of Cluster Validity Measures

Cluster validity measures are intended to help us measure the goodness of the clusters that we have obtained. Indeed, they typically give us a single number as a measure of that goodness. However, we are then faced with the problem of interpreting the significance of this number, a task that might be even more difficult.

The minimum and maximum values of cluster evaluation measures can provide some guidance in many cases. For instance, by definition, a purity of 0 is bad, while a purity of 1 is good, at least if we trust our class labels and

want our cluster structure to reflect the class structure. Likewise, an entropy of 0 is good, as is an SSE of 0.

Sometimes, however, there is no minimum or maximum value, or the scale of the data might affect the interpretation. Also, even if there are minimum and maximum values with obvious interpretations, intermediate values still need to be interpreted. In some cases, we can use an absolute standard. If, for example, we are clustering for utility, we are often willing to tolerate only a certain level of error in the approximation of our points by a cluster centroid.

But if this is not the case, then we must do something else. A common approach is to interpret the value of our validity measure in statistical terms. Specifically, we attempt to judge how likely it is that our observed value was achieved by random chance. The value is good if it is unusual; i.e., if it is unlikely to be the result of random chance. The motivation for this approach is that we are interested only in clusters that reflect non-random structure in the data, and such structures should generate unusually high (low) values of our cluster validity measure, at least if the validity measures are designed to reflect the presence of strong cluster structure.

Example 5.18 (Significance of SSE)**.** To show how this works, we present an example based on K-means and the SSE. Suppose that we want a measure of how good the well-separated clusters of Figure 5.30 are with respect to random data. We generate many random sets of 100 points having the same range as the points in the three clusters, find three clusters in each data set using K-means, and accumulate the distribution of SSE values for these clusterings. By using this distribution of the SSE values, we can then estimate the probability of the SSE value for the original clusters. Figure 5.34 shows the histogram of the SSE from 500 random runs. The lowest SSE shown in Figure 5.34 is 0.0173. For the three clusters of Figure 5.30, the SSE is 0.0050. We could therefore conservatively claim that there is less than a 1% chance that a clustering such as that of Figure 5.30 could occur by chance. ∎

In the previous example, randomization was used to evaluate the statistical significance of a cluster validity measure. However, for some measures, such as Hubert's Γ statistic, the distribution is known and can be used to evaluate the measure. In addition, a normalized version of the measure can be computed by subtracting the mean and dividing by the standard deviation. See Bibliographic Notes for more details.

We stress that there is more to cluster evaluation (unsupervised or supervised) than obtaining a numerical measure of cluster validity. Unless this value has a natural interpretation based on the definition of the measure, we need to interpret this value in some way. If our cluster evaluation measure

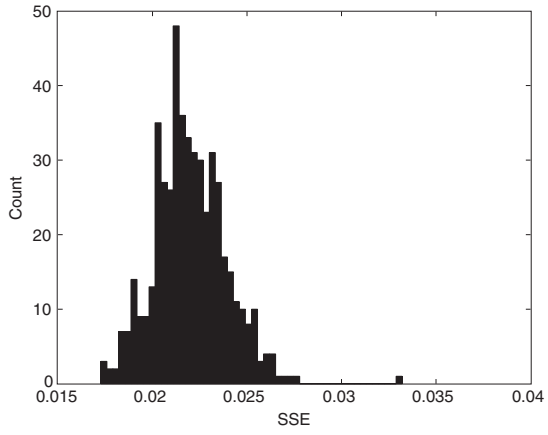


Figure 5.34. Histogram of SSE for 500 random data sets.

is defined such that lower (higher) values indicate stronger clusters, then we can use statistics to evaluate whether the value we have obtained is unusually low (high), provided we have a distribution for the evaluation measure. We have presented an example of how to find such a distribution, but there is considerably more to this topic, and we refer the reader to the Bibliographic Notes for more pointers.

Finally, even when an evaluation measure is used as a relative measure, i.e., to compare two clusterings, we still need to assess the significance in the difference between the evaluation measures of the two clusterings. Although one value will almost always be better than another, it can be difficult to determine if the difference is significant. Note that there are two aspects to this significance: whether the difference is statistically significant (repeatable) and whether the magnitude of the difference is meaningful with respect to the application. Many would not regard a difference of 0.1% as significant, even if it is consistently reproducible.

5.5.9 Choosing a Cluster Validity Measure

There are many more measures for evaluating cluster validity than have been discussed here. Various books and articles propose various measures as being superior to others. In this section, we offer some high-level guidance. First, it is important to distinguish whether the clustering is being performed for summarization or understanding. If summarization, typically class labels are not involved and the goal is maximum compression. This is often done by finding clusters that minimize the distance of objects to their closest cluster

and Euclidean distance, which are useful for non-sparse (dense) data, such as time series or multi-dimensional points. We also consider mutual information, which can be applied to many types of data and is good for detecting nonlinear relationships. In this discussion, we restrict ourselves to objects with relatively homogeneous attribute types, typically binary or continuous.

Next, we consider several important issues concerning proximity measures. This includes how to compute proximity between objects when they have heterogeneous types of attributes, and approaches to account for differences of scale and correlation among variables when computing distance between numerical objects. The section concludes with a brief discussion of how to select the right proximity measure.

Although this section focuses on the computation of proximity between data objects, proximity can also be computed between attributes. For example, for the document-term matrix of Figure 2.2(d), the cosine measure can be used to compute similarity between a pair of documents or a pair of terms (words). Knowing that two variables are strongly related can, for example, be helpful for eliminating redundancy. In particular, the correlation and mutual information measures discussed later are often used for that purpose.

2.4.1 Basics

Definitions

Informally, the **similarity** between two objects is a numerical measure of the degree to which the two objects are alike. Consequently, similarities are *higher* for pairs of objects that are more alike. Similarities are usually non-negative and are often between 0 (no similarity) and 1 (complete similarity).

The **dissimilarity** between two objects is a numerical measure of the degree to which the two objects are different. Dissimilarities are *lower* for more similar pairs of objects. Frequently, the term **distance** is used as a synonym for dissimilarity, although, as we shall see, distance often refers to a special class of dissimilarities. Dissimilarities sometimes fall in the interval $[0, 1]$, but it is also common for them to range from 0 to ∞.

Transformations

Transformations are often applied to convert a similarity to a dissimilarity, or vice versa, or to transform a proximity measure to fall within a particular range, such as [0,1]. For instance, we may have similarities that range from 1 to 10, but the particular algorithm or software package that we want to use may be designed to work only with dissimilarities, or it may work only with

similarities in the interval [0,1]. We discuss these issues here because we will employ such transformations later in our discussion of proximity. In addition, these issues are relatively independent of the details of specific proximity measures.

Frequently, proximity measures, especially similarities, are defined or transformed to have values in the interval [0,1]. Informally, the motivation for this is to use a scale in which a proximity value indicates the fraction of similarity (or dissimilarity) between two objects. Such a transformation is often relatively straightforward. For example, if the similarities between objects range from 1 (not at all similar) to 10 (completely similar), we can make them fall within the range $[0, 1]$ by using the transformation $s' = (s - 1)/9$, where s and s' are the original and new similarity values, respectively. In the more general case, the transformation of similarities to the interval $[0, 1]$ is given by the expression $s' = (s - min_s)/(max_s - min_s)$, where max_s and min_s are the maximum and minimum similarity values, respectively. Likewise, dissimilarity measures with a finite range can be mapped to the interval [0,1] by using the formula $d' = (d - min_d)/(max_d - min_d)$. This is an example of a linear transformation, which preserves the relative distances between points. In other words, if points, x_1 and x_2, are twice as far apart as points, x_3 and x_4, the same will be true after a linear transformation.

However, there can be complications in mapping proximity measures to the interval $[0, 1]$ using a linear transformation. If, for example, the proximity measure originally takes values in the interval [0,∞], then max_d is not defined and a nonlinear transformation is needed. Values will not have the same relationship to one another on the new scale. Consider the transformation $d' = d/(1 + d)$ for a dissimilarity measure that ranges from 0 to ∞. The dissimilarities 0, 0.5, 2, 10, 100, and 1000 will be transformed into the new dissimilarities 0, 0.33, 0.67, 0.90, 0.99, and 0.999, respectively. Larger values on the original dissimilarity scale are compressed into the range of values near 1, but whether this is desirable depends on the application.

Note that mapping proximity measures to the interval $[0, 1]$ can also change the meaning of the proximity measure. For example, correlation, which is discussed later, is a measure of similarity that takes values in the interval $[-1, 1]$. Mapping these values to the interval [0,1] by taking the absolute value loses information about the sign, which can be important in some applications. See Exercise 27 on page 131.

Transforming similarities to dissimilarities and vice versa is also relatively straightforward, although we again face the issues of preserving meaning and changing a linear scale into a nonlinear scale. If the similarity (or dissimilarity) falls in the interval [0,1], then the dissimilarity can be defined as $d = 1 - s$

($s = 1 - d$). Another simple approach is to define similarity as the negative of the dissimilarity (or vice versa). To illustrate, the dissimilarities 0, 1, 10, and 100 can be transformed into the similarities 0, -1, -10, and -100, respectively.

The similarities resulting from the negation transformation are not restricted to the range $[0, 1]$, but if that is desired, then transformations such as $s = \frac{1}{d+1}$, $s = e^{-d}$, or $s = 1 - \frac{d - min_d}{max_d - min_d}$ can be used. For the transformation $s = \frac{1}{d+1}$, the dissimilarities 0, 1, 10, 100 are transformed into 1, 0.5, 0.09, 0.01, respectively. For $s = e^{-d}$, they become 1.00, 0.37, 0.00, 0.00, respectively, while for $s = 1 - \frac{d - min_d}{max_d - min_d}$ they become 1.00, 0.99, 0.90, 0.00, respectively. In this discussion, we have focused on converting dissimilarities to similarities. Conversion in the opposite direction is considered in Exercise 28 on page 131.

In general, any monotonic decreasing function can be used to convert dissimilarities to similarities, or vice versa. Of course, other factors also must be considered when transforming similarities to dissimilarities, or vice versa, or when transforming the values of a proximity measure to a new scale. We have mentioned issues related to preserving meaning, distortion of scale, and requirements of data analysis tools, but this list is certainly not exhaustive.

2.4.2 Similarity and Dissimilarity between Simple Attributes

The proximity of objects with a number of attributes is typically defined by combining the proximities of individual attributes, and thus, we first discuss proximity between objects having a single attribute. Consider objects described by one nominal attribute. What would it mean for two such objects to be similar? Because nominal attributes convey only information about the distinctness of objects, all we can say is that two objects either have the same value or they do not. Hence, in this case similarity is traditionally defined as 1 if attribute values match, and as 0 otherwise. A dissimilarity would be defined in the opposite way: 0 if the attribute values match, and 1 if they do not.

For objects with a single ordinal attribute, the situation is more complicated because information about order should be taken into account. Consider an attribute that measures the quality of a product, e.g., a candy bar, on the scale {*poor*, *fair*, *OK*, *good*, *wonderful*}. It would seem reasonable that a product, P1, which is rated *wonderful*, would be closer to a product P2, which is rated *good*, than it would be to a product P3, which is rated *OK*. To make this observation quantitative, the values of the ordinal attribute are often mapped to successive integers, beginning at 0 or 1, e.g., {poor=0, fair=1, OK=2, $good = 3$, $wonderful = 4$}. Then, $d(\text{P1}, \text{P2}) = 3 - 2 = 1$ or, if we want the dissimilarity to fall between 0 and 1, $d(\text{P1}, \text{P2}) = \frac{3-2}{4} = 0.25$. A similarity for ordinal attributes can then be defined as $s = 1 - d$.

$s(\mathbf{x}, \mathbf{y})$ is the similarity between points $\mathbf{x}$ and $\mathbf{y}$, then the typical properties of similarities are the following:

1. $s(\mathbf{x}, \mathbf{y}) = 1$ only if $\mathbf{x} = \mathbf{y}$. $(0 \leq s \leq 1)$
2. $s(\mathbf{x}, \mathbf{y}) = s(\mathbf{y}, \mathbf{x})$ for all $\mathbf{x}$ and $\mathbf{y}$. (Symmetry)

There is no general analog of the triangle inequality for similarity measures. It is sometimes possible, however, to show that a similarity measure can easily be converted to a metric distance. The cosine and Jaccard similarity measures, which are discussed shortly, are two examples. Also, for specific similarity measures, it is possible to derive mathematical bounds on the similarity between two objects that are similar in spirit to the triangle inequality.

Example 2.15 (A Non-symmetric Similarity Measure)**.** Consider an experiment in which people are asked to classify a small set of characters as they flash on a screen. The **confusion matrix** for this experiment records how often each character is classified as itself, and how often each is classified as another character. Using the confusion matrix, we can define a similarity measure between a character x and a character y as the number of times that x is misclassified as y, but note that this measure is not symmetric. For example, suppose that "0" appeared 200 times and was classified as a "0" 160 times, but as an "o" 40 times. Likewise, suppose that "o" appeared 200 times and was classified as an "o" 170 times, but as "0" only 30 times. Then, s(0,o) = 40, but s(o, 0) = 30. In such situations, the similarity measure can be made symmetric by setting $s'(x, y) = s'(y, x) = (s(x, y) + s(y, x))/2$, where s' indicates the new similarity measure. ■

2.4.5 Examples of Proximity Measures

This section provides specific examples of some similarity and dissimilarity measures.

Similarity Measures for Binary Data

Similarity measures between objects that contain only binary attributes are called **similarity coefficients**, and typically have values between 0 and 1. A value of 1 indicates that the two objects are completely similar, while a value of 0 indicates that the objects are not at all similar. There are many rationales for why one coefficient is better than another in specific instances.

Let $\mathbf{x}$ and $\mathbf{y}$ be two objects that consist of n binary attributes. The comparison of two such objects, i.e., two binary vectors, leads to the following

four quantities (frequencies):

f_{00} = the number of attributes where **x** is 0 and **y** is 0
f_{01} = the number of attributes where **x** is 0 and **y** is 1
f_{10} = the number of attributes where **x** is 1 and **y** is 0
f_{11} = the number of attributes where **x** is 1 and **y** is 1

Simple Matching Coefficient One commonly used similarity coefficient is the **simple matching coefficient** (SMC), which is defined as

$$SMC = \frac{\text{number of matching attribute values}}{\text{number of attributes}} = \frac{f_{11} + f_{00}}{f_{01} + f_{10} + f_{11} + f_{00}}. \quad (2.4)$$

This measure counts both presences and absences equally. Consequently, the SMC could be used to find students who had answered questions similarly on a test that consisted only of true/false questions.

Jaccard Coefficient Suppose that **x** and **y** are data objects that represent two rows (two transactions) of a transaction matrix (see Section 2.1.2). If each asymmetric binary attribute corresponds to an item in a store, then a 1 indicates that the item was purchased, while a 0 indicates that the product was not purchased. Because the number of products not purchased by any customer far outnumbers the number of products that were purchased, a similarity measure such as SMC would say that all transactions are very similar. As a result, the Jaccard coefficient is frequently used to handle objects consisting of asymmetric binary attributes. The **Jaccard coefficient**, which is often symbolized by J, is given by the following equation:

$$J = \frac{\text{number of matching presences}}{\text{number of attributes not involved in 00 matches}} = \frac{f_{11}}{f_{01} + f_{10} + f_{11}}. \quad (2.5)$$

Example 2.16 (The SMC and Jaccard Similarity Coefficients)**.** To illustrate the difference between these two similarity measures, we calculate SMC and J for the following two binary vectors.

$\mathbf{x} = (1, 0, 0, 0, 0, 0, 0, 0, 0, 0)$
$\mathbf{y} = (0, 0, 0, 0, 0, 0, 1, 0, 0, 1)$

$f_{01} = 2$ the number of attributes where **x** was 0 and **y** was 1

$f_{10} = 1$ the number of attributes where $\mathbf{x}$ was 1 and $\mathbf{y}$ was 0
$f_{00} = 7$ the number of attributes where $\mathbf{x}$ was 0 and $\mathbf{y}$ was 0
$f_{11} = 0$ the number of attributes where $\mathbf{x}$ was 1 and $\mathbf{y}$ was 1

$$SMC = \frac{f_{11} + f_{00}}{f_{01} + f_{10} + f_{11} + f_{00}} = \frac{0 + 7}{2 + 1 + 0 + 7} = 0.7$$

$$J = \frac{f_{11}}{f_{01} + f_{10} + f_{11}} = \frac{0}{2 + 1 + 0} = 0$$

■

Cosine Similarity

Documents are often represented as vectors, where each component (attribute) represents the frequency with which a particular term (word) occurs in the document. Even though documents have thousands or tens of thousands of attributes (terms), each document is sparse since it has relatively few non-zero attributes. Thus, as with transaction data, similarity should not depend on the number of shared 0 values because any two documents are likely to "not contain" many of the same words, and therefore, if 0–0 matches are counted, most documents will be highly similar to most other documents. Therefore, a similarity measure for documents needs to ignores 0–0 matches like the Jaccard measure, but also must be able to handle non-binary vectors. The **cosine similarity**, defined next, is one of the most common measures of document similarity. If $\mathbf{x}$ and $\mathbf{y}$ are two document vectors, then

$$\cos(\mathbf{x}, \mathbf{y}) = \frac{\langle \mathbf{x}, \mathbf{y} \rangle}{\|\mathbf{x}\| \, \|\mathbf{y}\|} = \frac{\mathbf{x}'\mathbf{y}}{\|\mathbf{x}\| \, \|\mathbf{y}\|}, \tag{2.6}$$

where $'$ indicates vector or matrix transpose and $\langle \mathbf{x}, \mathbf{y} \rangle$ indicates the inner product of the two vectors,

$$\langle \mathbf{x}, \mathbf{y} \rangle = \sum_{k=1}^{n} x_k y_k = \mathbf{x}'\mathbf{y}, \tag{2.7}$$

and $\|\mathbf{x}\|$ is the length of vector $\mathbf{x}$, $\|\mathbf{x}\| = \sqrt{\sum_{k=1}^{n} x_k^2} = \sqrt{\langle \mathbf{x}, \mathbf{x} \rangle} = \sqrt{\mathbf{x}'\mathbf{x}}$.

The inner product of two vectors works well for asymmetric attributes since it depends only on components that are non-zero in both vectors. Hence, the similarity between two documents depends only upon the words that appear in both of them.

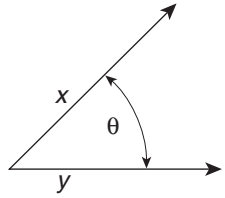


Figure 2.16. Geometric illustration of the cosine measure.

Example 2.17 (Cosine Similarity between Two Document Vectors)**.** This example calculates the cosine similarity for the following two data objects, which might represent document vectors:

$\mathbf{x} = (3, 2, 0, 5, 0, 0, 0, 2, 0, 0)$
$\mathbf{y} = (1, 0, 0, 0, 0, 0, 0, 1, 0, 2)$

$\langle \mathbf{x}, \mathbf{y} \rangle = 3 \times 1 + 2 \times 0 + 0 \times 0 + 5 \times 0 + 0 \times 0 + 0 \times 0 + 0 \times 0 + 2 \times 1 + 0 \times 0 + 0 \times 2 = 5$
$\|\mathbf{x}\| = \sqrt{3 \times 3 + 2 \times 2 + 0 \times 0 + 5 \times 5 + 0 \times 0 + 0 \times 0 + 0 \times 0 + 2 \times 2 + 0 \times 0 + 0 \times 0} = 6.48$
$\|\mathbf{y}\| = \sqrt{1 \times 1 + 0 \times 0 + 0 \times 0 + 0 \times 0 + 0 \times 0 + 0 \times 0 + 0 \times 0 + 1 \times 1 + 0 \times 0 + 2 \times 2} = 2.45$
$\mathbf{cos}(\mathbf{x}, \mathbf{y}) = \mathbf{0.31}$

■

As indicated by Figure 2.16, cosine similarity really is a measure of the (cosine of the) angle between $\mathbf{x}$ and $\mathbf{y}$. Thus, if the cosine similarity is 1, the angle between $\mathbf{x}$ and $\mathbf{y}$ is 0°, and $\mathbf{x}$ and $\mathbf{y}$ are the same except for length. If the cosine similarity is 0, then the angle between $\mathbf{x}$ and $\mathbf{y}$ is 90°, and they do not share any terms (words).

Equation 2.6 also can be written as Equation 2.8.

$$\cos(\mathbf{x}, \mathbf{y}) = \left\langle \frac{\mathbf{x}}{\|\mathbf{x}\|}, \frac{\mathbf{y}}{\|\mathbf{y}\|} \right\rangle = \langle \mathbf{x}', \mathbf{y}' \rangle, \tag{2.8}$$

where $\mathbf{x}' = \mathbf{x}/\|\mathbf{x}\|$ and $\mathbf{y}' = \mathbf{y}/\|\mathbf{y}\|$. Dividing $\mathbf{x}$ and $\mathbf{y}$ by their lengths normalizes them to have a length of 1. This means that cosine similarity does not take the *length* of the two data objects into account when computing similarity. (Euclidean distance might be a better choice when length is important.) For vectors with a length of 1, the cosine measure can be calculated by taking a simple inner product. Consequently, when many cosine similarities between objects are being computed, normalizing the objects to have unit length can reduce the time required.

Extended Jaccard Coefficient (Tanimoto Coefficient)

The extended Jaccard coefficient can be used for document data and that reduces to the Jaccard coefficient in the case of binary attributes. This coefficient, which we shall represent as EJ, is defined by the following equation:

$$EJ(\mathbf{x}, \mathbf{y}) = \frac{\langle \mathbf{x}, \mathbf{y} \rangle}{\|\mathbf{x}\|^2 + \|\mathbf{y}\|^2 - \langle \mathbf{x}, \mathbf{y} \rangle} = \frac{\mathbf{x}'\mathbf{y}}{\|\mathbf{x}\|^2 + \|\mathbf{y}\|^2 - \mathbf{x}'\mathbf{y}}. \tag{2.9}$$

Correlation

Correlation is frequently used to measure the linear relationship between two sets of values that are observed together. Thus, correlation can measure the relationship between two variables (height and weight) or between two objects (a pair of temperature time series). Correlation is used much more frequently to measure the similarity between attributes since the values in two data objects come from different attributes, which can have very different attribute types and scales. There are many types of correlation, and indeed correlation is sometimes used in a general sense to mean the relationship between two sets of values that are observed together. In this discussion, we will focus on a measure appropriate for numerical values.

Specifically, **Pearson's correlation** between two sets of numerical values, i.e., two vectors, $\mathbf{x}$ and $\mathbf{y}$, is defined by the following equation:

$$\text{corr}(\mathbf{x}, \mathbf{y}) = \frac{\text{covariance}(\mathbf{x}, \mathbf{y})}{\text{standard_deviation}(\mathbf{x}) \times \text{standard_deviation}(\mathbf{y})} = \frac{s_{xy}}{s_x \; s_y}, \tag{2.10}$$

where we use the following standard statistical notation and definitions:

$$\text{covariance}(\mathbf{x}, \mathbf{y}) = s_{xy} = \frac{1}{n-1} \sum_{k=1}^{n} (x_k - \overline{x})(y_k - \overline{y}) \tag{2.11}$$

$$\text{standard_deviation}(\mathbf{x}) = s_x = \sqrt{\frac{1}{n-1} \sum_{k=1}^{n} (x_k - \overline{x})^2}$$

$$\text{standard_deviation}(\mathbf{y}) = s_y = \sqrt{\frac{1}{n-1} \sum_{k=1}^{n} (y_k - \overline{y})^2}$$

$$\overline{x} = \frac{1}{n}\sum_{k=1}^{n} x_k \text{ is the mean of } \mathbf{x}$$
$$\overline{y} = \frac{1}{n}\sum_{k=1}^{n} y_k \text{ is the mean of } \mathbf{y}$$

Example 2.18 (Perfect Correlation)**.** Correlation is always in the range -1 to 1. A correlation of 1 (-1) means that $\mathbf{x}$ and $\mathbf{y}$ have a perfect positive (negative) linear relationship; that is, $x_k = ay_k + b$, where a and b are constants. The following two vectors $\mathbf{x}$ and $\mathbf{y}$ illustrate cases where the correlation is -1 and $+1$, respectively. In the first case, the means of $\mathbf{x}$ and $\mathbf{y}$ were chosen to be 0, for simplicity.

$\mathbf{x} = (-3,\ \ 6,\ 0,\ \ 3, -6)$
$\mathbf{y} = (\ \ 1, -2,\ 0, -1,\ \ 2)$
$corr(\mathbf{x}, \mathbf{y}) = -1 \quad x_k = -3y_k$

$\mathbf{x} = (3, 6, 0, 3, 6)$
$\mathbf{y} = (1, 2, 0, 1, 2)$
$corr(\mathbf{x}, \mathbf{y}) = 1 \quad x_k = 3y_k$ ■

Example 2.19 (Nonlinear Relationships)**.** If the correlation is 0, then there is no linear relationship between the two sets of values. However, nonlinear relationships can still exist. In the following example, $y_k = x_k^2$, but their correlation is 0.

$\mathbf{x} = (-3, -2, -1,\ 0,\ 1,\ 2,\ 3)$
$\mathbf{y} = (\ \ 9,\ \ \ 4,\ \ \ 1,\ 0,\ 1,\ 4,\ 9)$

■

Example 2.20 (Visualizing Correlation)**.** It is also easy to judge the correlation between two vectors $\mathbf{x}$ and $\mathbf{y}$ by plotting pairs of corresponding values of $\mathbf{x}$ and $\mathbf{y}$ in a scatter plot. Figure 2.17 shows a number of these scatter plots when $\mathbf{x}$ and $\mathbf{y}$ consist of a set of 30 pairs of values that are randomly generated (with a normal distribution) so that the correlation of $\mathbf{x}$ and $\mathbf{y}$ ranges from -1 to 1. Each circle in a plot represents one of the 30 pairs of $\mathbf{x}$ and $\mathbf{y}$ values; its x coordinate is the value of that pair for $\mathbf{x}$, while its y coordinate is the value of the same pair for $\mathbf{y}$. ■

If we transform $\mathbf{x}$ and $\mathbf{y}$ by subtracting off their means and then normalizing them so that their lengths are 1, then their correlation can be calculated by taking the dot product. Let us refer to these transformed vectors of $\mathbf{x}$ and $\mathbf{y}$ as $\mathbf{x}'$ and $\mathbf{y}'$, respectively. (Notice that this transformation is not the same

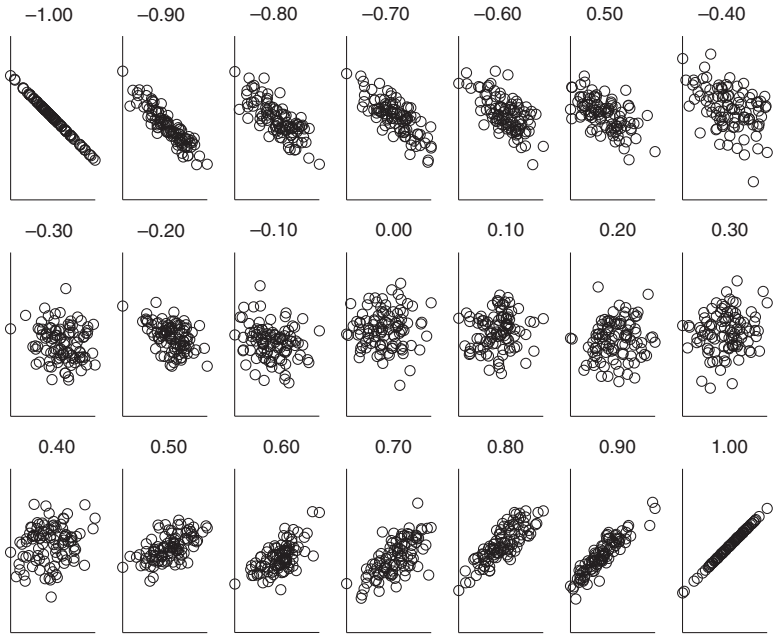


Figure 2.17. Scatter plots illustrating correlations from -1 to 1.

as the standardization used in other contexts, where we subtract the means and divide by the standard deviations, as discussed in Section 2.3.7.) This transformation highlights an interesting relationship between the correlation measure and the cosine measure. Specifically, the correlation between $\mathbf{x}$ and $\mathbf{y}$ is identical to the cosine between $\mathbf{x}'$ and $\mathbf{y}'$. However, the cosine between $\mathbf{x}$ and $\mathbf{y}$ is not the same as the cosine between $\mathbf{x}'$ and $\mathbf{y}'$, even though they both have the same correlation measure. In general, the correlation between two vectors is equal to the cosine measure only in the special case when the means of the two vectors are 0.

Differences Among Measures For Continuous Attributes

In this section, we illustrate the difference among the three proximity measures for continuous attributes that we have just defined: cosine, correlation, and Minkowski distance. Specifically, we consider two types of data transformations that are commonly used, namely, scaling (multiplication) by a constant factor and translation (addition) by a constant value. A proximity measure is considered to be invariant to a data transformation if its value remains unchanged even after performing the transformation. Table 2.12 compares the

Table 2.12. Properties of cosine, correlation, and Minkowski distance measures.

Property	Cosine	Correlation	Minkowski Distance
Invariant to scaling (multiplication)	Yes	Yes	No
Invariant to translation (addition)	No	Yes	No

behavior of cosine, correlation, and Minkowski distance measures regarding their invariance to scaling and translation operations. It can be seen that while correlation is invariant to both scaling and translation, cosine is only invariant to scaling but not to translation. Minkowski distance measures, on the other hand, are sensitive to both scaling and translation and are thus invariant to neither.

Let us consider an example to demonstrate the significance of these differences among different proximity measures.

Example 2.21 (Comparing proximity measures)**.** Consider the following two vectors $\mathbf{x}$ and $\mathbf{y}$ with seven numeric attributes.

$\mathbf{x} = (1, 2, 4, 3, 0, 0, 0)$
$\mathbf{y} = (1, 2, 3, 4, 0, 0, 0)$

It can be seen that both $\mathbf{x}$ and $\mathbf{y}$ have 4 non-zero values, and the values in the two vectors are mostly the same, except for the third and the fourth components. The cosine, correlation, and Euclidean distance between the two vectors can be computed as follows.

$$\begin{aligned} \mathbf{cos}(\mathbf{x}, \mathbf{y}) &= \frac{29}{\sqrt{30} \times \sqrt{30}} &&= 0.9667 \\ \mathbf{correlation}(\mathbf{x}, \mathbf{y}) &= \frac{2.3571}{1.5811 \times 1.5811} &&= 0.9429 \\ \mathbf{Euclidean\ distance}(\mathbf{x}, \mathbf{y}) &= ||\mathbf{x} - \mathbf{y}|| &&= 1.4142 \end{aligned}$$

Not surprisingly, $\mathbf{x}$ and $\mathbf{y}$ have a cosine and correlation measure close to 1, while the Euclidean distance between them is small, indicating that they are quite similar. Now let us consider the vector $\mathbf{y_s}$, which is a scaled version of $\mathbf{y}$ (multiplied by a constant factor of 2), and the vector $\mathbf{y_t}$, which is constructed by translating $\mathbf{y}$ by 5 units as follows.

$\mathbf{y_s} = 2 \times \mathbf{y} = (2, 4, 6, 8, 0, 0, 0)$

Table 2.13. Similarity between $(\mathbf{x}, \mathbf{y})$, $(\mathbf{x}, \mathbf{y_s})$, and $(\mathbf{x}, \mathbf{y_t})$.

Measure	$(\mathbf{x}, \mathbf{y})$	$(\mathbf{x}, \mathbf{y_s})$	$(\mathbf{x}, \mathbf{y_t})$
Cosine	0.9667	0.9667	0.7940
Correlation	0.9429	0.9429	0.9429
Euclidean Distance	1.4142	5.8310	14.2127

$\mathbf{y_t} = \mathbf{y} + 5 = (6, 7, 8, 9, 5, 5, 5)$

We are interested in finding whether $\mathbf{y_s}$ and $\mathbf{y_t}$ show the same proximity with $\mathbf{x}$ as shown by the original vector $\mathbf{y}$. Table 2.13 shows the different measures of proximity computed for the pairs $(\mathbf{x}, \mathbf{y})$, $(\mathbf{x}, \mathbf{y_s})$, and $(\mathbf{x}, \mathbf{y_t})$. It can be seen that the value of correlation between $\mathbf{x}$ and $\mathbf{y}$ remains unchanged even after replacing $\mathbf{y}$ with $\mathbf{y_s}$ or $\mathbf{y_t}$. However, the value of cosine remains equal to 0.9667 when computed for $(\mathbf{x}, \mathbf{y})$ and $(\mathbf{x}, \mathbf{y_s})$, but significantly reduces to 0.7940 when computed for $(\mathbf{x}, \mathbf{y_t})$. This highlights the fact that cosine is invariant to the scaling operation but not to the translation operation, in contrast with the correlation measure. The Euclidean distance, on the other hand, shows different values for all three pairs of vectors, as it is sensitive to both scaling and translation.

We can observe from this example that different proximity measures behave differently when scaling or translation operations are applied on the data. The choice of the right proximity measure thus depends on the desired notion of similarity between data objects that is meaningful for a given application. For example, if $\mathbf{x}$ and $\mathbf{y}$ represented the frequencies of different words in a document-term matrix, it would be meaningful to use a proximity measure that remains unchanged when $\mathbf{y}$ is replaced by $\mathbf{y_s}$, because $\mathbf{y_s}$ is just a scaled version of $\mathbf{y}$ with the same distribution of words occurring in the document. However, $\mathbf{y_t}$ is different from $\mathbf{y}$, since it contains a large number of words with non-zero frequencies that do not occur in $\mathbf{y}$. Because cosine is invariant to scaling but not to translation, it will be an ideal choice of proximity measure for this application.

Consider a different scenario in which $\mathbf{x}$ represents a location's temperature measured on the Celsius scale for seven days. Let $\mathbf{y}$, $\mathbf{y_s}$, and $\mathbf{y_t}$ be the temperatures measured on those days at a different location, but using three different measurement scales. Note that different units of temperature have different offsets (e.g., Celsius and Kelvin) and different scaling factors (e.g., Celsius and Fahrenheit). It is thus desirable to use a proximity measure that captures the proximity between temperature values without being affected by

9

Anomaly Detection

In anomaly detection, the goal is to find objects that do not conform to normal patterns or behavior. Often, anomalous objects are known as **outliers**, since, on a scatter plot of the data, they lie far away from other data points. Anomaly detection is also known as **deviation detection**, because anomalous objects have attribute values that deviate significantly from the expected or typical attribute values, or as **exception mining**, because anomalies are exceptional in some sense. In this chapter, we will mostly use the terms *anomaly* or *outlier*. There are a variety of anomaly detection approaches from several areas, including statistics, machine learning, and data mining. All try to capture the idea that an anomalous data object is unusual or in some way inconsistent with other objects.

Although unusual objects or events are, by definition, relatively rare, their detection and analysis provides critical insights that are useful in a number of applications. The following examples illustrate applications for which anomalies are of considerable interest.

- **Fraud Detection.** The purchasing behavior of someone who steals a credit card is often different from that of the original owner. Credit card companies attempt to detect a theft by looking for buying patterns that characterize theft or by noticing a change from typical behavior. Similar approaches are relevant in many domains such as detecting insurance claim fraud and insider trading.

- **Intrusion Detection.** Unfortunately, attacks on computer systems and computer networks are commonplace. While some of these attacks, such as those designed to disable or overwhelm computers and networks, are obvious, other attacks, such as those designed to secretly gather

information, are difficult to detect. Many of these intrusions can only be detected by monitoring systems and networks for unusual behavior.

- **Ecosystem Disturbances.** The Earth's ecosystem has been experiencing rapid changes in the last few decades due to natural or anthropogenic reasons. This includes an increased propensity for extreme events, such as heat waves, droughts, and floods, which have a huge impact on the environment. Identifying such extreme events from sensor recordings and satellite images is important for understanding their origins and behavior, as well as for devising sustainable adaptation policies.

- **Medicine and Public Health.** For a particular patient, unusual symptoms or test results, such as an anomalous MRI scan, may indicate potential health problems. However, whether a particular test result is anomalous may depend on many other characteristics of the patient, such as age, sex, and genetic makeup. Furthermore, the categorization of a result as anomalous or not incurs a cost—unneeded additional tests if a patient is healthy and potential harm to the patient if a condition is left undiagnosed and untreated. The detection of emerging disease outbreaks, such as H1N1-influenza or SARS, which result in unusual and alarming test results in a series of patients, is also important for monitoring the spread of diseases and taking preventive actions.

- **Aviation Safety.** Since aircrafts are highly complex and dynamic systems, they are prone to accidents—often with drastic consequences—due to mechanical, environmental or human factors. To monitor the occurrence of such anomalies, most commercial airplanes are equipped with a large number of sensors to measure different flight parameters, such as information from the control system, the avionics and propulsion systems, and pilot actions. Identifying abnormal events in these sensor recordings (e.g., an anomalous sequence of pilot actions or an abnormally functioning aircraft component) can help prevent aircraft accidents and promote aviation safety.

Although much of the recent interest in anomaly detection is driven by applications in which anomalies are the focus, historically, anomaly detection (and removal) has been viewed as a data preprocessing technique to eliminate erroneous data objects that may be recorded because of human error, a problem with the measuring device, or the presence of noise. Such anomalies provide no interesting information but only distort the analysis of normal objects. The identification and removal of such erroneous data objects is not

the focus of this chapter. Instead, the emphasis is on detecting anomalous objects that are interesting in their own right.

9.1 Characteristics of Anomaly Detection Problems

Anomaly detection problems are quite diverse in nature as they appear in multiple application domains under different settings. This diversity in problem characteristics has resulted in a rich variety of anomaly detection methods that are useful in different situations. Before we discuss these methods, it will be useful to describe some of the key characteristics of anomaly detection problems that motivate the different styles of anomaly detection methods.

9.1.1 A Definition of an Anomaly

An important characteristic of an anomaly detection problem is the way an anomaly is defined. Since anomalies are rare occurrences that are not fully understood, they can be defined in different ways depending on the problem requirements. However, the following high-level definition of an anomaly encompasses most of the definitions commonly employed.

Definition 9.1. An **anomaly** is an observation that doesn't fit the distribution of the data for normal instances, i.e., is unlikely under the distribution of the majority of instances.

We note the following points:

- This definition does not assume that the distribution is easy to express in terms of well-known statistical distributions. Indeed, the difficulty of doing so is the reason that many anomaly detection approaches use non-statistical approaches. Nonetheless, these approaches aim to find data objects that are not common.

- Conceptually, we can rank data objects according to the probability of seeing such an object or something more extreme. The lower the probability, the more likely the object is an anomaly. Often, the reciprocal of the probability is used as a ranking score. Again, this is only practical in some cases. Such approaches are discussed in Section 9.3.

- There can be various causes of an anomaly: noise, the object comes from another distribution, e.g., a few grapefruit mixed with oranges, or the object is just a rare occurrence of data from the distribution, e.g., a 7

foot tall person. As mentioned, we are not interested in anomalies due to noise.

9.1.2 Nature of Data

The nature of the input data plays a key role in deciding the choice of a suitable anomaly detection technique. Some of the common characteristics of the input data include the number and types of attributes, and the representation used for describing every data instance.

Univariate or Multivariate If the data contains a single attribute, the question of whether an object is anomalous depends on whether the object's value for that attribute is anomalous. However, if the data objects are represented using many attributes, a data object may have anomalous values for some attributes but ordinary values for other attributes. Furthermore, an object may be anomalous even if none of its attribute values are individually anomalous. For example, it is common to have people who are two feet tall (children) or are 100 pounds in weight, but uncommon to have a two-foot tall person who weighs 100 pounds. Identifying an anomaly in a multivariate setting is thus challenging, particularly when the dimensionality of the data is high.

Record Data or Proximity Matrix The most common approach for representing a data set is to use record data or its variants, e.g., a data matrix, where every data instance is described using the same set of attributes. However, for the purpose of anomaly detection, it is often sufficient to know how *different* an instance is in comparison to other instances. Hence, some anomaly detection methods work with a different representation of the input data known as a **proximity matrix**, where every entry in the matrix denotes the pairwise proximity (similarity or dissimilarity) between two instances. Note that a data matrix can always be converted to a proximity matrix by using an appropriate proximity measure. Also, a similarity matrix can be easily converted to a distance matrix using any of the transformations presented in Section 2.4.1.

Availability of Labels The label of a data instance denotes whether the instance is normal or anomalous. If we have a training set with labels for every data instance, then the problem of anomaly detection translates to a **supervised** learning (classification) problem. Classification techniques that

address the so-called rare class problem are particularly relevant because anomalies are relatively rare with respect to normal objects. See Section 6.11.

However, in most practical applications, we do not have a training set with accurate and representative labels of the normal and anomalous classes. Note that obtaining labels of the anomalous class is especially challenging because of their rarity. It is thus difficult for a human expert to catalog every type of anomaly since the properties of the anomalous class are often unknown. Hence, most anomaly detection problems are **unsupervised** in nature, i.e., the input data does not have any labels. All anomaly detection methods presented in this chapter operate in the unsupervised setting.

Note that in the absence of labels, it is challenging to differentiate anomalies from normal instances given an input data set. However, anomalies typically have some properties that techniques can take advantage of to make finding anomalies practical. Two key properties are the following:

Relatively Small in Number Since anomalies are infrequent, most input data sets have a predominance of normal instances. The input data set is thus often used as an imperfect representation of the normal class in most anomaly detection techniques. However, the performance of such methods needs to be robust to the presence of outliers in the input data. Some anomaly detection methods also provide a mechanism to specify the expected number of outliers in the input data. Such methods can work with a larger number of anomalies in the data.

Sparsely Distributed Anomalies, unlike normal objects, are often unrelated to each other and hence distributed sparsely in the space of attributes. Indeed, the successful operation of most anomaly detection methods depends on anomalies not being tightly clustered. However, some anomaly detection methods are specifically designed to find clustered anomalies (see Section 9.5.1), which are assumed to either be small in size or distant from other instances.

9.1.3 How Anomaly Detection is Used

There are two different ways in which any generic anomaly detection method can be used. In the first approach, we are given an input data that contains both normal and anomalous instances, and are required to identify anomalies in this input data. All anomaly detection approaches presented in this chapter are able to operate in this setup. In the second approach, we are also provided with test instances (appearing one at a time) that need to be identified

as anomalies. Most anomaly detection methods (with a few exceptions) are able to use the input data set to provide outputs on new test instances. Finding anomalies by finding anomalous clusters—Section 9.5.1—is one of the exceptions.

9.2 Characteristics of Anomaly Detection Methods

To cater to the diverse needs of anomaly detection problems, a number of techniques have been explored using concepts from different research disciplines. In this section, we provide a high-level description of some of the common characteristics of anomaly detection methods that are helpful in understanding their commonalities and differences.

Model-based vs. Model-free

Many approaches for anomaly detection use the input data to build *models* that can be used to identify whether a test instance is anomalous or not. Most **model-based** techniques for anomaly detection build a model of the normal class and identify anomalies that do not fit this model. For example, we can fit a Gaussian distribution to model the normal class and then identify anomalies that do not conform well to the learned distribution. The other type of model-based techniques learns a model of both the normal and anomalous classes, and identifies instances as anomalies if they are more likely to belong to the anomalous class. Although these approaches technically require representative labels from both classes, they often make assumptions about the nature of the anomalous class, e.g., that anomalies are rare and sparsely distributed, and thus can work even in an unsupervised setting.

In addition to identifying anomalies, model-based methods provide information about the nature the normal class and sometimes even the anomalous class. However, the assumptions they make about the properties of normal and anomalous classes may not hold true in every problem. In contrast, **model-free** approaches do not explicitly characterize the distribution of the normal or anomalous classes. Instead, they directly identify instances as anomalies without learning models from the input data. For example, an instance can be identified as an anomaly if it is quite different from other instances in its neighborhood. Model-free approaches are often intuitive and simple to use.

Global vs. Local Perspective

An instance can be identified as an anomaly either by considering the global context, e.g., by building a model over all normal instances and using this global model for anomaly detection, or by considering the local perspective of every data instance. Specifically, an anomaly detection approach is termed **local** if its output on a given instance does not change if instances outside its local neighborhood are modified or removed. The difference between the global and local perspective can result in significant differences in the results of an anomaly detection method, because an object may seem unusual with respect to all objects globally, but not with respect to objects in its local neighborhood. For example, a person whose height is 6 feet 5 inches is unusually tall with respect to the general population, but not with respect to professional basketball players.

Label vs. Score

Different approaches for anomaly detection produce their outputs in different formats. The most basic type of output is a binary **anomaly label**: an object is either identified as an anomaly or as a normal instance. However, labels do not provide any information about the degree to which an instance is anomalous. Frequently, some of the detected anomalies are more extreme than others, while some instances labeled as normal may be on the verge of being identified as anomalies.

Hence, many anomaly detection methods produce an **anomaly score** that indicates how strongly an instance is likely to be an anomaly. An anomaly score can easily be sorted and converted into ranks, so that an analyst can be provided with only the top-most scoring anomalies. Alternatively, a cutoff threshold can be applied to an anomaly score to obtain binary anomaly labels. The task of choosing the right threshold is often left to the discretion of the analyst. However, sometimes the scores have an associated meaning, e.g., statistical significance (see Section 9.3), which makes the analysis of anomalies easier and more interpretable.

In the following sections, we provide brief descriptions of six types of anomaly detection approaches. For each type, we will describe their basic idea, key features, and underlying assumptions using illustrative examples. At the end of every section, we also discuss their strengths and weakness in handling different aspects of anomaly detection problems. To follow common practice, we will use the terms outlier and anomaly interchangeably in the remainder of this chapter.

9.3 Statistical Approaches

Statistical approaches make use of probability distributions (e.g., the Gaussian distribution) to model the normal class. A key feature of such distributions is that they associate a probability value to every data instance, indicating how likely it is for the instance to be generated from the distribution. Anomalies are then identified as instances that are unlikely to be generated from the probability distribution of the normal class.

There are two types of models that can be used to represent the probability distribution of the normal class: parametric models and non-parametric models. While parametric models use well-known families of statistical distributions that require estimating parameters from the data, non-parametric models are more flexible and learn the distribution of the normal class directly from the available data. In the following, we discuss both of these types of models for anomaly detection.

9.3.1 Using Parametric Models

Some of the common types of parametric models that are widely used for describing many types of data sets, include the Gaussian distribution, the Poisson distribution, and the binomial distribution. They involve parameters that need to be learned from the data, e.g., a Gaussian distribution requires identifying the mean and variance parameters from the data.

Parametric models are quite effective in representing the behavior of the normal class, especially when the normal class is known to follow a specific distribution. The anomaly scores computed by parametric models also have strong theoretical properties, which can be used for analyzing the anomaly scores and assessing their statistical significance. In the following, we discuss the use of the Gaussian distribution for modeling the normal class, in the univariate and multivariate settings.

Using the Univariate Gaussian Distribution

The Gaussian (normal) distribution is one of the most frequently used distributions in statistics, and we will use it to describe a simple approach to statistical outlier detection. The Gaussian distribution has two parameters, μ and σ, which are the mean and standard deviation, respectively, and is represented using the notation $N(\mu, \sigma)$. The probability density function $f(x)$ of a point x under the Gaussian distribution is given as

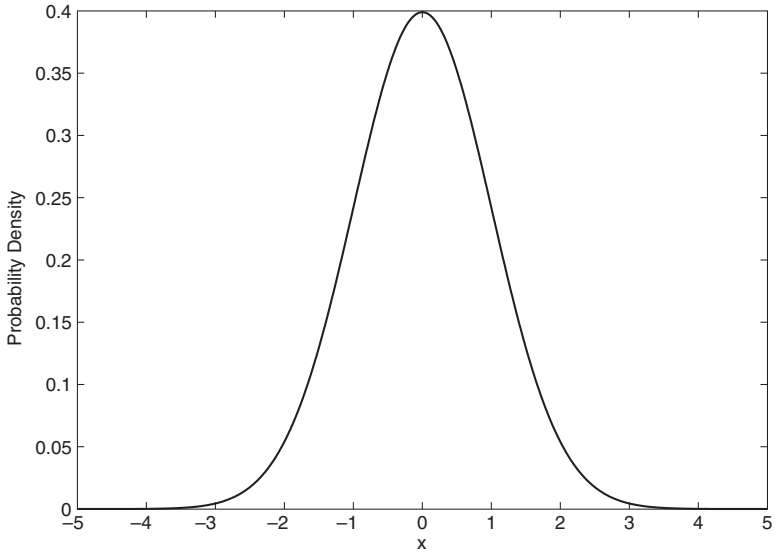


Figure 9.1. Probability density function of a Gaussian distribution with a mean of 0 and a standard deviation of 1.

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}} \tag{9.1}$$

Figure 9.1 shows the probability density function of $N(0, 1)$. We can see that the $p(x)$ declines as x moves farther from the center of the distribution. We can thus use the distance of a point x from the origin as an anomaly score. As we will see later in Section 9.3.4, this distance value has an interpretation in terms of probability that can be used to assess the confidence in calling x as an outlier.

If the attribute of interest x follows a Gaussian distribution with mean μ and standard deviation σ, i.e., $N(\mu, \sigma)$, a common approach is to transform the attribute x to a new attribute z, which has a $N(0, 1)$ distribution. This can be done by using $z = (x - \mu)/\sigma$, which is called the z-score. Note that z^2 is directly related to the probability density of the point x in Equation 9.1 since that equation can be rewritten as follows:

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-z^2/2} \tag{9.2}$$

The parameters μ and σ of the Gaussian distribution can be estimated from the training data of mostly normal instances, by using the sample mean

$\overline{x}$ as μ and the sample standard deviation s_x as σ. However, if we believe the outliers are distorting the estimates of these parameters too much, more robust estimates of these quantities can be used—see Bibliographic Notes.

Using the Multivariate Gaussian Distribution

For a data set comprised of more than one continuous attribute, we can use a multivariate Gaussian distribution to model the normal class. A multivariate Gaussian distribution $N(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ involves two parameters, the mean vector $\boldsymbol{\mu}$ and the covariance matrix $\boldsymbol{\Sigma}$, which need to be estimated from the data. The probability density of a point $\mathbf{x}$ distributed as $N(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ is given by

$$f(\mathbf{x}) = \frac{1}{(\sqrt{2\pi})^m |\boldsymbol{\Sigma}|^{1/2}} \, e^{-\frac{(\mathbf{x}-\boldsymbol{\mu})\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})}{2}}, \tag{9.3}$$

where p is the number of dimensions of $\mathbf{x}$ and $|\boldsymbol{\Sigma}|$ denotes the determinant of the covariance matrix $\boldsymbol{\Sigma}$.

In the case of a multivariate Gaussian distribution, the distance of a point $\mathbf{x}$ from the center μ cannot be directly used as a viable anomaly score. This is because a multivariate normal distribution is not symmetrical with respect to its center if there are correlations between the attributes. To illustrate this, Figure 9.2 shows the probability density of a two-dimensional multivariate Gaussian distribution with mean of (0,0) and a covariance matrix of

$$\boldsymbol{\Sigma} = \begin{pmatrix} 1.00 & 0.75 \\ 0.75 & 3.00 \end{pmatrix}.$$

The probability density varies asymmetrically as we move outward from the center in different directions. To account for this fact, we need a distance measure that takes the shape of the data into consideration. The Mahalanobis distance is one such distance measure. (See Equation 2.26 on page 116.) The Mahalanobis distance between a point $\mathbf{x}$ and the mean of the data $\overline{\mathbf{x}}$ is given by

$$Mahalanobis(\mathbf{x}, \overline{\mathbf{x}}) = (\mathbf{x} - \overline{\mathbf{x}})\mathbf{S}^{-1}(\mathbf{x} - \overline{\mathbf{x}})^T, \tag{9.4}$$

where $\mathbf{S}$ is the estimated covariance matrix of the data. Note that the Mahalanobis distance between $\mathbf{x}$ and $\overline{\mathbf{x}}$ is directly related to the probability density of $\mathbf{x}$ in Equation 9.3, when $\overline{\mathbf{x}}$ and $\mathbf{S}$ are used as estimates of $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$, respectively. (See Exercise 9 on page 771.)

Example 9.1 (Outliers in a Multivariate Normal Distribution)**.** Figure 9.3 shows the Mahalanobis distance (from the mean of the distribution) for points in a two-dimensional data set. The points A $(-4, 4)$ and B $(5, 5)$ are outliers

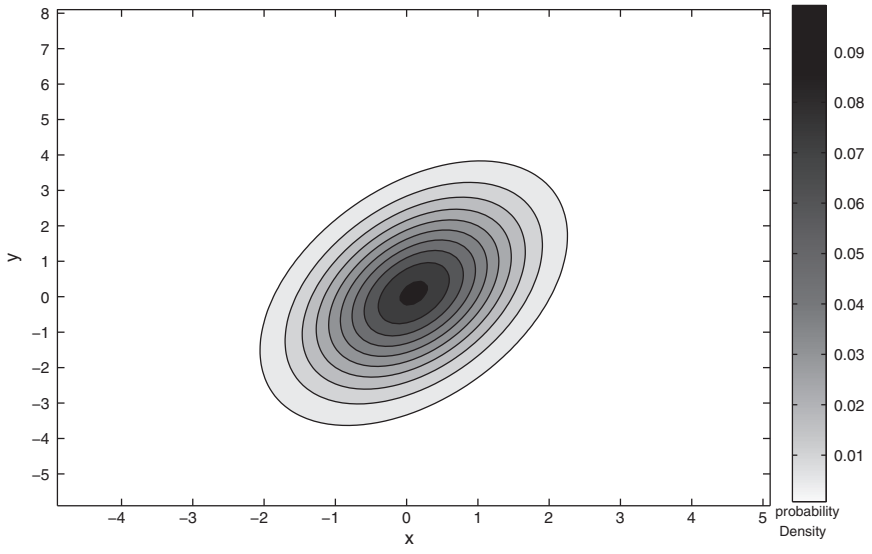


Figure 9.2. Probability density of points for the Gaussian distribution used to generate the points of Figure 9.3.

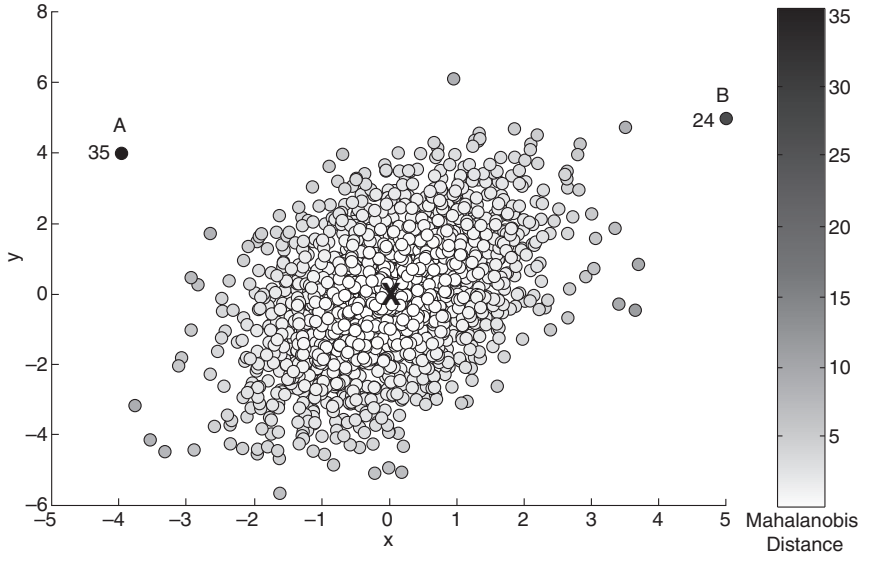


Figure 9.3. Mahalanobis distance of points from the center of a two-dimensional set of 2002 points.

that were added to the data set, and their Mahalanobis distance is indicated in the figure. The other 2000 points of the data set were randomly generated using the distribution used for Figure 9.2.

Both A and B have large Mahalanobis distances. However, even though A is closer to the center (the large black x at (0,0)) as measured by Euclidean distance, it is farther away than B in terms of the Mahalanobis distance because the Mahalanobis distance takes the shape of the distribution into account. In particular, point B has a Euclidean distance of $5\sqrt{2}$ and a Mahalanobis distance of 24, while the point A has a Euclidean distance of $4\sqrt{2}$ and a Mahalanobis distance of 35. ■

The above approaches assume that the normal class is generated from a single Gaussian distribution. Note that this may not always be the case, especially if there are multiple types of normal classes that have different means and variances. In such cases, we can use a Gaussian mixture model (as described in Chapter 8.2.2) to represent the normal class. For each point, the smallest Mahalanobis distance of the point to any of the Gaussian distributions is computed and used as the anomaly score. This approach is related to the clustering-based approaches for anomaly detection, which will be described in Section 9.5.

9.3.2 Using Non-parametric Models

An alternative for modeling the distribution of the normal class is to use kernel density estimation-based techniques that employ kernel functions (described in Section 8.3.3) to approximate the density of the normal class from the available data. This results in the construction of a non-parametric probability distribution of the normal class, such that regions with a dense occurrence of normal instances have high probability and vice-versa. Note that kernel-based approaches do not assume that the data conforms to any known family of distributions but instead derive the distribution purely from the data. Having learned a probability density for the normal class using the kernel density approach, the anomaly score of an instance is computed as the inverse of its probability with respect to the learned density.

A simpler non-parametric approach to modeling the normal class is to build a histogram of the normal data. For example, if the data contains a single continuous attribute, then we can construct bins for different ranges of the attribute, using the equal-width discretization technique described in Section 2.3.6. We can then check if a new test instance falls in any of the bins of the histogram. If it does not fall in any of the bins, we can identify it as

an anomaly. Otherwise, we can use the inverse of the height (frequency) of the bin in which it falls as its anomaly score. This approach is known as the frequency-based or counting-based approach for anomaly detection.

A key step in using frequency-based approaches for anomaly detection is choosing the size of the bin used for constructing the histogram. A small bin size can falsely identify many normal instances as anomalous, since they might fall in empty or sparsely populated bins. On the other hand, if the bin size is too large, many anomalous instances may fall in heavily populated bins and go unnoticed. Thus, choosing the right bin size is challenging, and often requires trying multiple size options or using expert knowledge.

9.3.3 Modeling Normal and Anomalous Classes

The statistical approaches described so far only model the distribution of the normal class but not the anomalous class. They assume that the training set predominantly has normal instances. However, if there are outliers present in the training data, which is common in most practical applications, the learning of the probability distributions corresponding to the normal class may be distorted, resulting in poor identification of anomalies.

Here, we present a statistical approach for anomaly detection that can tolerate a considerable fraction (λ) of outliers in the training set, provided that the outliers are uniformly distributed (and thus not clustered) in the attribute space. This approach makes use of a mixture modeling technique to learn the distribution of normal and anomalous classes. This approach is similar to the Expectation-Maximization (EM) based technique introduced in the context of clustering in Chapter 8.2.2. Note that λ), the fraction of outliers, is like a prior.

The basic idea of this approach is to assume that instances are generated with probability λ from the anomalous class, which has uniform distribution, p_A, and with probability $1 - \lambda$ and from the normal class, which has the distribution, $f_M(\theta)$, where θ represents the parameters of the distribution. The approach for assigning training instances to the normal and anomaly classes can be described as follows. Initially, all the objects are assigned to the normal class and the set of anomalous objects is empty. At every iteration of the EM algorithm, objects are transferred from the normal class to the anomaly class to improve the likelihood of the overall data. Let M_t and A_t be the set of normal and anomalous objects, respectively, at iteration t. The likelihood of the data set D, $\mathcal{L}_t(D)$, and its log-likelihood, $\log\mathcal{L}_t(D)$, are then given by the following equations:

$$\mathcal{L}_t(D) = \prod_{\mathbf{x}_i \in D} P(\mathbf{x}_i) = \left((1-\lambda)^{|M_t|} \prod_{\mathbf{x}_i \in M_t} P_M(\mathbf{x}_i, \theta_t)\right)\left(\lambda^{|A_t|} \prod_{\mathbf{x}_i \in A_t} P_A(\mathbf{x}_i)\right) \quad (9.5)$$

$$\log\mathcal{L}_t(D) = |M_t|\log(1-\lambda) + \sum_{\mathbf{x}_i \in M_t} \log P_M(\mathbf{x}_i, \theta_t) + |A_t|\log\lambda + \sum_{\mathbf{x}_i \in A_t} \log P_A(\mathbf{x}_i) \quad (9.6)$$

where $|M_t|$ and $|A_t|$ are the number of objects in the normal and anomaly classes, respectively, and θ_t represents the parameters of the distribution of the normal class, which can be estimated using M_t. If the transfer of an object $\mathbf{x}$ from M_t to A_t results in a significant increase in the log-likelihood of the data (greater than a threshold c), then $\mathbf{x}$ is assigned to the set of outliers A_t. The set of outliers A_t keeps growing till we achieve the maximum likelihood of the data using M_t and A_t. This approach is summarized in Algorithm 9.1.

Algorithm 9.1 Likelihood-based outlier detection.

1: Initialization: At time $t = 0$, let M_t contain all the objects, while A_t is empty.
2: **for** each object $\mathbf{x}$ that belongs to M_t **do**
3: Move $\mathbf{x}$ from M_t to A_t to produce the new data sets A_{t+1} and M_{t+1}.
4: Compute the new log-likelihood of D, $\log\mathcal{L}_{t+1}(D)$
5: Compute the difference, $\Delta = \log\mathcal{L}_{t+1}(D) - \log\mathcal{L}_t(D)$
6: **if** $\Delta > c$, where c is some threshold **then**
7: Classify $\mathbf{x}$ as an anomaly.
8: Increment t by one and use M_{t+1} and A_{t+1} in the next iteration.
9: **end if**
10: **end for**

Because the number of normal objects is large compared to the number of anomalies, the distribution of the normal objects may not change much when an object is moved to the set of anomalies. In that case, the contribution of each normal object to the overall likelihood of the normal objects will remain relatively constant. Furthermore, each object moved to the set of anomalies contributes a fixed amount to the likelihood of the anomalies. Thus, the overall change in the total likelihood of the data when an object is moved to the set of anomalies is roughly equal to the probability of the object under a uniform distribution (weighted by λ) minus the probability of the object under the distribution of the normal data objects (weighted by $1-\lambda$). Consequently, the set of anomalies will tend to consist of those objects that have significantly higher probability under a uniform distribution than under the distribution of the normal objects.

In the situation just discussed, the approach described by Algorithm 9.1 is roughly equivalent to classifying objects with a low probability under the distribution of normal objects as outliers. For example, when applied to the points in Figure 9.3, this technique would classify points A and B (and other points far from the mean) as outliers. However, if the distribution of the normal objects changes significantly as anomalies are removed or the distribution of the anomalies can be modeled in a more sophisticated manner, then the results produced by this approach will be different than the results of simply classifying low-probability objects as outliers. Also, this approach can work even when the distribution of normal objects is multi-modal, e.g., by using a mixture of Gaussian distributions for $f_M(\theta)$. Also, conceptually, it should be possible to use this approach with distributions other than Gaussian.

9.3.4 Assessing Statistical Significance

Statistical approaches provide a way to assign a measure of confidence for the instances detected as anomalies. For example, since the anomaly scores computed by statistical approaches have a probabilistic meaning, we can apply a threshold to these scores with statistical guarantees. Alternatively, it is possible to define statistical tests (also termed as **discordancy tests**) that can identify the statistical significance of an instance being identified as an anomaly by a statistical approach. Many of these discordancy tests are highly specialized and assume a level of statistical knowledge beyond the scope of this text. Thus, we illustrate the basic ideas with a simple example that uses univariate Gaussian distributions, and refer the reader to the Bibliographic Notes for further pointers.

Consider the Gaussian distribution $N(0, 1)$ shown in Figure 9.1. As discussed previously in Section 9.3.1, most of the probability density is centered around zero and there is little probability that an object (value) belonging to $N(0, 1)$ will occur in the tails of the distribution. For instance, there is only a probability of 0.0027 that an object lies beyond the central area between ± 3 standard deviations. More generally, if c is a constant and x is the attribute value of an object, then the probability that $|x| \geq c$ decreases rapidly as c increases. Let $\alpha = prob(|x| \geq c)$. Table 9.1 shows some sample values for c and the corresponding values for α when the distribution is $N(0, 1)$. Note that a value that is more than 4 standard deviations from the mean is a one-in-ten-thousand occurrence.

This interpretation of the distance of a point from the center can be used as the basis of a test to assess whether an object is an outlier, using the following definition.

Table 9.1. Sample pairs (c, α), $\alpha = prob(|x| \geq c)$ for a Gaussian distribution with mean 0 and standard deviation 1.

c	α for $N(0,1)$
1.00	0.3173
1.50	0.1336
2.00	0.0455
2.50	0.0124
3.00	0.0027
3.50	0.0005
4.00	0.0001

Definition 9.2 (Outlier for a Single *N(0,1)* Gaussian Attribute)**.** An object with attribute value x from a Gaussian distribution with mean of 0 and standard deviation 1 is an outlier if

$$|x| \geq c, \tag{9.7}$$

where c is a constant chosen so that $P(|x| \geq c) = \alpha$, where P represents probability.

To use this definition it is necessary to specify a value for α. From the viewpoint that unusual values (objects) indicate a value from a different distribution, α indicates the probability that we mistakenly classify a value from the given distribution as an outlier. From the viewpoint that an outlier is a rare value of a $N(0, 1)$ distribution, α specifies the degree of rareness.

More generally, for a Gaussian distribution with mean μ and standard deviation σ, we can first compute the z score of x and then apply the above test on x. In practice, this works well when μ and σ are estimated from a large population. A more sophisticated statistical procedure (Grubbs' test), which takes into account the distortion of parameter estimates caused by outliers, is explored in Exercise 7 on page 770.

The approach to outlier detection presented here is equivalent to testing data objects for statistical significance and classifying the statistically significant objects as anomalies. This is discussed in more detail in Chapter 10.

9.3.5 Strengths and Weaknesses

Statistical approaches to outlier detection have a firm theoretical foundation and build on standard statistical techniques. When there is sufficient knowledge of the data and the type of test that should be applied, these approaches are statistically justifiable and can be very effective. They can also provide

confidence intervals associated with the anomaly scores, which can be very helpful in making decisions about test instances, e.g., determining thresholds on the anomaly score.

However, if the wrong model is chosen, then a normal instance can be erroneously identified as an outlier. For example, the data may be modeled as coming from a Gaussian distribution, but may actually come from a distribution that has a higher probability (than the Gaussian distribution) of having values far from the mean. Statistical distributions with this type of behavior are common in practice and are known as **heavy-tailed distributions**. Also, we note that while there are a wide variety of statistical outlier tests for single attributes, far fewer options are available for multivariate data, and these tests can perform poorly for high-dimensional data.

9.4 Proximity-based Approaches

Proximity-based methods identify anomalies as those instances that are most distant from the other objects. This relies on the assumption that normal instances are related and hence appear *close* to each other, while anomalies are different from the other instances and hence are relatively far from other instances. Since many of the proximity-based techniques are based on distances, they are also referred to as **distance-based outlier detection techniques**.

Proximity-based approaches are *model-free* anomaly detection techniques, since they do not construct an explicit model of the normal class for computing the anomaly score. They make use of the local perspective of every data instance to compute its anomaly score. They are more general than statistical approaches, since it is often easier to determine a meaningful proximity measure for a data set than to determine its statistical distribution. In the following, we present some of the basic proximity-based approaches for defining an anomaly score. Primarily, these techniques differ in the way they analyze the locality of a data instance.

9.4.1 Distance-based Anomaly Score

One of the simplest ways to define a proximity-based anomaly score of a data instance $\mathbf{x}$ is to use the distance to its kth nearest neighbor, $dist(\mathbf{x}, k)$. If an instance $\mathbf{x}$ has many other instances located close to it (characteristic of the normal class), it will have a low value of $dist(\mathbf{x}, k)$. On other hand, an anomalous instance $\mathbf{x}$ will be quite distant from its k-neighboring instances and would thus have a high value of $dist(\mathbf{x}, k)$.

Figure 9.4 shows a set of points in a two-dimensional space that have been shaded according to their distance to the kth nearest neighbor, $dist(\mathbf{x}, k)$ (where $k = 5$). Note that point C has been correctly assigned a high anomaly score, as it is located far away from other instances.

Note that $dist(\mathbf{x}, k)$ can be quite sensitive to the value of k. If k is too small, e.g., 1, then a small number of outliers located close to each other can show a low anomaly score. For example, Figure 9.5 shows anomaly scores using $k = 1$ for a set of normal points and two outliers that are located close to each other (shading reflects anomaly scores). Note that both C and its neighbor have a low anomaly score. If k is too large, then it is possible for all objects in a cluster that has fewer than k objects to become anomalies. For example, Figure 9.6 shows a data set that has a small cluster of size 5 and a larger cluster of size 30. For $k = 5$, the anomaly score of all points in the smaller cluster is very high.

An alternative distance-based anomaly score that is more robust to the choice of k is the average distance to the first k-nearest neighbors, $avg.dist(\mathbf{x}, k)$. Indeed, $avg.dist(\mathbf{x}, k)$ is widely used in a number of applications as a reliable proximity-based anomaly score.

9.4.2 Density-based Anomaly Score

The density around an instance can be defined as $n/V(d)$, where n is the number of instances within a specified distance d from the instance, and $V(d)$ is the volume of the neighborhood. Since $V(d)$ is constant for a given d, the density around an instance is often represented using the number of instances n within a fixed distance d. This definition is similar to the one used by the DBSCAN clustering algorithm in Section 5.4. From a density-based viewpoint, anomalies are instances that are in regions of low density. Hence, an anomaly will have a smaller number of instances within a distance d than a normal instance.

Similar to the trade-off in choosing the parameter k in distance-based measures, it is challenging to choose the parameter d in density-based measures. If d is too small, then many normal instances can incorrectly show low density values. If d is too large, then many anomalies may have densities that are similar to normal instances.

Note that the distance-based and density-based views of proximity are quite similar to each other. To realize this, consider the k-nearest neighbors of a data instance $\mathbf{x}$, whose distance to the kth nearest neighbor is given by $dist(\mathbf{x}, k)$. In this approach, $dist(\mathbf{x}, k)$ provides a measure of the density around $\mathbf{x}$, using a different value of d for every instance. If $dist(\mathbf{x}, k)$ is large,

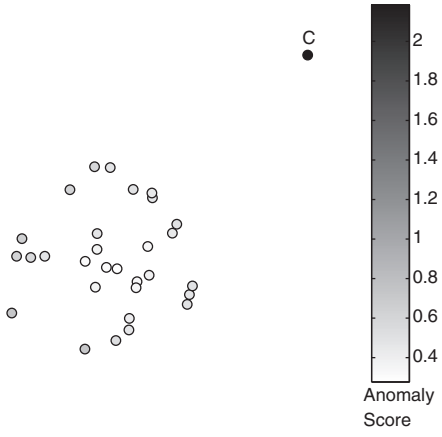


Figure 9.4. Anomaly score based on the distance to fifth nearest neighbor.

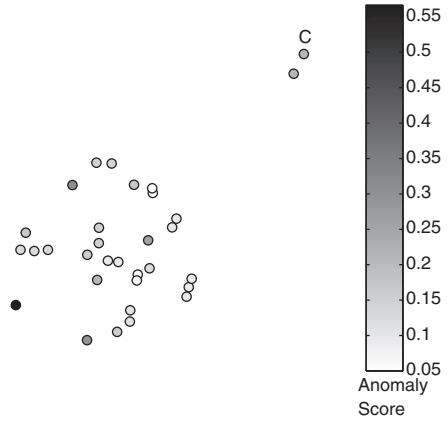


Figure 9.5. Anomaly score based on the distance to the first nearest neighbor. Nearby outliers have low anomaly scores.

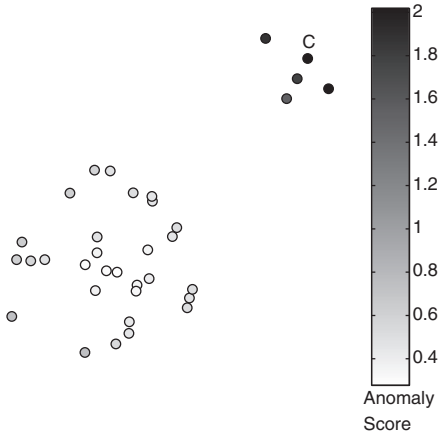


Figure 9.6. Anomaly score based on distance to the fifth nearest neighbor. A small cluster becomes an outlier.

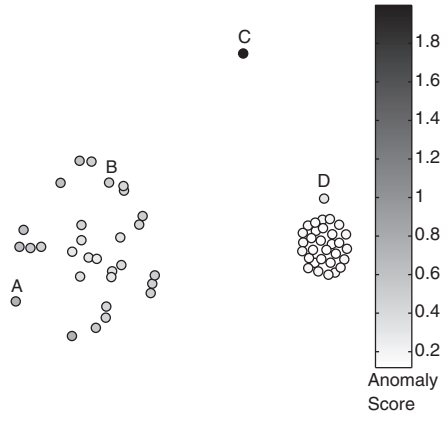


Figure 9.7. Anomaly score based on the distance to the fifth nearest neighbor, when there are clusters of varying densities.

the density around $\mathbf{x}$ is small, and vice-versa. Distance-based and density-based anomaly scores thus follow an inverse relationship. This can be used to define the following measures of density that are based on the two distance measures, $dist(\mathbf{x}, k)$ and $avg.dist(\mathbf{x}, k)$:

$$density(\mathbf{x}, k) = 1/dist(\mathbf{x}, k),$$
$$avg.density(\mathbf{x}, k) = 1/avg.dist(\mathbf{x}, k).$$

9.4.3 Relative Density-based Anomaly Score

The above proximity-based approaches only consider the locality of an individual instance for computing its anomaly score. In scenarios where the data contains regions of varying densities, such methods would not be able to correctly identify anomalies, as the notion of a *normal* locality would change across regions.

To illustrate this, consider the set of two-dimensional points in Figure 9.7. This figure has one rather loose cluster of points, another dense cluster of points, and two points, C and D, which are quite far from these two clusters. Assigning anomaly scores to points according to $dist(\mathbf{x}, k)$ with $k = 5$ correctly identifies point C to be an anomaly, but shows a low score for point D. In fact, the score for D is much lower than many points that are part of the loose cluster. To correctly identify anomalies in such data sets, we need a notion of density that is relative to the densities of neighboring instances. For example, point D in Figure 9.7 has a higher absolute density than point A, but its density is lower relative to its nearest neighbors.

There are many ways to define the relative density of an instance. For a point $\mathbf{x}$, One approach is to compute the ratio of the average density of its k-nearest neighbors, $\mathbf{y_1}$ to $\mathbf{y_k}$ to the density of $\mathbf{x}$, as follows:

$$relative\ density(\mathbf{x}, k) = \frac{\sum_{i=1}^{k} density(\mathbf{y_i}, k)/k}{density(\mathbf{x}, k)}. \tag{9.8}$$

The relative density of a point is high when the average density of points in its neighborhood is significantly higher than the density of the point.

Note that by replacing $density(\mathbf{x}, k)$ with $avg.density(\mathbf{x}, k)$ in the above equation, we can obtain a more robust measure of relative density. The above approach is similar to that used by the Local Outlier Factor (LOF) score, which is a widely-used measure for detecting anomalies using relative density. (See Bibliographic Notes.) However, LOF uses a somewhat different definition of density to achieve results that are more robust.

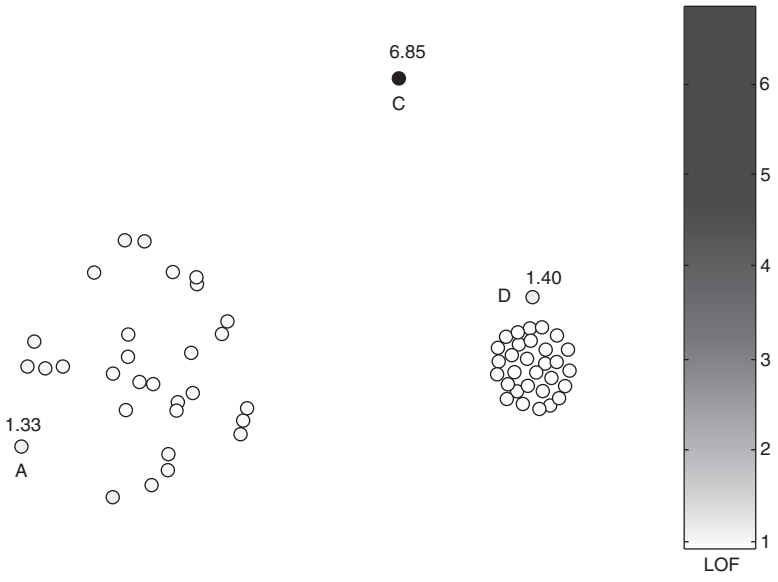


Figure 9.8. Relative density (LOF) outlier scores for two-dimensional points of Figure 9.7.

Example 9.2 (Relative Density Anomaly Detection)**.** Figure 9.8 shows the performance of the relative density-based anomaly detection method on the example data set used previously in Figure 9.7. The anomaly score of every point is computed using Equation 9.8 (with $k = 10$). The shading of every point represents its score, i.e., points with a higher score are darker. We have labeled points A, C, and D, which have the largest anomaly scores. Respectively, these points are the most extreme anomaly, the most extreme point with respect to the compact set of points, and the most extreme point in the loose set of points. ■

9.4.4 Strengths and Weaknesses

Proximity-based approaches are non-parametric in nature and hence are not restricted to any particular form of distribution of the normal and anomalous classes. They have a broad applicability over a wide range of anomaly detection problems where a reasonable proximity measure can be defined between instances. They are quite intuitive and visually appealing, since proximity-based anomalies can be interpreted visually when the data can be displayed in two- or three-dimensional scatter plots.

However, the effectiveness of proximity-based methods depends greatly on the choice of the distance measure. Defining distances in high-dimensional spaces can be challenging. In some cases, dimensionality reduction techniques can be used to map the instances into a lower dimensional feature space. Proximity-based methods can then be applied in the reduced space for detecting anomalies. Another challenge common to all proximity-based methods is their high computational complexity. Given n points, computing the anomaly score for every point requires considering all pair-wise distances, resulting in an $O(n^2)$ running time. For large data sets this can be too expensive, although specialized algorithms can be used to improve performance in some cases, e.g., with low-dimensional data sets. Choosing the right value of parameters (k or d) in proximity-based methods is also difficult and often requires domain expertise.

9.5 Clustering-based Approaches

Clustering-based methods for anomaly detection use clusters to represent the normal class. This relies on the assumption that normal instances appear close to each other and hence can be grouped into clusters. Anomalies are then identified as instances that do not fit well in the clustering of the normal class, or appear in small clusters that are far apart from the clusters of the normal class. Clustering-based methods can be categorized into two types: methods that consider small clusters as anomalies, and methods that define a point as anomalous if does not fit the clustering well, typically as measured by distance from a cluster center. We describe both types of clustering-based methods next.

9.5.1 Finding Anomalous Clusters

This approach assumes the presence of *clustered anomalies* in the data, where the anomalies appear in tight groups of small size. Clustered anomalies appear when the anomalies are being generated from the same anomalous class. For example, a network attack may have a common pattern in its occurrence, possibly because of a common attacker, who appears in similar ways in multiple instances.

Clusters of anomalies are generally small in size, since anomalies are rare in nature. They are also expected to be quite distant from the clusters of the normal class, since anomalies do not conform to normal patterns or behavior. Hence, a basic approach for detecting anomalous clusters is to cluster the

overall data and flag clusters that are either too small in size or too far from other clusters.

For instance, if we use a prototype-based method for clustering the overall data, e.g., using K-means, every cluster can be represented by its prototype, e.g., the centroid of the cluster. We can then treat every prototype as a point and straightforwardly identify clusters that are distant from the rest. As another example, if we are using hierarchical techniques such as MIN, MAX, or Group Average—see Section 5.3—then anomalies are often identified as those instances that are in small clusters or remain singletons even after almost all other points have been clustered.

9.5.2 Finding Anomalous Instances

From a clustering perspective, another way of describing an anomaly is as an instance that cannot be explained well by any of the normal clusters. Hence, a basic approach for anomaly detection is to first cluster all the data (comprised mainly of normal instances) and then assess the degree to which every instance belongs to its respective cluster. For example, if we use K-means clustering, the distance of an instance to its cluster centroid represents how strongly it belongs to the cluster. Instances that are quite distant from their respective cluster centroids can thus be identified as anomalies.

Although clustering-based methods for anomaly detection are quite intuitive and simple to use, there are a number of considerations that must be kept in mind while using them, as we discuss in the following.

Assessing the Extent to Which an Object Belongs to a Cluster

For prototype-based clusters, there are several ways to assess the extent to which an instance belongs to a cluster. One method is to measure the distance of an instance from its cluster prototype and consider this as the anomaly score of the instance. However, if the clusters are of differing densities, then we can construct an anomaly score that measures the relative distance of an instance from the cluster prototype with respect to the distances of the other instances in the cluster. Another possibility, provided that the clusters can be accurately modeled in terms of Gaussian distributions, is to use the Mahalanobis distance.

For clustering techniques that have an objective function, we can assign an anomaly score to an instance that reflects the improvement in the objective function when that instance is eliminated from the overall data. However, such an approach is often computationally intensive. For that reason, the distance-based approaches of the previous paragraph are usually preferred.

Example 9.3 (Clustering-Based Example)**.** This example is based on the set of points shown in Figure 9.7. Prototype-based clustering in this example uses the K-means algorithm, and the anomaly score of a point is computed in two ways: (1) by the point's distance from its closest centroid, and (2) by the point's relative distance from its closest centroid, where the relative distance is the ratio of the point's distance from the centroid to the median distance of all points in the cluster from the centroid. The latter approach is used to adjust for the large difference in density between compact and loose clusters.

The resulting anomaly scores are shown in Figures 9.9 and 9.10. As before, the anomaly score, measured in this case by the distance or relative distance, is indicated by the shading. We use two clusters in each case. The approach based on raw distance has problems with the differing densities of the clusters, e.g., D is not considered an outlier. For the approach based on relative distances, the points that have previously been identified as outliers using LOF (A, C, and D) also show up as anomalies here. ■

Impact of Outliers on the Initial Clustering

Clustering based schemes are often sensitive to the presence of outliers in the data. Hence, the presence of outliers can degrade the quality of clusters corresponding to the normal class since these clusters are discovered by clustering the overall data, which is comprised of normal and anomalous instances. To address this issue, the following approach can be used: instances are clustered, outliers, which are the points farthest from any cluster, are removed, and then the instances are clustered again. This approach is used at every iteration of the K-means algorithm. The K-means-- algorithm is an example of such an algorithm. While there is no guarantee that this approach will yield optimal results, it is easy to use.

A more sophisticated approach is to have a special group for instances that do not currently fit well in any cluster. This group represents potential outliers. As the clustering process proceeds, clusters change. Instances that no longer belong strongly to any cluster are added to the set of potential outliers, while instances currently in the outlier group are tested to see if they now strongly belong to a cluster and can be removed from the set of potential outliers. The instances remaining in the set at the end of the clustering are classified as outliers. Again, there is no guarantee of an optimal solution or even that this approach will work better than the simpler one described previously.

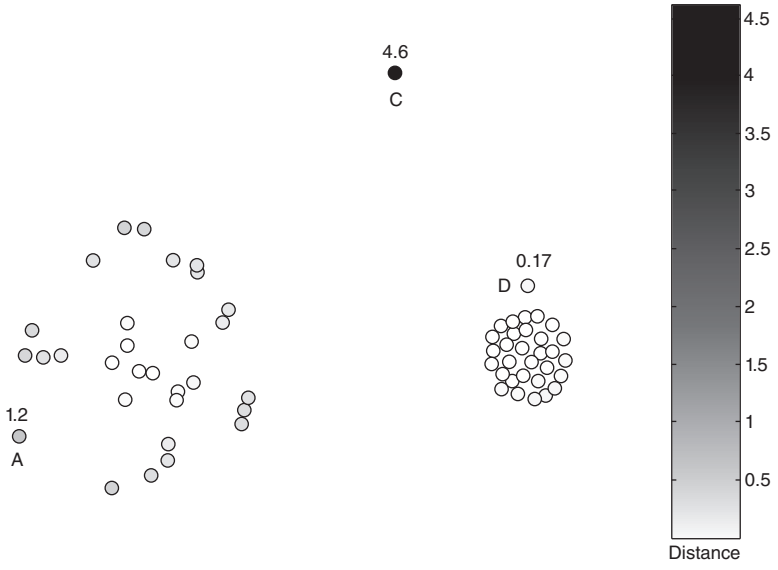


Figure 9.9. Distance of points from closest centroid.

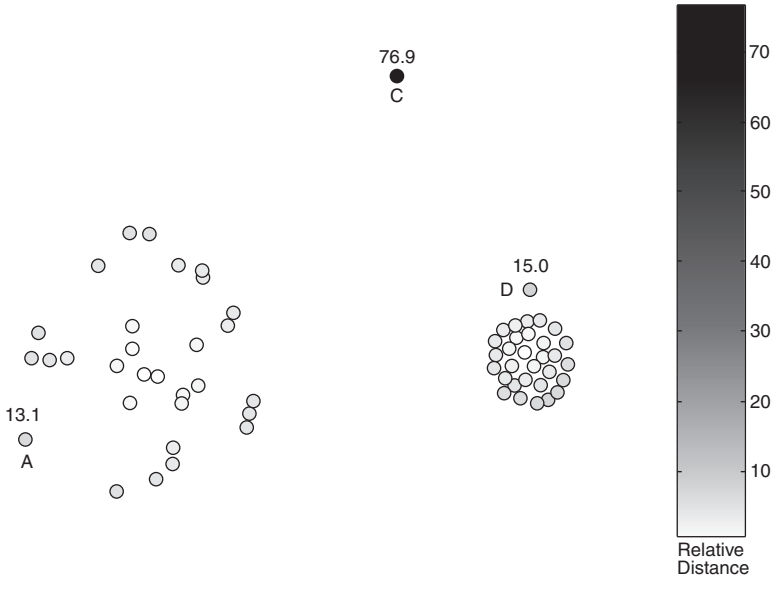


Figure 9.10. Relative distance of points from closest centroid.